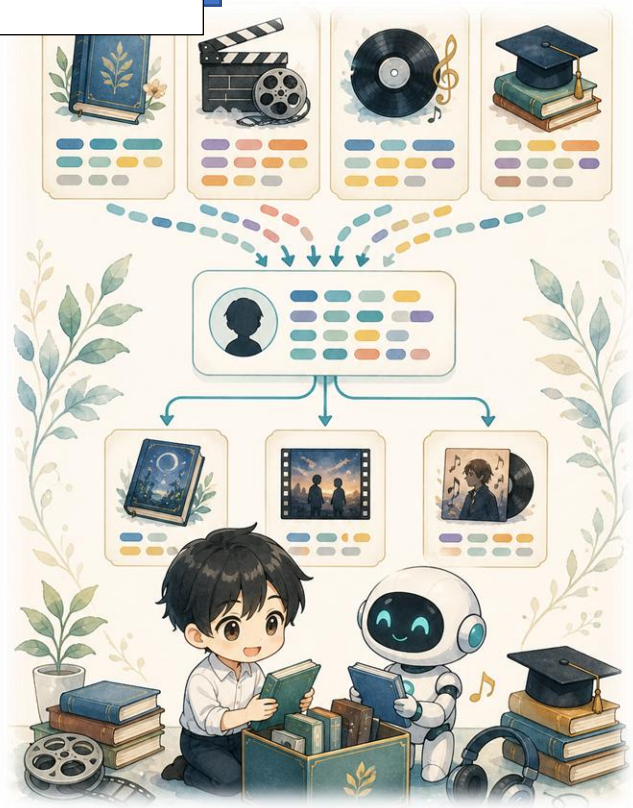
3

ระบบผู้แนะนำอิงเนื้อหา
(Content-Based Recommender System)



ระบบผู้แนะนำอิงเนื้อหา (Content-Based Recommender System) ตัดสินใจจากสิ่งที่รายการนั้น “เป็น” ไม่ใช่จากพฤติกรรมของผู้ใช้คนอื่น จึงแนะนำรายการใหม่ได้ตั้งแต่วันแรก และอธิบายเหตุผลของคำแนะนำได้ตรงไปตรงมา บทนี้จำกัดขอบเขตไว้ที่เนื้อหาที่เป็นข้อความ ได้แก่ ชื่อเรื่อง แนว และเรื่องย่อของภาพยนตร์

เนื้อหาเดินตามกระบวนการของระบบจริงทีละขั้น คือ ข้อความ ⟶ เวกเตอร์ของรายการ (เชิงคำ เชิงความหมายแบบสถิต และตามบริบท) ⟶ โปรไฟล์ผู้ใช้ ⟶ การจับคู่และจัดอันดับ ⟶ การค้นคืนในคลังขนาดใหญ่ การจัดอันดับซ้ำ ความหลากหลาย และการประเมินผล ทุกเทคนิคอธิบายด้วยตัวอย่างชุดเดียวกันคือ MovieMini-12 พร้อมการแทนค่าทีละขั้นที่ตรวจสอบด้วยมือได้

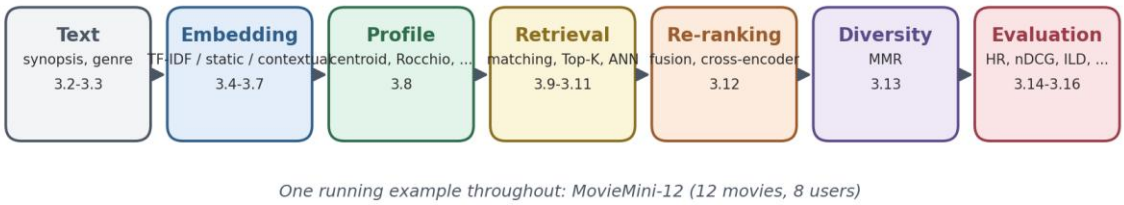
เนื้อหา

วัตถุประสงค์การเรียนรู้ 2
3.1 ระบบผู้แนะนำอิงเนื้อหาคืออะไร 2
3.2 ชุดข้อมูล MovieMini-12 4
3.3 การเตรียมข้อความ 7
3.4 เวกเตอร์เชิงคำ (Lexical) 9
3.5 เวกเตอร์ความหมายแบบสถิต 18
3.6 เวกเตอร์ความหมายตามบริบท 22
3.7 สรุปการสร้างเวกเตอร์ของรายการ 30
3.8 การสร้างโปรไฟล์ผู้ใช้ 32
3.9 การจับคู่โปรไฟล์กับรายการ 47
3.10 การจัดอันดับ Top-K 53
3.11 การค้นคืนในคลังขนาดใหญ่ 56
3.12 การผสานคะแนนและจัดอันดับซ้ำ 62
3.13 ความหลากหลายและ MMR 69
3.14 การประเมินผล 73
3.15 ข้อจำกัดและการวินิจฉัย 78
3.16 สรุปสายข้อมูลทั้งระบบ 80

วัตถุประสงค์การเรียนรู้ประจำบท

เมื่อศึกษาบทนี้จบแล้ว ผู้เรียนจะสามารถ

- **อธิบาย** สถาปัตยกรรมสามส่วนของระบบผู้แนะนำอิงเนื้อหา และกระบวนการตั้งแต่ข้อความ เวกเตอร์ การค้นคืน การจัดอันดับซ้ำ ความหลากหลาย ไปจนถึงการประเมินผล
- **สร้างเวกเตอร์ของรายการจากข้อความ** ได้สามแบบ คือ เวกเตอร์เชิงคำ (TF-IDF, BM25) เวกเตอร์ความหมายแบบสถิต (Static Embedding) และเวกเตอร์ความหมายตามบริบท (Contextual Embedding) พร้อมคำนวณตัวอย่างบน MovieMini-12 ด้วยมือ
- **อธิบาย** บทบาทของตัวเข้ารหัสคู่ (Bi-Encoder) ตัวเข้ารหัสไขว้ (Cross-Encoder) และหลักการฝึกด้วย InfoNCE ว่าเวกเตอร์ความหมายเรียนรู้จากคู่บวกและคู่ลบอย่างไร
- **สร้างโปรไฟล์ผู้ใช้** ในรูปแบบต่าง ๆ ได้แก่ ค่าเฉลี่ยศูนย์กลาง การถ่วงน้ำหนักด้วยคะแนน วิธีร็อกคีโย การลดทอนตามเวลา โปรไฟล์หลายกลุ่มความสนใจ และโปรไฟล์แบบเรียนรู้ได้ แล้วเลือกใช้ให้เหมาะกับข้อมูลที่มี
- **คำนวณคะแนนจับคู่** ระหว่างโปรไฟล์กับรายการ อธิบายได้ว่ามิติใดทำให้คะแนนสูง และจัดอันดับเป็นรายการแนะนำ Top-K พร้อมตรวจสอบด้วยการกันออกทีละรายการ
- **อธิบาย** เหตุผลที่ต้องใช้ดัชนีเพื่อนบ้านใกล้สุดโดยประมาณ (ANN) แนวคิดของ HNSW และ IVF-PQ และความสัมพันธ์ระหว่าง Bi-Encoder กับสถาปัตยกรรมสองหอคอย (Two-Tower)
- **ผสานคะแนน** เชิงคำกับเชิงความหมาย จัดอันดับซ้ำด้วยตัวเข้ารหัสไขว้ และคุมความหลากหลายด้วย MMR พร้อมคำนวณตัวอย่างได้
- **ประเมิน** ระบบด้วยตัวชี้วัดความแม่นยำ ความหลากหลาย ความครอบคลุม และความแปลกใหม่ ระบุข้อจำกัดของระบบอิงเนื้อหา และวินิจฉัยอาการผิดปกติจากตารางวินิจฉัย



ภาพที่ 3.1 กระบวนการของบทที่ 3: จากข้อความสู่ระบบแนะนำที่ใช้งานได้จริง และหัวข้อที่รับผิดชอบแต่ละขั้น

3.1 ระบบผู้แนะนำอิงเนื้อหาคืออะไร

3.1.1 ปัญหาที่แนวทางนี้เกิดมาเพื่อแก้

ลองนึกภาพแพลตฟอร์มภาพยนตร์แห่งหนึ่งที่มีภาพยนตร์ใหม่เข้าคลังทุกสัปดาห์ ภาพยนตร์ที่เพิ่งเข้ามายังไม่มีใครดู ไม่มีใครให้คะแนน ถ้าระบบแนะนำทำงานด้วยการกรองร่วม (Collaborative Filtering) ซึ่งเรียนรู้จากพฤติกรรมของผู้ใช้จำนวนมาก ระบบจะยังแนะนำภาพยนตร์เหล่านี้ให้ใครไม่ได้เลย เพราะยังไม่มีสัญญาณพฤติกรรมให้เรียน สภาวะเช่นนี้เรียกว่า **ปัญหาการเริ่มต้นเย็นของรายการ (Item Cold Start)**

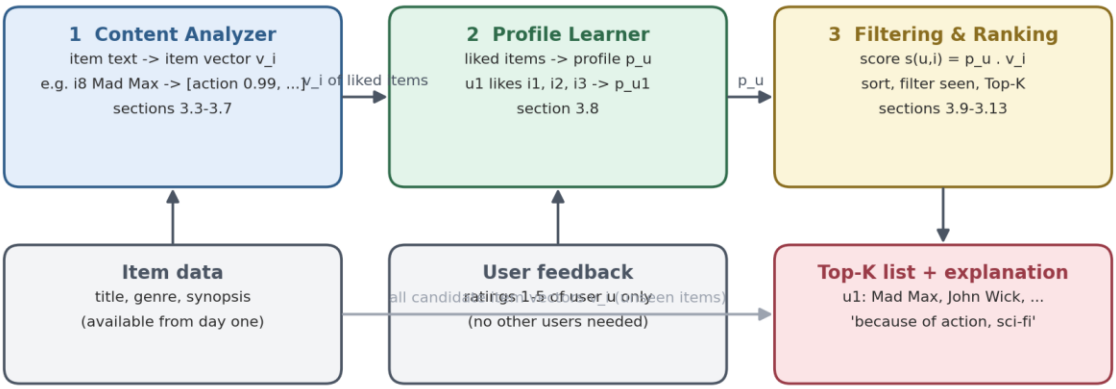
แนวทางอิงเนื้อหาแก้ปัญหานี้ได้ทันที เพราะภาพยนตร์ทุกเรื่องมี “เนื้อหา” ติดตัวมาตั้งแต่วินาทีแรก ได้แก่ ชื่อเรื่อง แนว (Genre) และเรื่องย่อ (Synopsis) ระบบจึงเปรียบเทียบเนื้อหาของภาพยนตร์ใหม่กับเนื้อหาของภาพยนตร์ที่ผู้ใช้เคยชอบได้โดยไม่ต้องรอผู้ใช้คนอื่นแม้แต่คนเดียว

> **นิยาม 3.1 ระบบผู้แนะนำอิงเนื้อหา (Content-Based Recommender System)**
>
> ระบบผู้แนะนำที่ประมาณค่าความชอบของผู้ใช้ u ต่อรายการ i จาก **ความคล้ายระหว่างการแทนเนื้อหาของรายการ i กับโปรไฟล์ของผู้ใช้ u** โดยโปรไฟล์นั้นสรุปขึ้นจากเนื้อหาของรายการที่ผู้ใช้ u เคยมีปฏิสัมพันธ์ด้วยเท่านั้น ระบบชนิดนี้ไม่ใช้ข้อมูลพฤติกรรมของผู้ใช้คนอื่นในการตัดสินใจ

วลีสุดท้ายของนิยาม “ไม่ใช้ข้อมูลพฤติกรรมของผู้ใช้คนอื่น” คือทั้งจุดแข็งและจุดอ่อนในประโยคเดียว จุดแข็งคือระบบทำงานได้แม้มีผู้ใช้เพียงคนเดียว และบอกได้ว่าแนะนำเพราะอะไร จุดอ่อนคือระบบไม่มีทางค้นพบสิ่งที่ผู้ใช้ควรจะชอบแต่ไม่มีเนื้อหาร่วมกับสิ่งที่เคยชอบมาก่อน ประเด็นนี้จะกลับมาอีกครั้งในหัวข้อ 3.15

3.1.2 สถาปัตยกรรมสามส่วน

ระบบผู้แนะนำอิงเนื้อหาทุกระบบ ไม่ว่าจะสร้างด้วยการนับคำหรือด้วยแบบจำลองภาษาขนาดใหญ่ ล้วนประกอบด้วยสามส่วนเดียวกัน (ภาพที่ 3.2)



ภาพที่ 3.2 สถาปัตยกรรมสามส่วนของระบบผู้แนะนำอิงเนื้อหา พร้อมตัวอย่างการไหลของข้อมูลของผู้ใช้ u1 บน MovieMini-12

ตารางที่ 3.1 สามส่วนของระบบ หน้าที่ ผลลัพธ์ และหัวข้อที่อธิบาย

ส่วน	หน้าที่	ผลลัพธ์	หัวข้อ
1 ส่วนวิเคราะห์เนื้อหา (Content Analyzer)	แปลงข้อความของรายการให้เป็นตัวเลขที่คำนวณได้	เวกเตอร์ของรายการ v_i	3.3–3.7
2 ส่วนเรียนรู้โปรไฟล์ (Profile Learner)	สรุปความชอบของผู้ใช้จากรายการที่เคยให้ผลป้อนกลับ	เวกเตอร์โปรไฟล์ p_uในปริภูมิเดียวกับ v_i	3.8

ส่วน	หน้าที่	ผลลัพธ์	หัวข้อ
3 ส่วนกรองและจัดอันดับ (Filtering)	ให้คะแนนจับคู่ทุกรายการที่ยังไม่เคยเห็น แล้วเลือก K อันดับแรก	รายการแนะนำ Top-K	3.9–3.13

> **ประเด็นสำคัญ คอขวดอยู่ที่ส่วนแรก ไม่ใช่ส่วนสุดท้าย**
>
> คุณภาพของทั้งระบบถูกจำกัดที่ส่วนวิเคราะห์เนื้อหา ถ้าเวกเตอร์ของรายการมองไม่เห็นว่า assassin กับ killers หมายถึงสิ่งเดียวกัน ส่วนที่สามก็ทำได้เพียง “เรียงลำดับสิ่งที่ส่วนแรกมองเห็น” ไม่ว่าจะเปลี่ยนฟังก์ชันจัดอันดับเป็นวิธีใดก็ไม่ช่วย บทนี้จึงใช้เนื้อที่เกือบครึ่งหนึ่ง (หัวข้อ 3.3–3.7) กับคำถามเดียวว่า **จะเปลี่ยนข้อความให้เป็นเวกเตอร์ที่ดีได้อย่างไร**

3.1.3 กระบวนการของบทนี้

บทนี้เรียงเนื้อหาตามกระบวนการของระบบจริง ซึ่งสรุปได้เป็นลูกโซ่หกขั้นตามภาพที่ 3.1

Text → Embedding → Retrieval → Re-ranking → Diversity → Evaluation

ตารางที่ 3.2 แผนที่ของบท: คำถามที่แต่ละช่วงตอบ

ช่วง	หัวข้อ	คำถามที่ตอบ	เทคนิคหลัก
ก ข้อมูลและการเตรียมข้อความ	3.2–3.3	ใช้ข้อมูลอะไร และทำข้อความให้พร้อมคำนวณอย่างไร	MovieMini-12 · การตัดคำ · คำหยุด
ข สร้างเวกเตอร์ของรายการ	3.4–3.7	จะแทนภาพยนตร์หนึ่งเรื่องด้วยตัวเลขอย่างไร	TF-IDF · BM25 · Static Embedding · Contextual Embedding · Bi-Encoder
ค สร้างโปรไฟล์ผู้ใช้	3.8	จะสรุปรสนิยมของผู้ใช้เป็นเวกเตอร์อย่างไร	Centroid · ถ่วงคะแนน · Rocchio · ลดทอนตามเวลา · หลายกลุ่ม · Naive Bayes
ง จับคู่และจัดอันดับ	3.9–3.10	จะให้คะแนนและเรียงรายการอย่างไร	โคไซน์ · Item-kNN · BM25 · Top-K · LOO
จ ระบบขนาดใหญ่และการจัดอันดับขั้นสูง	3.11–3.13	จะค้นเร็ว ผสานคะแนน จัดอันดับซ้ำ และคุมความหลากหลายอย่างไร	ANN · Two-Tower · Convex/RRF · Cross-Encoder · MMR
ฉ ประเมินและวินิจฉัย	3.14–3.16	จะรู้ได้อย่างไรว่าระบบดี และแก้ตรงไหนก่อน	HR · nDCG · ILD · Coverage · Novelty · ตารางวินิจฉัย

3.2 ชุดข้อมูลตัวอย่าง MovieMini-12

ทุกตัวเลขในบทนี้คำนวณจากคลังข้อมูลขนาดเล็กชุดเดียว เรียกว่า **MovieMini-12** ประกอบด้วยภาพยนตร์ 12 เรื่อง และคะแนนของผู้ใช้ 8 คน คลังนี้เล็กพอที่จะคำนวณด้วยมือได้ครบทุกช่อง

แต่ก็เล็กเกินไปสำหรับสถิติที่เชื่อถือได้ ผู้เรียนจึงควรมองตัวเลขในบทนี้เป็นเครื่องมือทำความเข้าใจกลไก ไม่ใช่ข้อสรุปว่าวิธีใดดีที่สุดในโลกจริง

3.2.1 ข้อมูลรายการ: ชื่อเรื่อง แนว และเรื่องย่อ

ภาพยนตร์แต่ละเรื่องมีสี่เขตข้อมูล ได้แก่ รหัส ชื่อเรื่อง แนว และเรื่องย่อ (ตารางที่ 3.3 และ 3.4) เขตข้อมูล **แนว** เป็นป้ายหมวดหมู่ที่มนุษย์ติดไว้ ส่วน **เรื่องย่อ** เป็นข้อความอิสระ บทนี้จะแสดงให้เห็นว่าทั้งสองเขตข้อมูลให้เวกเตอร์คนละแบบ และให้คำแนะนำที่ต่างกันได้

ตารางที่ 3.3 ข้อมูลบรรณานุกรมของคลัง MovieMini-12

รหัส	ชื่อเรื่อง	ปี	แนว (Genre)
i1	The Matrix	1999	Action, Sci-Fi
i2	Inception	2010	Sci-Fi, Thriller
i3	Interstellar	2014	Drama, Sci-Fi
i4	Titanic	1997	Drama, Romance
i5	La La Land	2016	Musical, Romance
i6	The Notebook	2004	Drama, Romance
i7	John Wick	2014	Action, Thriller
i8	Mad Max: Fury Road	2015	Action, Sci-Fi
i9	Parasite	2019	Drama, Thriller
i10	Toy Story	1995	Animation, Family
i11	Coco	2017	Animation, Family
i12	Spirited Away	2001	Animation, Fantasy

ตารางที่ 3.4 เรื่องย่อ (Synopsis) ฉบับเต็มของทั้ง 12 เรื่อง ซึ่งใช้คำนวณทุกตัวเลขในบทนี้

รหัส	เรื่องย่อ
i1	A hacker discovers that his reality is a simulated world built by machine intelligence, and he must choose between illusion and the harsh reality hidden beneath that virtual world, while rebels battle machine armies to free the real world from a false reality.
i2	A thief who steals secrets inside a shared dream accepts one final job: plant an idea in a rival heir dream within a dream, while his own dream of family, a fading memory and a lost dream of reality blur time and world.
i3	A widowed pilot leaves his daughter and travels through a wormhole into space, where relativity bends time near a black hole while a father fights to save humanity, and space travel costs him lost time in space.

รหัส	เรื่องย่อ
i4	A young woman of high class and a penniless artist fall in love aboard a doomed ocean liner, and their romance ends when the ship strikes an iceberg and sinks, leaving a survivor with a promise of love.
i5	An aspiring actress and a jazz pianist chase a dream in a bright city, falling in love while love and ambition pull them apart, until each finally reaches the career dream they wanted at the cost of the love they shared.
i6	An elderly man reads from a worn notebook to a woman whose memory is fading, retelling a summer of first love, unsent love letters, and a love that outlasts war and the theft of memory, so love may survive memory.
i7	A retired assassin returns to a criminal underworld after gangsters kill the dog left to him by his late wife, and he cuts a violent path of revenge through hotels, clubs, and rival killers of the night.
i8	In a scorched desert a lone drifter joins a rebel driver fleeing a tyrant war lord across dunes, and a road chase becomes a fight for water, freedom, and survival.
i9	A poor family schemes its way into the household of a wealthy family through deception, until a hidden secret beneath the mansion exposes the violent gap between one family and another family, and one household loses its home.
i10	A cowboy toy fears replacement when a flashy space ranger figure arrives, yet rivalry turns to friendship as the two toys work together to find their way home to a child bedroom.
i11	A boy who loves music crosses into the land of the dead at night, meets a musician, and learns that a family song can hold a fading memory alive, so that family memory outlives every family bond.
i12	A timid girl wanders into a spirit world where a witch steals her name, and she must labour in a strange bathhouse world, befriend a river dragon, and find the courage to free her family and return to the human world.

3.2.2 ข้อมูลผู้ใช้: คะแนนของผู้ใช้แปดคน

ครึ่งหลังของบทต้องใช้คะแนนของผู้ใช้เพื่อสร้างโปรไฟล์และวัดผล ตารางที่ 3.5 สรุปคะแนนของผู้ใช้ 8 คน โดยถือว่าคะแนน 4–5 คือชอบ คะแนน 1–2 คือไม่ชอบ และคะแนน 3 เป็นกลาง

ตารางที่ 3.5 คะแนนของผู้ใช้แปดคน (ชอบ = คะแนน 4–5, ไม่ชอบ = คะแนน 1–2)

ผู้ใช้	ชอบ (คะแนน)	ไม่ชอบ (คะแนน)	กลาง (3)	ยังไม่เคยให้คะแนน
u1	i1 (5), i2 (5), i3 (4)	i4 (1), i6 (1)	—	i5, i7, i8, i9, i10, i11, i12
u2	i4 (5), i5 (4), i6 (5)	i1 (2)	—	i2, i3, i7–i12
u3	i10 (5), i11 (5), i12 (4)	i7 (1)	—	i1–i6, i8, i9
u4	i7 (5), i8 (5), i1 (4)	i5 (1)	—	i2–i4, i6, i9–i12
u5	i9 (5), i3 (4)	i10 (2)	i6	i1, i2, i4, i5, i7, i8, i11, i12
u6	i2 (5), i5 (4)	—	i11	i1, i3, i4, i6–i10, i12

ผู้ใช้	ชอบ (คะแนน)	ไม่ชอบ (คะแนน)	กลาง (3)	ยังไม่เคยให้คะแนน
u7	i12 (5), i11 (4)	i8 (2)	i3	i1, i2, i4–i7, i9, i10
u8	i6 (5), i4 (4)	i2 (2)	i11	i1, i3, i5, i7–i10, i12

ผู้ใช้ที่ใช้เป็น **ตัวอย่างหลัก** ตลอดบทคือ u1 ซึ่งชอบ The Matrix, Inception และ Interstellar แต่ไม่ชอบ Titanic และ The Notebook คะแนนของ u1 ชี้ไปทางเดียวกันชัดเจน คือชอบภาพยนตร์ Sci-Fi ว่าด้วยความจริง ความฝัน และเวลา แต่ไม่ชอบเรื่องรักโรแมนติก งานของระบบคือเรียงลำดับ 7 เรื่องที่ u1 ยังไม่เคยดู (i5, i7, i8, i9, i10, i11, i12) ว่า u1 น่าจะชอบเรื่องใดมากที่สุด ผู้เรียนจึงมี “คำตอบที่มนุษย์คาดหวัง” เช่น Mad Max: Fury Road หรือ John Wick ไว้เทียบกับผลที่ระบบคำนวณได้ ซึ่งบางครั้งจะตรงกันและบางครั้งจะไม่ตรง และความไม่ตรงนั้นคือบทเรียนสำคัญของบทนี้

3.3 การเตรียมข้อความก่อนสร้างเวกเตอร์

คอมพิวเตอร์บวกลบตัวเลขได้ แต่ไม่รู้จักคำว่า dream ก่อนจะสร้างเวกเตอร์ใด ๆ ต้องเปลี่ยนเรื่องย่อซึ่งเป็นสายอักขระ ให้กลายเป็น **ลำดับของหน่วยคำ (Token)** ที่สะอาดเสียก่อน ขั้นนี้ดูเรียบง่ายแต่มีผลต่อคุณภาพของทั้งระบบมากกว่าไฮเพอร์พารามิเตอร์ใด ๆ เพราะทุกเวกเตอร์ในหัวข้อ 3.4–3.6 ล้วนสร้างจากโทเคนเหล่านี้

3.3.1 กระบวนงานหกขั้น

ตารางที่ 3.6 หกขั้นของการเตรียมข้อความ พร้อมผลที่เกิดกับเรื่องย่อของ i1 The Matrix

ขั้น	ทำอะไร	ตัวอย่างผลลัพธ์
1 ทำให้เป็นมาตรฐาน (Normalization)	เปลี่ยนเป็นตัวพิมพ์เล็ก ตัดเครื่องหมายวรรคตอนและตัวเลข	“A hacker discovers…” ⟶ “a hacker discovers…”
2 ตัดคำ (Tokenization)	แยกสายอักขระเป็นคำ	ได้ 43 โทเคน: a, hacker, discovers, that, his, reality, …
3 ตัดคำหยุด (Stopword Removal)	ลบคำไวยากรณ์ที่ไม่บอกเนื้อหา เช่น a, the, his, and	ตัดออก 18 โทเคน เหลือ 25 โทเคน
4 ลดรูปคำ (Stemming / Lemmatization)	รวมรูปผันของคำเดียวกัน เช่น discovers ⟶ discover	บทนี้ **ไม่ใช้** เพื่อให้ตรวจสอบตัวเลขด้วยมือได้ง่าย
5 สร้าง n-gram (ทางเลือก)	เก็บคู่คำที่ติดกัน เช่น machine_intelligence	บทนี้ใช้เฉพาะ unigram
6 สร้างคลังศัพท์ (Vocabulary)	รวบรวมคำที่ไม่ซ้ำจากทุกเอกสาร	ได้คลังศัพท์ 203 คำจาก 263 โทเคน

ตัวอย่างการคำนวณ 3.1 เตรียมข้อความของ i1 The Matrix ทีละขั้น

ขั้นที่ 1 — ข้อความดิบ

"A hacker discovers that his reality is a simulated world built by machine intelligence, and he must choose between illusion and the harsh reality hidden beneath that virtual world, while rebels battle machine armies to free the real world from a false reality."

ขั้นที่ 2 — ทำให้เป็นมาตรฐานและตัดคำ ได้ 43 โทเคน

a hacker discovers that his reality is a simulated world built by machine intelligence and he must choose between illusion and the harsh reality hidden beneath that virtual world while rebels battle machine armies to free the real world from a false reality

ขั้นที่ 3 — ตัดคำหยุด 18 โทเคน

คำที่ถูกตัด: a, that, his, is, a, by, and, he, must, between, and, the, that, while, to, the, from, a

ขั้นที่ 4 — ผลลัพธ์: ลำดับโทเคนที่สะอาด 25 โทเคน

hacker discovers reality simulated world built machine intelligence choose illusion harsh reality hidden beneath virtual world rebels battle machine armies free real world false reality

ขั้นที่ 5 — ตีความ

คำที่เหลือทั้ง 25 คำล้วนเป็นคำเนื้อหา (reality, machine, world, hacker, ...) และมีคำซ้ำอยู่สามคำ คือ reality 3 ครั้ง world 3 ครั้ง และ machine 2 ครั้ง จำนวนครั้งเหล่านี้คือวัตถุดิบของเวกเตอร์เชิงคำในหัวข้อ 3.4 สังเกตว่าถ้าไม่ตัดคำหยุด คำที่ปรากฏบ่อยที่สุดจะเป็น a, the และ world ซึ่งสองคำแรกไม่บอกอะไรเลยเกี่ยวกับภาพยนตร์

3.3.2 สถิติของคลังหลังเตรียมข้อความ

ตารางที่ 3.7 สถิติของคลัง MovieMini-12 ก่อนและหลังตัดคำหยุด

สถิติ	ก่อนตัดคำหยุด	หลังตัดคำหยุด
จำนวนเอกสาร N	12	12
จำนวนโทเคนรวม	457	263
ขนาดคลังศัพท์ $\|V\|$	249	203
ความยาวเฉลี่ยต่อเอกสาร $avgdl$	38.0833	21.9167

ตารางที่ 3.8 จำนวนโทเคนของแต่ละเรื่องหลังตัดคำหยุด (ใช้เป็นความยาวเอกสาร $|d|$ ใน BM25)

รหัส	i1	i2	i3	i4	i5	i6	i7	i8	i9	i10	i11	i12
|d|	25	25	23	22	22	23	20	19	20	21	21	22

3.3.3 ข้อควรรู้สำหรับภาษาไทย

ทุกขั้นข้างต้นใช้กับภาษาไทยได้ แต่ขั้นที่ 2 ยากกว่ามาก เพราะภาษาไทยไม่มีช่องว่างคั่นคำ การตัดคำต้องใช้พจนานุกรมหรือแบบจำลอง เช่น PyThaiNLP และผลการตัดคำที่ผิด เช่น "ตากลม" ที่อาจเป็น ตา-กลม หรือ ตาก-ลม จะส่งผลผิดต่อทุกเวกเตอร์ที่สร้างตามมา กฎเชิงปฏิบัติคือ **ให้สุ่มอ่านผลการตัดคำด้วยตาอย่างน้อย 100 ประโยคก่อนปรับพารามิเตอร์อื่นใด**

นอกจากนี้ควรทำให้เป็นมาตรฐานเรื่องสระและวรรณยุกต์ที่พิมพ์ต่างกัน และตัดคำหยุดภาษาไทย เช่น ที่ ซึ่ง อัน ของ ด้วยรายการคำหยุดชุดเดียวกันทั้งระบบ

> **ข้อควรระวัง การเตรียมข้อความทำลายสารสนเทศเสมอ ต้องระบุการตั้งค่าให้ครบ**
>
> การตัดคำหยุดลบคำว่า not ทิ้ง ทำให้ “not a love story” กลายเป็น “love story” การลดรูปคำอาจรวม universe กับ university เข้าด้วยกัน และการเปลี่ยนรายการคำหยุดเพียงคำเดียวก็เปลี่ยนตัวเลขทุกตัวในบทนี้ ดังนั้นการรายงานผลของระบบต้องระบุเสมอว่าใช้การตัดคำแบบใด รายการคำหยุดชุดใด และลดรูปคำหรือไม่ ตัวเลขทุกตัวในบทนี้ผูกกับเรื่องย่อในตารางที่ 3.4 และรายการคำหยุดชุดเดียว ผู้ที่นำไปคำนวณซ้ำด้วยการตั้งค่าเดียวกันจะได้ตัวเลขตรงกันทุกตำแหน่ง

3.4 เวกเตอร์เชิงคำ (Lexical Vector): TF-IDF และ BM25

เวกเตอร์แบบแรกและเก่าแก่ที่สุดคือ **เวกเตอร์เชิงคำ** ซึ่งใช้ “คำ” เป็นมิติโดยตรง ภาพยนตร์หนึ่งเรื่องถูกแทนด้วยตัวเลขที่บอกว่าแต่ละคำในคลังศัพท์สำคัญต่อเรื่องนั้นเพียงใด วิธีนี้ไม่ต้องฝึกแบบจำลอง คำนวณได้ในพริบตา และอธิบายได้ว่าเหตุใดสองเรื่องจึงคล้ายกัน (เพราะมีคำใดร่วมกัน) หัวข้อนี้สร้างเวกเตอร์ชนิดนี้ทีละขั้น จากการนับคำ การถ่วงน้ำหนัก ไปจนถึงการวัดความคล้าย

3.4.1 ถุงคำและเมทริกซ์เอกสาร–คำ

แบบจำลองถุงคำ (Bag of Words) มองเอกสารเป็นถุงที่ใส่คำไว้โดยไม่สนใจลำดับ “the dog bites the man” กับ “the man bites the dog” จึงเป็นถุงเดียวกัน สมมติฐานนี้ผิดอย่างเห็นได้ชัด แต่ใช้ได้ดีอย่างน่าประหลาดใจในงานวัดความคล้ายของหัวข้อ เพราะคำที่ปรากฏบอก “เรื่องนี้เกี่ยวกับอะไร” ได้มากแม้ไม่รู้ลำดับ

เมื่อนับจำนวนครั้งของทุกคำในทุกเอกสาร จะได้ **เมทริกซ์เอกสาร–คำ (Document–Term Matrix)** ขนาด $N \times |V|$ ซึ่งสำหรับ MovieMini-12 คือ 12 × 203 ตารางที่ 3.9 แสดงเฉพาะ 10 คอลัมน์ที่จะใช้บ่อยในบท ช่องที่เป็นศูนย์คือคำนั้นไม่ปรากฏในเรื่องนั้น สังเกตว่าแถวส่วนใหญ่เต็มไปด้วยศูนย์ เวกเตอร์เชิงคำจึงเป็น **เวกเตอร์เบาบาง (Sparse Vector)**

ตารางที่ 3.9 เมทริกซ์เอกสาร–คำ (จำนวนครั้ง $f(t,d)$) ของ MovieMini-12 แสดง 10 คำจาก 203 คำ พร้อมจำนวนเอกสารที่มีคำนั้น n_t

รหัส	reality	world	machine	dream	space	time	love	memory	family	music
i1	3	3	2	·	·	·	·	·	·	·
i2	1	1	·	5	·	1	·	1	1	·
i3	·	·	·	·	3	2	·	·	·	·
i4	·	·	·	·	·	·	2	·	·	·
i5	·	·	·	2	·	·	3	·	·	·
i6	·	·	·	·	·	·	4	3	·	·
i7	·	·	·	·	·	·	·	·	·	·
i8	·	·	·	·	·	·	·	·	·	·

รหัส	reality	world	machine	dream	space	time	love	memory	family	music
i9	·	·	·	·	·	·	·	·	4	·
i10	·	·	·	·	1	·	·	·	·	·
i11	·	·	·	·	·	·	·	2	3	1
i12	·	3	·	·	·	·	·	·	1	·
n_t	2	3	1	2	2	2	3	3	4	1

3.4.2 ความถี่ของคำ (TF): นับดิบยังไม่ใช่ความสำคัญ

จำนวนครั้ง $f(t,d)$ บอกว่าคำ t ปรากฏในเอกสาร d กี่ครั้ง แต่ “กี่ครั้ง” ยังไม่ใช่ “สำคัญเพียงใด” คำที่ปรากฏ 10 ครั้งไม่ได้สำคัญกว่าคำที่ปรากฏ 1 ครั้งถึง 10 เท่า จึงมีสูตร TF หลายรูปแบบให้เลือกตามสมมติฐานของงาน (ตารางที่ 3.10) บทนี้ใช้ **TF แบบลอการิทึม** เป็นค่าตั้งต้น เพราะกดคำที่ซ้ำมากให้โตช้าลง

ตารางที่ 3.10 รูปแบบ TF ที่พบบ่อย พร้อมค่าของคำ reality ใน i1 ($f = 3$, $|d| = 25$)

รูปแบบ	สูตร	ค่าที่ได้	สมมติฐาน
ดิบ (Raw)	$f(t,d)$	2	ปรากฏสองเท่า สำคัญสองเท่า
ทวิภาค (Binary)	1 ถ้า $f(t,d) > 0$	1	สนใจแค่ว่ามีหรือไม่มี
สัดส่วน (Normalized)	$f(t,d)/\|d\|$	0.1200	ชดเชยเอกสารยาว
ลอการิทึม (Log) — ใช้ในบทนี้	$1 + \log_2 f(t,d)$	2.5850	ความสำคัญโตช้ากว่าจำนวนครั้ง

ตารางที่ 3.11 แสดงว่า TF แบบลอการิทึมโตช้าเพียงใด คำที่ปรากฏ 10 ครั้งได้น้ำหนักเพียง 4.32 เท่าของคำที่ปรากฏครั้งเดียว ไม่ใช่ 10 เท่า

ตารางที่ 3.11 การอิ่มตัวของ TF แบบลอการิทึม $1 + \log_2 f$

$f(t,d)$	1	2	3	4	6	10
$1 + \log_2 f$	1.0000	2.0000	2.5850	3.0000	3.5850	4.3219

3.4.3 ความถี่ผกผันของเอกสาร (IDF): คำที่พบทุกที่ไม่ช่วยแยกอะไร

TF มองภายในเอกสารเดียว แต่คำว่า family ซึ่งปรากฏใน 4 เรื่องจาก 12 เรื่อง ย่อมช่วยแยกภาพยนตร์ได้น้อยกว่าคำว่า machine ที่ปรากฏเรื่องเดียว **IDF (Inverse Document Frequency)** จึงลดน้ำหนักคำที่พบในเอกสารจำนวนมาก บทนี้ใช้รูปแบบปรับเรียบ (Smoothed IDF) ซึ่งไม่มีวันเป็นศูนย์และไม่หารด้วยศูนย์

นิยาม 3.2 IDF แบบปรับเรียบและน้ำหนัก TF-IDF

$$idf(t) = \ln\frac{1+N}{1+n_t} + 1 \tag{3.1}$$

$$w_{t,d} = \left(1 + \log_2 f(t,d)\right) \times idf(t) \tag{3.2}$$

โดย N คือจำนวนเอกสารทั้งหมด และ n_t คือ **จำนวนเอกสารที่มีคำ t อย่างน้อยหนึ่งครั้ง** (ไม่ใช่จำนวนครั้งทั้งหมด ซึ่งเป็นจุดที่ผู้เริ่มต้นสับสนบ่อยที่สุด)

ตัวอย่างการคำนวณ 3.2 คำนวณ IDF ของคำสี่คำใน MovieMini-12 ($N = 12$)

ขั้นที่ 1 — นับ n_t จากตารางที่ 3.9

machine ปรากฏใน 1 เรื่อง · reality 2 เรื่อง · world 3 เรื่อง · family 4 เรื่อง

ขั้นที่ 2 — แทนค่าในสมการ (3.1)

$$idf(\text{machine}) = \ln\frac{13}{2} + 1 = 1.8718 + 1 = 2.8718$$

$$idf(\text{reality}) = \ln\frac{13}{3} + 1 = 1.4663 + 1 = 2.4663$$

$$idf(\text{world}) = \ln\frac{13}{4} + 1 = 1.1787 + 1 = 2.1787$$

$$idf(\text{family}) = \ln\frac{13}{5} + 1 = 1.9555$$

ขั้นที่ 3 — ตีความ

ยิ่งคำปรากฏในหลายเรื่อง ค่า IDF ยิ่งต่ำ family ซึ่งพบใน 4 เรื่องได้ 1.9555 ส่วน machine ซึ่งพบเรื่องเดียวได้ 2.8718 ค่า IDF ไม่ขึ้นกับว่าคำนั้นปรากฏกี่ครั้งในแต่ละเรื่อง แต่ขึ้นกับว่า “กี่เรื่อง” ที่มีคำนั้น

3.4.4 เวกเตอร์ TF-IDF และการปรับความยาวแบบ L2

นำ TF คูณ IDF ได้น้ำหนัก $w_{t,d}$ ของทุกคำ เรียงเป็นเวกเตอร์ 203 มิติ ขั้นสุดท้ายคือ **ปรับความยาวแบบ L2** ให้เวกเตอร์ยาวเท่ากับ 1 เพื่อให้เรื่องยาวกับสั้นเปรียบเทียบกันได้ และทำให้การคำนวณโคไซน์ในหัวข้อถัดไปเหลือเพียงผลคูณจุด

$$\hat{v}_d = \frac{w_d}{\|v_d\|_2}, \qquad \| v_d \|_2 = \sqrt{\sum_t w_{t,d}^2} \tag{3.3}$$

ตัวอย่างการคำนวณ 3.3 สร้างเวกเตอร์ TF-IDF ของ i1 The Matrix ครบทุกคำ

ขั้นที่ 1 — ข้อมูลเข้า: โทเคน 25 ตัวจากตัวอย่างการคำนวณ 3.1 นับได้ 20 คำที่ไม่ซ้ำกัน และ n_t ของแต่ละคำจากทั้งคลัง

ขั้นที่ 2 — คำนวณ TF, IDF, ผลคูณ และค่าหลังปรับ L2 ของทุกคำ (ตารางที่ 3.12)

ตัวอย่างแทนค่าของคำ reality: $tf = 1 + \log_2 3 = 2.5850$, $idf = 2.4663$, $w = 2.5850 \times 2.4663 = 6.3754$

ตารางที่ 3.12 น้ำหนัก TF-IDF ของทุกคำใน i1 The Matrix (Log TF × Smoothed IDF) และค่าหลังปรับความยาวแบบ L2

คำ t	$f(t,d)$	$1+\log_2 f$	n_t	$idf(t)$	$w_{t,d}$	หลัง L2
reality	3	2.5850	2	2.4663	6.3754	0.4123
machine	2	2.0000	1	2.8718	5.7436	0.3715
world	3	2.5850	3	2.1787	5.6317	0.3642
armies	1	1.0000	1	2.8718	2.8718	0.1857
battle	1	1.0000	1	2.8718	2.8718	0.1857
built	1	1.0000	1	2.8718	2.8718	0.1857
choose	1	1.0000	1	2.8718	2.8718	0.1857
discovers	1	1.0000	1	2.8718	2.8718	0.1857
false	1	1.0000	1	2.8718	2.8718	0.1857
hacker	1	1.0000	1	2.8718	2.8718	0.1857
harsh	1	1.0000	1	2.8718	2.8718	0.1857
illusion	1	1.0000	1	2.8718	2.8718	0.1857
intelligence	1	1.0000	1	2.8718	2.8718	0.1857
real	1	1.0000	1	2.8718	2.8718	0.1857
rebels	1	1.0000	1	2.8718	2.8718	0.1857
simulated	1	1.0000	1	2.8718	2.8718	0.1857
virtual	1	1.0000	1	2.8718	2.8718	0.1857
beneath	1	1.0000	2	2.4663	2.4663	0.1595
free	1	1.0000	2	2.4663	2.4663	0.1595
hidden	1	1.0000	2	2.4663	2.4663	0.1595

ขั้นที่ 3 — ความยาวของเวกเตอร์

$$\| v_{i1} \|_2 = \sqrt{6.3754^2 + 5.7436^2 + 5.6317^2 + 14 \times 2.8718^2 + 3 \times 2.4663^2}$$
$$= \sqrt{239.06} = 15.4616$$

ขั้นที่ 4 — หารทุกมิติด้วยความยาว

$$\hat{w}_{\text{reality},i1} = \frac{6.3754}{15.4616} = 0.4123$$

$$\hat{w}_{extmachine,i1} = \frac{5.7436}{15.4616} = 0.3715$$

ขั้นที่ 5 — ตรวจสอบและตีความ

ผลรวมกำลังสองของคอลัมน์สุดท้ายเท่ากับ 1.0000 พอดี แสดงว่าปรับความยาวถูกต้อง ข้อสังเกตสำคัญคือ machine ปรากฏเพียง 2 ครั้ง แต่ได้น้ำหนัก 0.3715 สูงกว่า world ที่ปรากฏ 3 ครั้ง (0.3642) เพราะ machine พบในเรื่องเดียวทั้งคลัง ($n_t = 1$) ขณะที่ world พบถึง 3 เรื่อง นี่คือ IDF ที่กำลังทำงาน มิติอีก 183 มิติที่ไม่ปรากฏในตารางมีค่าเป็นศูนย์ทั้งหมด

ตารางที่ 3.13 สรุปห้าคำที่มีน้ำหนักสูงสุดของทุกเรื่อง คำเหล่านี้คือ “ลายนิ้วมือ” ของแต่ละเรื่องในสายตาของเวกเตอร์เชิงคำ และจะถูกอ้างถึงซ้ำเมื่ออธิบายคะแนนจับคู่ในหัวข้อ 3.9

ตารางที่ 3.13 ห้าคำที่มีน้ำหนัก TF-IDF (หลัง L2) สูงสุดของแต่ละเรื่อง

รหัส	ชื่อเรื่อง	คำเด่น (น้ำหนัก)
i1	The Matrix	reality (0.4123) · machine (0.3715) · world (0.3642) · armies (0.1857) · false (0.1857)
i2	Inception	dream (0.5734) · accepts (0.2010) · final (0.2010) · blur (0.2010) · inside (0.2010)
i3	Interstellar	space (0.4386) · time (0.3394) · black (0.1976) · bends (0.1976) · fights (0.1976)
i4	Titanic	love (0.3232) · aboard (0.2130) · class (0.2130) · artist (0.2130) · ends (0.2130)
i5	La La Land	love (0.4065) · dream (0.3561) · apart (0.2073) · actress (0.2073) · career (0.2073)
i6	The Notebook	love (0.4638) · memory (0.3997) · elderly (0.2038) · outlasts (0.2038) · man (0.2038)
i7	John Wick	assassin (0.2281) · clubs (0.2281) · cuts (0.2281) · criminal (0.2281) · dog (0.2281)
i8	Mad Max: Fury Road	becomes (0.2327) · dunes (0.2327) · desert (0.2327) · fight (0.2327) · driver (0.2327)
i9	Parasite	family (0.4473) · household (0.4379) · exposes (0.2189) · deception (0.2189) · loses (0.2189)
i10	Toy Story	arrives (0.2239) · bedroom (0.2239) · child (0.2239) · cowboy (0.2239) · figure (0.2239)
i11	Coco	family (0.3868) · memory (0.3334) · bond (0.2197) · alive (0.2197) · dead (0.2197)
i12	Spirited Away	world (0.4227) · befriend (0.2156) · dragon (0.2156) · bathhouse (0.2156) · labour (0.2156)

3.4.5 การวัดความคล้ายด้วยโคไซน์

มีเวกเตอร์แล้วต้องวัดว่าสองเรื่องคล้ายกันเพียงใด ระยะทางแบบยุคลิดใช้ไม่ได้ดีกับข้อความ

เพราะเอกสารยาวมีเวกเตอร์ยาว ระยะทางจึงถูกครอบงำด้วยความยาว: ก่อนปรับ L2 ระยะทางระหว่าง i1 กับ i2

เท่ากับ 19.6782 และระหว่าง i1 กับ i12 เท่ากับ 18.4651 ซึ่งใกล้กันมากจนแทบไม่บอกอะไร **ความคล้ายเชิงโคไซน์ (Cosine Similarity)** วัดมุมระหว่างเวกเตอร์แทนความยาว จึงถามว่า “สัดส่วนของคำเหมือนกันหรือไม่” ไม่ใช่ “ยาวเท่ากันหรือไม่”

นิยาม 3.3 ความคล้ายเชิงโคไซน์

$$\cos(v_a, v_b) = \frac{v_a \cdot v_b}{\|v_a\|_2 \, \|v_b\|_2} = \sum_t \hat{w}_{t,a} \; \hat{w}_{t,b} \tag{3.4}$$

เครื่องหมายเท่ากับตัวหลังเป็นจริงเมื่อทั้งสองเวกเตอร์ปรับความยาวแบบ L2 แล้ว โคไซน์จึงเหลือเพียง **ผลรวมของผลคูณรายมิติ** และมีเพียงมิติที่ทั้งสองฝ่ายมีค่าไม่เป็นศูนย์ (คำร่วม) เท่านั้นที่มีผล ค่าอยู่ในช่วง 0 ถึง 1 สำหรับเวกเตอร์ TF-IDF ค่า 0 หมายถึงไม่มีคำร่วมกันเลย

ตัวอย่างการคำนวณ 3.4 ความคล้ายระหว่าง i2 Inception กับ i5 La La Land

ขั้นที่ 1 — ข้อมูลเข้า: หาคำที่ปรากฏในทั้งสองเรื่อง

ทั้งสองเรื่องมีคำร่วมกันเพียง 2 คำ คือ shared และ dream (ตารางที่ 3.14) คำอื่น ๆ อีกกว่า 30 คำของทั้งสองเรื่องไม่มีผลต่อโคไซน์เลย

ตารางที่ 3.14 คำร่วมของ i2 กับ i5 และผลคูณรายมิติ

คำร่วม	f ใน i2	f ใน i5	$\hat{w}$ ใน i2	$\hat{w}$ ใน i5	ผลคูณ
shared	1	1	0.1726	0.1780	0.0307
dream	5	2	0.5734	0.3561	0.2042
รวม					0.2349

ค่า $\hat{W}$ใน i5 คือ น้ำหนัก TF-IDF หลังปรับความยาวเวกเตอร์แบบ L2 โดยคำนวณจาก

$$\hat{w}_{t,i5} = \frac{w_{t,i5}}{\|\mathbf{w}_{i5}\|_2}$$

และ

$$w_{t,i5} = \left(1 + \log_2 f_{t,i5}\right)\left[\ln\left(\frac{N+1}{df_t + 1}\right) + 1\right]$$

ในชุดข้อมูลนี้ $N = 12$ และความยาวเวกเตอร์ TF-IDF ของ i5 ซึ่งรวม ทุกคำใน i5 คือ

$$\|\mathbf{w}_{i5}\|_2 = \sqrt{\sum_t w_{t,i5}^2} = 13.8536$$

คำ shared

$$f_{\text{shared},i5} = 1, \qquad df_{\text{shared}} = 2$$

$$TF = 1 + \log_2(1) = 1$$

$$IDF = \ln\left(\frac{12+1}{2+1}\right) + 1 = 2.4663$$

$$\hat{w}_{\text{shared},i5} = \frac{1 \times 2.4663}{13.8536} = 0.178028 \approx \boxed{0.1780}$$

คำ dream

$$f_{\text{dream},i5} = 2, \qquad df_{\text{dream}} = 2$$

$$TF = 1 + \log_2(2) = 2$$

$$\hat{w}_{\text{dream},i5} = \frac{2 \times 2.4663}{13.8536} = 0.356057 \approx \boxed{0.3561}$$

จุดสำคัญคือ ตัวหาร 13.8536 คำนวณจากน้ำหนักของ ทุกคำใน i5 ไม่ใช่เฉพาะ shared และ dream ที่แสดงในตาราง 3.14 จากนั้นจึงนำไปหาผลคูณกับ i2

ขั้นที่ 2 — รวมผลคูณ

$$\cos(i2, i5) = 0.0307 + 0.2042 = 0.2349$$

ขั้นที่ 3 — ตีความ

ค่า 0.2349 เป็นความคล้ายที่ **สูงที่สุดในคลัง** (ตารางที่ 3.15) และเกือบทั้งหมดมาจากคำเดียวคือ dream ซึ่งใน Inception หมายถึงความฝันขณะหลับ แต่ใน La La Land หมายถึงความใฝ่ฝันในอาชีพ เวกเตอร์เชิงคำมองไม่เห็นความต่างนี้ เพราะ “คำเดียวกันคือมิติเดียวกัน” เสมอ

ตารางที่ 3.15 หกคู่ที่มีความคล้ายเชิงโคไซน์สูงสุดในปริภูมิ TF-IDF จากทั้งหมด 66 คู่ (มี 42 คู่ที่ได้ศูนย์พอดี)

อันดับ	คู่ภาพยนตร์	โคไซน์	คำร่วมที่มีผล
1	i2 Inception – i5 La La Land	0.2349	dream, shared
2	i5 La La Land – i6 The Notebook	0.1886	love
3	i1 The Matrix – i12 Spirited Away	0.1835	world, free
4	i4 Titanic – i6 The Notebook	0.1819	love, woman
5	i9 Parasite – i11 Coco	0.1730	family
6	i6 The Notebook – i11 Coco	0.1590	memory, fading

ตารางนี้บอกเรื่องสำคัญสองเรื่อง หนึ่ง คู่ที่มนุษย์เห็นว่าคล้ายกันอย่าง John Wick กับ Mad Max (แนวบู๊ทั้งคู่) หรือ Toy Story กับ Coco (แอนิเมชันครอบครัวทั้งคู่) ได้ศูนย์ เพราะไม่มีคำร่วมกันแม้แต่คำเดียว สองคู่ที่ได้คะแนนสูงสุดกลับเป็นคู่ที่มนุษย์ไม่คิดว่าคล้าย จากทั้งหมด 66 คู่ มีถึง 42 คู่ที่ได้ศูนย์พอดี นี่คือเพดานของการจับคู่คำที่ตรงกัน ซึ่งหัวข้อ 3.5 และ 3.6 จะแก้

3.4.6 BM25: การให้คะแนนแบบอิ่มตัวและปรับความยาวเอกสาร

การจัดอันดับความคล้ายด้วย Cosine จาก TF-IDF มีจุดอ่อนสองข้อ ข้อแรก TF แบบลอการิทึมยังโตได้เรื่อย ๆ ไม่มีเพดาน ข้อสอง การปรับ L2 ทำให้เอกสารยาวเสียเปรียบโดยไม่มีเหตุผล BM25 เป็นฟังก์ชันให้คะแนนเชิงคำที่ระบบค้นคืนเชิงพาณิชย์แทบทุกระบบใช้เป็นระบบฐาน แก้ทั้งสองข้อด้วยพารามิเตอร์ k_{1} (คุมการอิ่มตัว) และ b (คุมการปรับความยาว) BM25 ให้คะแนนแก่คู่ (ข้อคำถาม q, เอกสาร d) โดยตรง จึงมองว่าเป็น ฟังก์ชันจับคู่ที่รับข้อคำถามซึ่งอาจเป็นคำเด่นของโปรไฟล์ผู้ใช้ (หัวข้อ 3.8.8) โดย BM25 วัดความเกี่ยวข้องระหว่างข้อคำถามกับเอกสาร

นิยาม 3.4 คะแนน BM25

$$BM25(q,d) = \sum_{t \in q} i\,df_{BM25}(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d)+k_1\left(1-b+b\frac{|d|}{avgdl}\right)} \tag{3.5}$$

$$idf_{BM25}(t) = \ln\left(\frac{N-n_t+0.5}{n_t+0.5}+1\right) \tag{3.6}$$

โดย $|d|$ คือความยาวของเอกสาร (จำนวนโทเคนหลังตัดคำหยุด) $avgdl$ คือความยาวเฉลี่ยของคลัง ค่าตั้งต้นที่นิยมคือ $k_1 = 1.5$ และ $b = 0.75$

ตารางที่ 3.16 การอิ่มตัวของ BM25 เทียบกับ TF แบบลอการิทึม (เมื่อ $|d| = avgdl$, $k_1 = 1.5$, $b = 0.75$)

$f(t,d)$	1	2	3	4	6	10
ส่วน TF ของ BM25	1.0000	1.4286	1.6667	1.8182	2.0000	2.1739
$1+\log_2 f$	1.0000	2.0000	2.5850	3.0000	3.5850	4.3219

ส่วน TF ของ BM25 มีเพดานที่ $k_1 + 1 = 2.5$ คำที่ปรากฏ 10 ครั้งได้ 2.1739 ใกล้เพดานแล้ว ขณะที่ TF แบบลอการิทึมยังโตต่อไปถึง 4.3219 ในทางปฏิบัติแปลว่า BM25 ไม่ยอมให้เรื่องย่อที่พูดคำเดียวซ้ำ ๆ ได้คะแนนสูงเกินไป

ตัวอย่างการคำนวณ 3.5 คะแนน BM25 ของข้อคำถาม {dream, reality} ต่อ i2 Inception และ i5 La La Land

ขั้นที่ 1 — ข้อมูลเข้า

ตารางที่ 3.17 ข้อมูลเข้าของ BM25: ความถี่ ความยาวเอกสาร และ IDF แบบ BM25 ($N = 12$, $avgdl = 21.9167$)

เอกสาร	$\|d\|$	$f(\text{dream},d)$	$f(\text{reality},d)$	$idf_{BM25}(\text{dream})$	$idf_{BM25}(\text{reality})$
i2 Inception	25	5	1	1.6487	1.6487
i5 La La Land	22	2	0	1.6487	1.6487
i1 The Matrix	25	0	3	1.6487	1.6487

ขั้นที่ 2 — IDF แบบ BM25 (ทั้งสองคำมี $n_t = 2$)

$$idf_{BM25}(\text{dream}) = \ln\left(\frac{12-2+0.5}{2+0.5}+1\right) = \ln(5.2) = 1.6487$$

ขั้นที่ 3 — ตัวปรับความยาวของ i2 ($|d| = 25$)

$$1 - b + b\,\frac{|d|}{avgdl} = 1 - 0.75 + 0.75 \times \frac{25}{21.9167} = 1.1055$$

ขั้นที่ 4 — คะแนนรายคำของ i2 แล้วรวม

$$\text{dream: } 1.6487 \times \frac{5\times 2.5}{5+1.5\times 1.1055} = 3.0951$$

$$\text{reality: } 1.6487 \times \frac{1 \times 2.5}{1 + 1.5 \times 1.1055} = 1.5505$$

$$BM25(q, i2) = 3.0951 + 1.5505 = 4.6456$$

ขั้นที่ 5 — ทำเช่นเดียวกันกับทุกเรื่อง

ตารางที่ 3.18 คะแนน BM25 ของข้อคำถาม {dream, reality} ต่อเรื่องที่ได้คะแนนไม่เป็นศูนย์

เอกสาร	$\|d\|$	คะแนนจาก dream	คะแนนจาก reality	BM25 รวม	อันดับ
i2 Inception	25	3.0951	1.5505	4.6456	1
i1 The Matrix	25	0	2.6544	2.6544	2
i5 La La Land	22	2.3524	0	2.3524	3
อีก 9 เรื่อง	—	0	0	0.0000	—

ขั้นที่ 6 — ตีความ

Inception มี dream ถึง 5 ครั้ง แต่ได้คะแนนจากคำนี้เพียง 3.0951 ไม่ใช่ 5 เท่าของ La La Land ที่มี 2 ครั้ง (2.3524) เพราะการอิ่มตัวทำงาน และ i2 ยาวกว่าค่าเฉลี่ยจึงถูกปรับลงเล็กน้อยด้วยตัวปรับความยาว 1.1055 คะแนน BM25 ไม่ได้อยู่ในช่วง 0–1 เหมือนโคไซน์ จึงเปรียบเทียบข้ามระบบตรง ๆ ไม่ได้ ต้องปรับมาตราก่อน (หัวข้อ 3.12)

3.4.7 ตัวเลือกที่ต้องตัดสินใจเมื่อสร้างเวกเตอร์เชิงคำ

ตารางที่ 3.19 สิ่งที่ต้องตัดสินใจเมื่อสร้างเวกเตอร์เชิงคำของรายการ และค่าที่แนะนำ

ประเด็น	ทางเลือก	ผลที่เกิด	ที่แนะนำ
เขตข้อมูลที่ใช้	เรื่องย่ออย่างเดียว / รวมชื่อเรื่องและแนว / ถ่วงน้ำหนักรายเขต	เขตข้อมูลสั้นแต่สำคัญ (ชื่อเรื่อง แนว) จะถูกเรื่องย่อกลบถ้ารวมตรง ๆ	ต่อข้อความทุกเขต แล้วซ้ำชื่อเรื่องและแนว 2–3 ครั้ง หรือใช้ BM25F

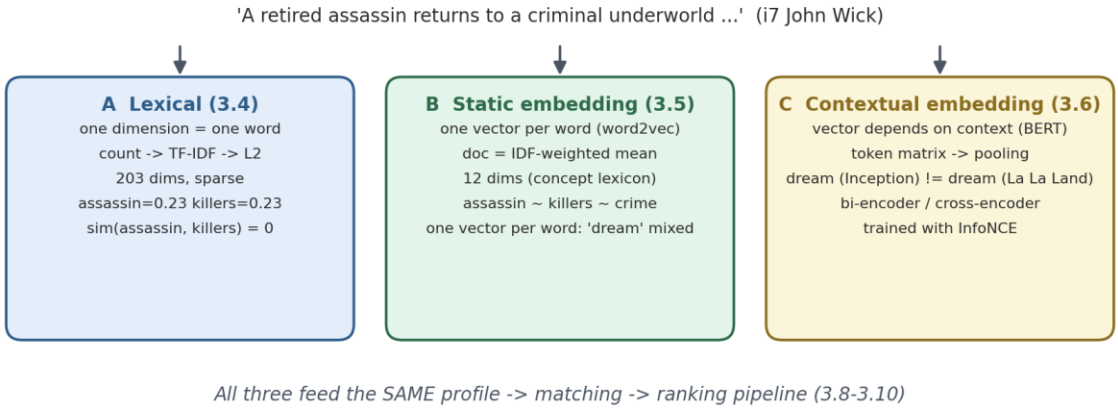
ประเด็น	ทางเลือก	ผลที่เกิด	ที่แนะนำ
คลังศัพท์	min_df, max_df, max_features	คำที่พบครั้งเดียวสร้างมิติที่ไม่มีวันจับคู่ได้ คำที่พบเกือบทุกเรื่องคือคำหยุดแฝง	min_df = 2, max_df = 0.85 ในคลังจริง (คลังเล็กในบทนี้ใช้ทุกคำ)
หน่วยคำ	unigram / unigram + bigram	bigram จับวลี เช่น space ranger แต่ทำให้คลังศัพท์โตหลายเท่า	เริ่มจาก unigram แล้วเพิ่ม bigram เมื่อวัดผลแล้วดีขึ้น
การถ่วงน้ำหนัก	Raw / Log TF × IDF / BM25	เปลี่ยนอันดับของคำเด่นและอันดับของเอกสาร	Log TF × Smoothed IDF สำหรับเวกเตอร์ BM25 สำหรับข้อคำถาม
การปรับความยาว	ไม่ปรับ / L2	ไม่ปรับ ⟶ เอกสารยาวชนะเสมอ	ปรับ L2 เสมอก่อนใช้โคไซน์หรือผลคูณจุด

ประเด็นสำคัญ สิ่งที่เวกเตอร์เชิงคำทำได้และทำไม่ได้

ทำได้ จับสายอักขระที่ต้องตรงเป๊ะ (ชื่อคน ชื่อรุ่น รหัสสินค้า) ไม่ต้องฝึก คำนวณเร็ว และอธิบายผลได้เป็นรายคำ **ทำไม่ได้** มองเห็นว่า assassin กับ killers คือเรื่องเดียวกัน (คำพ้องความหมาย Synonymy) และแยกว่า dream สองแห่งหมายถึงคนละอย่าง (คำพ้องรูป Polysemy) หัวข้อถัดไปเปลี่ยนจาก “คำเป็นมิติ” ไปเป็น “ความหมายเป็นมิติ” เพื่อแก้ข้อแรก และหัวข้อ 3.6 แก้ข้อที่สอง

3.5 เวกเตอร์ความหมายแบบสถิต (Static Embedding)

หัวข้อ 3.4 จบลงที่เพดานของการนับคำ: ถ้าสองเรื่องไม่มีคำร่วมกัน ความคล้ายเป็นศูนย์เสมอ John Wick กับ Mad Max จึงได้ 0.0000 ทั้งที่เป็นแนวบู๊ทั้งคู่ ทางแก้คือเลิกใช้ “คำ” เป็นมิติ แล้วใช้ “ความหมาย” เป็นมิติแทน **เวกเตอร์ฝัง (Embedding)** คือเวกเตอร์หนาแน่นขนาดคงที่ ที่ถูกฝึกจากคลังข้อความขนาดมหาศาล ให้คำที่ใช้ในบริบทคล้ายกันมีเวกเตอร์ใกล้กัน แบบแรกที่จะเรียนคือแบบ **สถิต (Static)** ได้แก่ word2vec, GloVe และ fastText ซึ่งให้ **หนึ่งเวกเตอร์ต่อหนึ่งคำ** ไม่ว่าคำนั้นจะอยู่ในประโยคใด



ภาพที่ 3.3 สามวิธีสร้างเวกเตอร์ของรายการจากข้อความเดียวกัน: เชิงคำ (3.4) ความหมายแบบสถิต (3.5) และความหมายตามบริบท (3.6) ทุกวิธีส่งเวกเตอร์เข้าสู่กระบวนงานเดียวกันในหัวข้อ 3.8–3.10

3.5.1 เวกเตอร์คำและคลังแนวคิดจำลอง

เวกเตอร์คำจริงมี 100–300 มิติที่มนุษย์อ่านไม่ออก เพื่อให้ตรวจสอบทุกตัวเลขได้ด้วยมือ บทนี้จำลอง “เวกเตอร์คำที่ฝึกมาแล้ว” ด้วย **คลังแนวคิด (Concept Lexicon)** 12 มิติ ได้แก่ tech, mind, space, romance, crime, action, family, music, fantasy, toys, society และ ambition คำหนึ่งคำมีน้ำหนัก 1.0 ในมิติที่เป็นความหมายหลัก และ 0.5 ในมิติที่เป็นความหมายรอง (ตารางที่ 3.20) คลังนี้ครอบคลุม 190 จาก 263 โทเคนของทั้งคลัง (72.24%) คำที่เหลือถูกข้ามไป เทียบได้กับคำนอกคลังศัพท์ (Out-of-Vocabulary) ของแบบจำลองจริง

ตารางที่ 3.20 ตัวอย่างเวกเตอร์คำจากคลังแนวคิด 12 มิติ (แสดงเฉพาะมิติที่ไม่เป็นศูนย์)

คำ	เวกเตอร์คำ w_t	ข้อสังเกต
assassin	crime 1.0, action 0.5	เขียนต่างกับ killers แต่ได้เวกเตอร์เดียวกัน
killers	crime 1.0, action 0.5	⟶ cos(assassin, killers) = 1.0 ขณะที่เวกเตอร์เชิงคำได้ 0
fight	action 1.0	อยู่ในมิติเดียวกับ battle, war, chase
wormhole	space 1.0	คำหายากแต่ความหมายชัด
love	romance 1.0	—
actress	music 0.5, ambition 0.5	ความหมายรองสองมิติ
dream (แบบสถิต)	mind 0.5, ambition 0.5	คำหลายความหมาย: เวกเตอร์เดียวต้อง **ปนทุกความหมาย** เข้าด้วยกัน
space (แบบสถิต)	space 0.5, toys 0.5	อวกาศ กับ space ranger (ของเล่น) ปนกัน

แถวสองแถวสุดท้ายคือหัวใจของข้อจำกัดของแบบสถิต คำว่า dream มีความหมายสองอย่างในคลังนี้ (ความฝันขณะหลับใน Inception และความใฝ่ฝันใน La La Land) แต่แบบจำลองสถิตมีเวกเตอร์ให้คำนี้เพียงตัวเดียว จึงต้องเฉลี่ยทั้งสองความหมายไว้ด้วยกัน ไม่ว่าจะพบคำนี้ที่ใด

3.5.2 จากเวกเตอร์คำสู่เวกเตอร์ของรายการ

รายการหนึ่งรายการมีหลายคำ จึงต้องรวมเวกเตอร์คำให้เหลือเวกเตอร์เดียว วิธีที่ใช้กันมีสองแบบหลัก คือ เฉลี่ยธรรมดา และ **เฉลี่ยถ่วงน้ำหนักด้วย IDF** ซึ่งให้คำเฉพาะมีเสียงดังกว่าคำทั่วไป บทนี้ใช้แบบหลัง แล้วปรับความยาวแบบ L2 เหมือนหัวข้อ 3.4 เพื่อให้ใช้โคไซน์และผลคูณจุดได้เหมือนเดิม

นิยาม 3.5 เวกเตอร์ของรายการจากเวกเตอร์คำแบบสถิต

$$e_d = \frac{\sum_{t \in d} idf(t)\, w_t}{\sum_{t \in d} idf(t)}, \qquad \hat{e}_d = \frac{e_d}{\|e_d\|_2} \tag{3.7}$$

โดย w_t คือเวกเตอร์คำของโทเคน t ผลรวมวิ่งบนทุกโทเคนของเอกสาร (คำที่ซ้ำนับซ้ำ) และข้ามคำที่ไม่มีเวกเตอร์ การหารด้วยผลรวมของ IDF ทำให้เอกสารยาวไม่ได้เวกเตอร์ที่ยาวกว่า

ตัวอย่างการคำนวณ 3.6 เวกเตอร์ความหมายของ i7 John Wick และความคล้ายกับ i8 Mad Max: Fury Road

ขั้นที่ 1 — ข้อมูลเข้า: โทเคนของ i7 ที่อยู่ในคลังแนวคิด พร้อม IDF และเวกเตอร์คำ

เรื่องย่อของ i7 มี 20 โทเคน อยู่ในคลังแนวคิด 11 คำ (ตารางที่ 3.21) ส่วน retired, returns, dog, left, late, path, hotels, clubs, night ถูกข้าม

ตารางที่ 3.21 โทเคนของ i7 John Wick ที่อยู่ในคลังแนวคิด พร้อม IDF และเวกเตอร์คำ

โทเคน t	$idf(t)$	เวกเตอร์คำ w_t (มิติที่ไม่เป็นศูนย์)
assassin	2.8718	crime 1.00, action 0.50
criminal	2.8718	crime 1.00
underworld	2.8718	crime 1.00
gangsters	2.8718	crime 1.00
kill	2.8718	crime 1.00, action 0.50
wife	2.8718	family 1.00
cuts	2.8718	action 0.50
violent	2.4663	crime 1.00, action 0.50
revenge	2.8718	crime 1.00, action 0.50
rival	2.4663	action 0.50
killers	2.8718	crime 1.00, action 0.50
ผลรวม IDF	30.7789	

ขั้นที่ 2 — รวมเวกเตอร์คำแบบถ่วงน้ำหนักด้วย IDF ทีละมิติ

มิติ crime ได้ผลรวมจากแปดคำ: assassin 2.8718×1.0 + criminal 2.8718×1.0 + underworld 2.8718×1.0 + … = 22.5689 · มิติ action ได้จากเจ็ดคำที่มี action 0.5 รวม 9.6459 · มิติ family ได้จาก wife เพียงคำเดียว 2.8718

$$e_{i7,\text{crime}} = \frac{22.5689}{30.7789} = 0.7333$$

$$e_{i7,\text{action}} = \frac{9.6459}{30.7789} = 0.3134$$

$$e_{i7,\text{family}} = \frac{2.8718}{30.7789} = 0.0933$$

ขั้นที่ 3 — ปรับความยาวแบบ L2

$$\| e_{i7} \|_2 = \sqrt{0.7333^2 + 0.3134^2 + 0.0933^2} = 0.8029$$

$$\hat{e}_{i7,\text{crime}} = \frac{0.7333}{0.8029} = 0.9133$$

ขั้นที่ 4 — ทำเช่นเดียวกันกับ i8 แล้วคูณรายมิติ (ตารางที่ 3.22)

ตารางที่ 3.22 เวกเตอร์ความหมายแบบสถิต 12 มิติ (หลัง L2) ของ i7 และ i8 พร้อมผลคูณรายมิติ

มิติ	$\hat{e}_{i7}$ John Wick	$\hat{e}_{i8}$ Mad Max	ผลคูณ
tech	0	0	0
mind	0	0	0
space	0	0	0
romance	0	0	0
crime	0.9133	0	0
action	0.3903	0.9924	0.3874
family	0.1162	0	0
music	0	0	0
fantasy	0	0	0
toys	0	0	0
society	0	0.1129	0
ambition	0	0.0485	0
รวม (โคไซน์)			0.3874

ขั้นที่ 5 — ตีความ

โคไซน์ TF-IDF ของคู่นี้เท่ากับ 0.0000 เพราะไม่มีคำร่วมกันเลย แต่ในปริภูมิความหมายได้ 0.3874 เพราะ assassin, kill, revenge ของ John Wick กับ fight, chase, survival ของ Mad Max ถูกดึงเข้าหามิติ action เดียวกัน ระบบจึงเห็นว่าทั้งสองเรื่องเป็นแนวบู๊ ทั้งที่ผู้เขียนเรื่องย่อไม่ได้ใช้คำเดียวกันเลย ค่าไม่สูงมากเพราะ John Wick เอนไปทาง crime (0.91) ขณะที่ Mad Max เป็น action เกือบล้วน (0.99)

ตารางที่ 3.23 โคไซน์เชิงคำเทียบกับเชิงความหมายแบบสถิตของคู่ภาพยนตร์ที่น่าสนใจ

คู่ภาพยนตร์	TF-IDF	แบบสถิต	สิ่งที่เกิดขึ้น
i7 John Wick – i8 Mad Max	0.0000	0.3874	คำพ้องความหมาย: จาก 0 เป็นบวก (แก้ Synonymy ได้)
i10 Toy Story – i11 Coco	0.0000	0.2676	แอนิเมชันครอบครัว: จาก 0 เป็นบวก
i4 Titanic – i6 The Notebook	0.1819	0.8339	Romance ทั้งคู่: เพิ่มขึ้นชัดเจน

คู่ภาพยนตร์	TF-IDF	แบบสถิต	สิ่งที่เกิดขึ้น
i2 Inception – i5 La La Land	0.2349	0.3451	**ยิ่งสูงขึ้น** เพราะ dream แบบสถิตปนสองความหมาย (Polysemy ยังไม่ถูกแก้)
i1 The Matrix – i12 Spirited Away	0.1835	0.3296	world แบบสถิตปน tech กับ fantasy: ยังสูงอยู่

ข้อควรระวัง แบบสถิตแก้ Synonymy ได้ แต่แก้ Polysemy ไม่ได้ และอาจทำให้แย่ลง

คู่ Inception – La La Land ไม่เพียงยังคล้ายกัน แต่คล้ายกัน **มากขึ้น** จาก 0.2349 เป็น 0.3451 เพราะเวกเตอร์เดียวของ dream แบกทั้งความหมาย mind และ ambition Inception จึงติดมิติ ambition มาด้วยทั้งที่ไม่เกี่ยว และ La La Land ก็ติดมิติ mind มาด้วย แบบจำลองจริงอย่าง word2vec มีปัญหาเดียวกันนี้กับคำว่า bank (ธนาคาร/ตลิ่ง) หรือ apple (ผลไม้/บริษัท) ทางแก้คือให้เวกเตอร์ของคำ **เปลี่ยนตามประโยครอบข้าง** ซึ่งเป็นเรื่องของหัวข้อถัดไป

3.6 เวกเตอร์ความหมายตามบริบท (Contextual Embedding)

แบบจำลองภาษาตระกูล Transformer เช่น BERT, E5 และ BGE ให้เวกเตอร์ของ **แต่ละโทเคนในประโยค** โดยดูคำรอบข้างประกอบ คำว่า dream ใน “thief steals secrets inside a shared dream” จึงได้เวกเตอร์คนละตัวกับ dream ใน “chase a dream in a bright city” นี่คือสิ่งที่แบบสถิตทำไม่ได้ หัวข้อนี้อธิบายกลไกด้วยแบบจำลองอย่างง่ายที่คำนวณด้วยมือได้ แล้วต่อยอดไปสู่ตัวเข้ารหัสคู่ ตัวเข้ารหัสไขว้ และการฝึกด้วย InfoNCE ซึ่งเป็นแกนของระบบค้นคืนเชิงความหมายสมัยใหม่

3.6.1 บริบทเลือกความหมาย: แบบจำลองอย่างง่าย

แบบจำลองจริงคำนวณเวกเตอร์ตามบริบทผ่านกลไกความสนใจ (Attention) หลายชั้น เพื่อให้เห็นแก่นของแนวคิด บทนี้ใช้แบบจำลองอย่างง่ายที่มีสองขั้น ขั้นแรก สรุปบริบทของเอกสารเป็น **เวกเตอร์บริบท** c_d คือค่าเฉลี่ยของเวกเตอร์คำแบบสถิตของคำอื่นทั้งหมดในเอกสาร ขั้นที่สอง สำหรับคำที่มีหลายความหมาย เลือกความหมาย (Sense) ที่มีโคไซน์กับ c_d สูงสุด

นิยาม 3.6 เวกเตอร์คำตามบริบท (แบบจำลองอย่างง่าย)

$$c_d = \frac{1}{|d|-1}\sum_{t' \in d,\, t' \neq t} w_{t'}, \qquad w_t^{(d)} = \arg\max_{s \in \text{senses}(t)} \cos(s, c_d) \tag{3.8}$$

โดย $\text{senses}(t)$ คือเซตของเวกเตอร์ความหมายที่เป็นไปได้ของคำ t คำที่มีความหมายเดียวใช้เวกเตอร์สถิตตามเดิม ผลลัพธ์คือคำเดียวกันได้เวกเตอร์ต่างกันในเอกสารต่างกัน ซึ่งเป็นสมบัติสำคัญของเวกเตอร์ตามบริบท

ตัวอย่างการคำนวณ 3.7 คำว่า dream ใน i2 Inception กับใน i5 La La Land ได้เวกเตอร์ต่างกันอย่างไร

ขั้นที่ 1 — ข้อมูลเข้า: คำรอบข้างของ dream ในแต่ละเรื่อง (เฉพาะคำที่อยู่ในคลังแนวคิด)

i2 Inception: thief, steals, secrets, job, idea, rival, family, fading, memory, reality, blur, time, world

i5 La La Land: aspiring, actress, jazz, pianist, chase, bright, city, falling, love, love, ambition, reaches, career, wanted, love

ขั้นที่ 2 — เวกเตอร์บริบท c_d = ค่าเฉลี่ยของเวกเตอร์คำรอบข้าง และเวกเตอร์ของสองความหมายของ dream

ตารางที่ 3.24 เวกเตอร์บริบทของ i2 และ i5 เทียบกับสองความหมายของ dream

มิติ	c_{i2} (Inception)	c_{i5} (La La Land)	sense 1: ความฝันขณะหลับ	sense 2: ความใฝ่ฝัน
tech	0.0278	0	0	0
mind	0.4167	0.0588	1.0000	0
space	0.0278	0	0	0
romance	0	0.2059	0	0
crime	0.1667	0	0	0
action	0.0278	0.0294	0	0
family	0.0556	0	0	0
music	0	0.1471	0	0
fantasy	0.0278	0	0	0
toys	0	0	0	0
society	0	0.0294	0	0
ambition	0.1389	0.4118	0	1.0000

ขั้นที่ 3 — โคไซน์ระหว่างแต่ละความหมายกับเวกเตอร์บริบท

ใน Inception: cos(sense 1, c_{i2}) = 0.8748 และ cos(sense 2, c_{i2}) = 0.2916 ⟶ เลือก **sense 1 (mind)** เพราะคำรอบข้างอย่าง memory, reality, idea, secrets ล้วนอยู่ในมิติ mind

ใน La La Land: cos(sense 1, c_{i5}) = 0.1204 และ cos(sense 2, c_{i5}) = 0.8427 ⟶ เลือก **sense 2 (ambition)** เพราะคำรอบข้างคือ aspiring, ambition, career, reaches

ขั้นที่ 4 — ตีความ

คำเดียวกันได้เวกเตอร์ต่างกันตามประโยครอบข้าง และผลต่อความคล้ายของทั้งเรื่องแสดงในตารางที่ 3.25: Inception – La La Land ลดจาก 0.3451 เหลือ 0.0000 เพราะไม่มีมิติร่วมกันอีกต่อไป เช่นเดียวกับ world ใน The Matrix (โลกเสมือน ⟶ tech) กับใน Spirited Away (โลกวิญญาณ ⟶ fantasy) และ space ใน Interstellar (อวกาศ) กับใน Toy Story (space ranger ⟶ toys)

ตารางที่ 3.25 โคไซน์ของคู่ภาพยนตร์ในสามปริภูมิ: เชิงคำ ความหมายแบบสถิต และความหมายตามบริบท

คู่ภาพยนตร์	TF-IDF	แบบสถิต	ตามบริบท	สิ่งที่เกิดขึ้น
i2 Inception – i5 La La Land	0.2349	0.3451	0.0000	Polysemy ของ dream ถูกแก้
i1 The Matrix – i12 Spirited Away	0.1835	0.3296	0.0907	world สองความหมายถูกแยก
i7 John Wick – i8 Mad Max	0.0000	0.3874	0.3881	Synonymy ยังถูกแก้เหมือนเดิม
i10 Toy Story – i11 Coco	0.0000	0.2676	0.2551	คงเดิม
i4 Titanic – i6 The Notebook	0.1819	0.8339	0.8339	คงเดิม (ไม่มีคำหลายความหมาย)

3.6.2 จากเมทริกซ์โทเคนสู่เวกเตอร์ของรายการ: การรวมค่า (Pooling)

แบบจำลองตามบริบทคืน **เมทริกซ์** ขนาด (จำนวนโทเคน × มิติ) ไม่ใช่เวกเตอร์เดียว การได้เวกเตอร์ของรายการจึงต้อง **รวมค่า (Pooling)** ซึ่งเป็นการตัดสินใจที่เปลี่ยนผลได้มากกว่าที่คนส่วนใหญ่คาด วิธีที่พบบ่อยคือ ใช้โทเคนพิเศษ [CLS] ตัวแรก เฉลี่ยทุกโทเคน (Mean) เอาค่าสูงสุดรายมิติ (Max) และเฉลี่ยถ่วงน้ำหนัก (เช่น ถ่วงด้วย IDF)

นิยาม 3.7 กลยุทธ์การรวมค่า

$$\rho_{\text{mean}}(H) = \frac{1}{L}\sum_{i=1}^{L} H_{i,\cdot} \qquad \rho_{\max}(H)_j = \max_i H_{i,j} \qquad \rho_{\text{idf}}(H) = \frac{\sum_i idf(t_i)\, H_{i,\cdot}}{\sum_i idf(t_i)} \tag{3.9}$$

โดย $H \in \mathbb{R}^{L\times n}$ คือเมทริกซ์เวกเตอร์รายโทเคน แถวที่ i คือเวกเตอร์ตามบริบทของโทเคนที่ i หลังรวมค่าแล้วต้องปรับ L2 เสมอก่อนวัดความคล้าย

ตัวอย่างการคำนวณ 3.8 รวมค่าเมทริกซ์โทเคนของ i2 Inception สามแบบ

ขั้นที่ 1 — ข้อมูลเข้า: เมทริกซ์ H ของ i2 มี 18 แถว (โทเคนที่อยู่ในคลังแนวคิด นับซ้ำ) × 12 มิติ แต่ละแถวคือเวกเตอร์ตามบริบทจากหัวข้อ 3.6.1

ตารางที่ 3.26 โทเคนของ i2 Inception พร้อม IDF และเวกเตอร์ตามบริบท (มิติที่ไม่เป็นศูนย์)

ลำดับ	โทเคน	idf	เวกเตอร์ตามบริบท
1	thief	2.8718	crime 1.00
2	steals	2.4663	crime 1.00
3	secrets	2.8718	mind 0.50, crime 0.50
4	dream	2.4663	mind 1.00
5	job	2.8718	crime 0.50
6	idea	2.8718	mind 1.00
7	rival	2.4663	action 0.50
8	dream	2.4663	mind 1.00

ลำดับ	โทเคน	idf	เวกเตอร์ตามบริบท
9	dream	2.4663	mind 1.00
10	dream	2.4663	mind 1.00
11	family	1.9555	family 1.00
12	fading	2.1787	mind 0.50
13	memory	2.1787	mind 1.00
14	dream	2.4663	mind 1.00
15	reality	2.4663	mind 1.00
16	blur	2.8718	mind 0.50
17	time	2.4663	mind 0.50, space 0.50
18	world	2.1787	tech 1.00
	ผลรวม IDF	45.0475	

ขั้นที่ 2 — รวมค่าสามแบบ แล้วปรับ L2 (ตารางที่ 3.27)
ตัวอย่างมิติ mind แบบเฉลี่ย: มี 8 โทเคนที่ mind = 1.0 (dream ×5, idea, memory, reality) และ 4 โทเคนที่ mind = 0.5 (secrets, fading, blur, time) จาก 18 โทเคน จึงได้ $(8 + 2)/18 = 0.5556$ แบบค่าสูงสุด: มิติใดที่มีโทเคนใดถึง 1.0 ก็ได้ 1.0 ทันที แบบถ่วงน้ำหนัก IDF: $(2.4663 \times 5 + 2.8718 + 2.1787 + \cdots)/45.0475 = 0.5559$

ตารางที่ 3.27 เวกเตอร์ของ i2 จากการรวมค่าสามแบบ ก่อนและหลังปรับ L2

มิติ	Mean	Max	IDF-weighted	Mean (L2)	Max (L2)	IDF (L2)
tech	0.0556	1.0000	0.0484	0.0947	0.4714	0.0820
mind	0.5556	1.0000	0.5559	0.9470	0.4714	0.9424
space	0.0278	0.5000	0.0274	0.0474	0.2357	0.0464
romance	0	0	0	0	0	0
crime	0.1667	1.0000	0.1823	0.2841	0.4714	0.3090
action	0.0278	0.5000	0.0274	0.0474	0.2357	0.0464
family	0.0556	1.0000	0.0434	0.0947	0.4714	0.0736
music	0	0	0	0	0	0
fantasy	0	0	0	0	0	0
toys	0	0	0	0	0	0

มิติ	Mean	Max	IDF-weighted	Mean (L2)	Max (L2)	IDF (L2)
society	0	0	0	0	0	0
ambition	0	0	0	0	0	0

ขั้นที่ 3 — ผลต่อความคล้าย

ตารางที่ 3.28 โคไซน์ระหว่างเวกเตอร์ของ i2 (แต่ละแบบรวมค่า) กับ i1 The Matrix

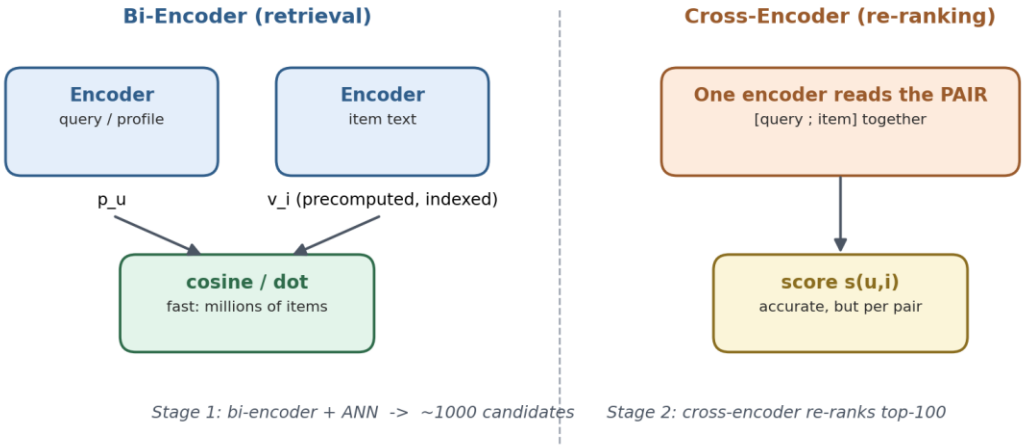
กลยุทธ์รวมค่า	cos(i2, i1)	ข้อสังเกต
Mean	0.6474	ทุกโทเคนเสียงเท่ากัน
Max	0.7012	โทเคนเดียว (world ⟶ tech, thief ⟶ crime) ทำให้มิตินั้นเต็ม 1.0 เวกเตอร์จึงแบนและสูญเสียสัดส่วน
IDF-weighted	0.6348	คำเฉพาะเสียงดังกว่า family (idf 1.96) — **ใช้เป็นค่าตั้งต้นของบท**

ขั้นที่ 4 — ตีความ

แค่เปลี่ยนวิธีรวมค่า โคไซน์กับ The Matrix ก็แกว่งจาก 0.63 ถึง 0.70 แบบ Max ทำให้ i2 ดูมี tech, crime และ family มากพอ ๆ กับ mind ทั้งที่มีคำในมิติเหล่านั้นเพียงคำเดียว แบบจำลองจริงที่ฝึกเพื่องานความคล้ายโดยเฉพาะ (E5, BGE) **ถูกฝึกให้ใช้กลยุทธ์รวมค่าแบบใดแบบหนึ่งอย่างเจาะจง** ต้องใช้ตามที่เอกสารกำหนด และเมื่อประมวลผลเป็นชุดต้องคูณหน้ากาก (Mask) ก่อนเฉลี่ย มิฉะนั้นโทเคนเติมเต็มที่เป็นศูนย์จะดึงค่าเฉลี่ยลง

3.6.3 ตัวเข้ารหัสคู่กับตัวเข้ารหัสไขว้: แบ่งงานกันคนละขั้น

เมื่อรวมค่าจนได้เวกเตอร์เดียวต่อรายการแล้ว คำถามถัดไปคือจะนำเวกเตอร์ไปเปรียบเทียบอย่างไรให้ทั้งเร็วและแม่น คำตอบในทางปฏิบัติคือมีตัวเข้ารหัสสองแบบที่รับผิดชอบคนละขั้น (ภาพที่ 3.4)



ภาพที่ 3.4 ตัวเข้ารหัสคู่เข้ารหัสสองฝั่งแยกกันจึงคำนวณเวกเตอร์รายการล่วงหน้าได้ ส่วนตัวเข้ารหัสไขว้อ่านทั้งคู่พร้อมกันจึงแม่นกว่าแต่ต้องคำนวณใหม่ทุกคู่

ตัวเข้ารหัสคู่ (Bi-Encoder) เข้ารหัสข้อคำถาม (หรือโปรไฟล์ผู้ใช้) และรายการ **แยกกัน** เป็นเวกเตอร์คนละตัว แล้ววัดความคล้ายด้วยโคไซน์หรือผลคูณจุด ผลคือคำนวณเวกเตอร์ของทุกรายการไว้ล่วงหน้าและเก็บลงดัชนีได้ เมื่อผู้ใช้เข้ามาจริงเหลือเพียงงานหาเพื่อนบ้านใกล้สุด ทุกเวกเตอร์ในหัวข้อ 3.4–3.6 ล้วนอยู่ในกรอบนี้ ข้อเสียคือทั้งสองฝั่งไม่เคย “เห็นกัน” จึงจับความสัมพันธ์ระดับละเอียด เช่น การปฏิเสธหรือเงื่อนไขซ้อน ได้น้อยกว่า

ตัวเข้ารหัสไขว้ (Cross-Encoder) ป้อนข้อคำถามและรายการเข้าไปในแบบจำลองเดียวกัน **พร้อมกัน** เป็นลำดับเดียว [query ; item] ทุกโทเคนของทั้งสองฝั่งจึงโต้ตอบกันได้เต็มที่ผ่านกลไกความสนใจ ให้ความแม่นยำสูงที่สุด แต่ **คำนวณล่วงหน้าไม่ได้เลย** เพราะผลลัพธ์ขึ้นกับคู่ ต้องรันใหม่ทุกคู่ทุกครั้ง

ตัวอย่างการคำนวณ 3.9 ต้นทุนของสองตัวเข้ารหัสในคลัง 5,000,000 รายการ

ขั้นที่ 1 — ตัวเข้ารหัสคู่: เวกเตอร์รายการคำนวณไว้แล้ว เหลือผลคูณจุด 0.4 ไมโครวินาทีต่อคู่

$$T_{\text{bi}} = 5{,}000{,}000 \times 0.4\ \mu s = 2.0 \text{ วินาที (ค้นครบทุกรายการ)} \rightarrow$$

หลักมิลลิวินาทีเมื่อใช้ดัชนี ANN (หัวข้อ 3.11)

ขั้นที่ 2 — ตัวเข้ารหัสไขว้: ต้องรัน Transformer 5 มิลลิวินาทีต่อคู่

$$T_{\text{cross}} = 5{,}000{,}000 \times 5\ \text{ms} = 25{,}000\ \text{s} = 6.9 \text{ ชั่วโมงต่อการค้นหนึ่งครั้ง}$$

ขั้นที่ 3 — แต่ถ้าใช้ตัวเข้ารหัสไขว้กับ CANDIDATE ITEM เพียง 100 รายการที่ตัวเข้ารหัสคู่คัดมาแล้ว

$$T_{\text{cross}}^{(100)} = 100 \times 5\ \text{ms} = 0.5 \text{ วินาที}$$

ขั้นที่ 4 — ตีความ

นี่คือเหตุผลของ **สถาปัตยกรรมสองขั้น** ที่อุตสาหกรรมใช้จริง: ขั้นค้นคืน (Retrieval) ใช้ตัวเข้ารหัสคู่ร่วมกับดัชนี ANN ลดจากรายการนับล้านเหลือประมาณ 1,000 รายการภายในไม่กี่มิลลิวินาที

แล้วขั้นจัดอันดับซ้ำ (Re-ranking) ใช้ตัวเข้ารหัสไขว้กับ CANDIDATE ITEM 100 อันดับแรก ได้ทั้งความเร็วของขั้นแรกและความแม่นยำของขั้นที่สอง (หัวข้อ 3.12.2)

ตารางที่ 3.29 เปรียบเทียบตัวเข้ารหัสคู่กับตัวเข้ารหัสไขว้

ประเด็น	ตัวเข้ารหัสคู่ (Bi-Encoder)	ตัวเข้ารหัสไขว้ (Cross-Encoder)
ข้อมูลเข้า	ข้อคำถามและรายการแยกกัน	ข้อคำถามและรายการต่อกันเป็นลำดับเดียว
ผลลัพธ์	เวกเตอร์สองตัว ⟶ โคไซน์	คะแนนหนึ่งค่าโดยตรง
คำนวณล่วงหน้า	ได้ (เก็บเวกเตอร์รายการลงดัชนี)	ไม่ได้ (ขึ้นกับคู่)
ความเร็ว	มิลลิวินาทีต่อคลังหลายล้าน	มิลลิวินาทีต่อคู่
ความแม่นยำ	ดี	ดีที่สุด
บทบาทในระบบ	ขั้นค้นคืน (Retrieval) และ Two-Tower (หัวข้อ 3.11.4)	ขั้นจัดอันดับซ้ำ (Re-ranking) บน CANDIDATE ITEM หลักร้อย

3.6.4 การฝึกตัวเข้ารหัสคู่ด้วย InfoNCE

เวกเตอร์ความหมายไม่ได้เกิดขึ้นเอง ตัวเข้ารหัสคู่ถูกฝึกด้วย **การเรียนรู้เชิงเปรียบต่าง (Contrastive Learning)**: ป้อน **คู่บวก** (ข้อคำถามกับรายการที่เกี่ยวข้องจริง) และ **คู่ลบ** (ข้อคำถามกับรายการที่ไม่เกี่ยว) แล้วปรับพารามิเตอร์ให้คู่บวกใกล้กันขึ้นและคู่ลบไกลกันออก ฟังก์ชันสูญเสียที่ใช้กันมากที่สุดคือ **InfoNCE** ซึ่งมองการฝึกเป็นการจำแนกว่า “ในบรรดารายการเหล่านี้ รายการใดคือคู่บวก” ด้วยซอฟต์แมกซ์

นิยาม 3.8 ค่าสูญเสีย InfoNCE

$$\mathcal{L} = -\ln \frac{\exp(s^+/\tau)}{\exp(s^+/\tau)+\sum_j \exp(s_j^-/\tau)} \tag{3.10}$$

โดย s^+ คือคะแนนความคล้ายของคู่บวก s_j^- คือคะแนนของคู่ลบตัวที่ j และ $\tau > 0$ คืออุณหภูมิ (Temperature) ที่คุมความ “คม” ของซอฟต์แมกซ์ เศษส่วนในลอการิทึมคือความน่าจะเป็นที่แบบจำลองเลือกคู่บวกได้ถูก ค่าสูญเสียจึงเป็นศูนย์เมื่อเลือกถูกด้วยความมั่นใจเต็มร้อย

ตัวอย่างการคำนวณ 3.10 ฝึกตัวเข้ารหัสด้วยข้อคำถาม “thief steals secrets dream” ที่มีคู่บวกคือ i2 Inception

ขั้นที่ 1 — ข้อมูลเข้า: คะแนนความคล้ายก่อนฝึก (ตัวเข้ารหัสแบบสถิตจากหัวข้อ 3.5 ซึ่งยังแยกความหมายของ dream ไม่ออก)

เวกเตอร์ข้อคำถามคือค่าเฉลี่ยของเวกเตอร์คำ thief, steals, secrets, dream หลังปรับ L2 ได้ mind 0.3651, crime 0.9129, ambition 0.1826 คู่บวกคือ i2 และสุ่มคู่ลบมาสี่รายการ (ตารางที่ 3.30)

ตารางที่ 3.30 คะแนนโคไซน์ระหว่างข้อคำถามกับคู่บวกและคู่ลบ แล้วคำนวณซอฟต์แมกซ์ที่ $\tau = 0.1$

คู่	รายการ	s	$z = s/\tau$	$z - \max z$	$\exp(\cdot)$	ความน่าจะเป็น
คู่บวก	i2 Inception	0.7148	7.1480	-1.1890	0.304526	0.232640

คู่	รายการ	s	$z = s/\tau$	$z - \max z$	$\exp(\cdot)$	ความน่าจะเป็น
คู่ลบ 1	i5 La La Land	0.1985	1.9850	-6.3520	0.001743	0.001332
คู่ลบ 2	i7 John Wick	0.8337	8.3370	0.0000	1.000000	0.763941
คู่ลบ 3	i1 The Matrix	0.2334	2.3340	-6.0030	0.002471	0.001888
คู่ลบ 4	i8 Mad Max: Fury Road	0.0089	0.0890	-8.2480	0.000262	0.000200
	ผลรวม				1.309002	1.000000

ขั้นที่ 2 — หารด้วยอุณหภูมิ แล้วลบค่าสูงสุดออกเพื่อกัน exp ล้นค่า (ไม่เปลี่ยนผลของซอฟต์แมกซ์)

$$z = (7.1480, 1.9850, 8.3370, 2.3340, 0.0890) \qquad z - \max z = (-1.1890, -6.3520, 0.0000, -6.0030, -8.2480)$$

ขั้นที่ 3 — ซอฟต์แมกซ์และค่าสูญเสีย

$$P(\text{คู่บวก}) = \frac{0.304526}{1.309002} = 0.232640 \qquad \mathcal{L} = -\ln 0.232640 = 1.4583$$

ขั้นที่ 4 — ตีความ: คู่ลบยาก (Hard Negative)

ก่อนฝึก แบบจำลองให้ i7 John Wick (0.8337) สูงกว่าคู่บวก i2 (0.7148) เสียอีก เพราะ thief และ steals ดึงข้อคำถามไปทางมิติ crime ซึ่ง John Wick เต็มไปด้วยคำแนวนี้ ความน่าจะเป็นของคู่บวกจึงเหลือเพียง 0.2326 และค่าสูญเสียสูงถึง 1.4583 เกรเดียนต์ของค่าสูญเสียนี้จะ **ดึง** เวกเตอร์ข้อคำถามเข้าหา i2 และ **ผลัก** ออกจาก i7 แรงที่สุด (สัดส่วนตามความน่าจะเป็นของแต่ละคู่ลบ) i7 จึงเป็น **คู่ลบยาก** ที่ให้สัญญาณเรียนรู้มากที่สุด ส่วน i8 Mad Max ที่ได้ความน่าจะเป็นเกือบศูนย์แทบไม่มีผลต่อการฝึกเลย

ขั้นที่ 5 — เทียบกับการสุ่มคู่ลบง่าย และผลของอุณหภูมิ

ตารางที่ 3.31 ค่าสูญเสียเมื่อเปลี่ยนชุดคู่ลบและอุณหภูมิ

ชุดคู่ลบ	τ	$P(\text{คู่บวก})$	$\mathcal{L}$	สิ่งที่แบบจำลองได้เรียน
ยาก: i5, i7, i1, i8	0.05	0.0849	2.4667	คมมาก: เกือบทั้งหมดของเกรเดียนต์ไปที่ i7
ยาก: i5, i7, i1, i8	0.10	0.2326	1.4583	สัญญาณชัดและกระจายพอเหมาะ
ยาก: i5, i7, i1, i8	0.50	0.3077	1.1787	เรียบเกิน: แทบไม่แยกคู่ลบยากจากคู่ลบง่าย
ง่าย: i4, i8, i10, i12	0.10	0.9899	0.0101	**แทบไม่ได้เรียนอะไร** เพราะแยกได้ตั้งแต่แรก

ค่าสูญเสียที่ร่วงเกือบศูนย์ตั้งแต่ต้นไม่ใช่ข่าวดี แต่มักแปลว่าคู่ลบง่ายเกินไป ทางแก้คือ **ขุดคู่ลบยาก (Hard Negative Mining)** คือเลือกรายการที่แบบจำลองปัจจุบันให้คะแนนสูงผิด ๆ มาเป็นคู่ลบ ไม่ใช่การลด τ ในทางปฏิบัตินิยมใช้รายการบวกของข้อคำถามอื่นในชุดเดียวกันเป็นคู่ลบ (In-Batch Negatives) เพราะได้มาฟรี แต่ต้องระวัง **คู่ลบลวง (False Negative)** คือรายการที่จริง ๆ เกี่ยวข้องแต่ยังไม่มีป้ายกำกับ การผลักรายการเหล่านั้นออกจะทำให้แบบจำลองแย่ลง

3.6.5 เวกเตอร์ความหมายตามบริบทของทั้งคลัง MovieMini-12

ตารางที่ 3.32 คือผลลัพธ์สุดท้ายของหัวข้อ 3.5–3.6: เวกเตอร์ความหมายตามบริบท 12 มิติ (รวมค่าถ่วง IDF แล้วปรับ L2) ของทั้ง 12 เรื่อง ซึ่งจะเป็น **ปริภูมิความหมาย** ที่ใช้สร้างโปรไฟล์และให้คะแนนตั้งแต่หัวข้อ 3.8 เป็นต้นไป ควบคู่กับปริภูมิ TF-IDF และปริภูมิแนวภาพยนตร์

ตารางที่ 3.32 เวกเตอร์ความหมายตามบริบท (หลัง L2) ของทั้ง 12 เรื่อง (มิติเป็นแถว รายการเป็นคอลัมน์)

มิติ	i1	i2	i3	i4	i5	i6	i7	i8	i9	i10	i11	i12
tech	0.76	0.08	·	·	·	·	·	·	·	·	·	·
mind	0.59	0.94	0.10	·	·	0.39	·	·	0.13	·	0.39	0.12
space	·	0.05	0.94	·	·	·	·	·	·	0.07	·	·
romance	·	·	0.06	0.88	0.31	0.88	·	·	·	·	0.08	·
crime	·	0.31	·	·	·	0.07	0.91	·	0.33	·	·	0.17
action	0.27	0.05	0.17	0.30	·	0.20	0.39	0.99	0.06	0.07	·	0.06
family	·	0.07	0.28	·	·	0.15	0.12	·	0.63	0.38	0.67	0.15
music	·	·	·	·	0.28	·	·	·	·	·	0.58	·
fantasy	·	·	·	·	·	·	·	·	·	·	0.25	0.96
toys	·	·	·	·	·	·	·	·	·	0.92	·	·
society	·	·	0.06	0.37	0.06	·	·	0.11	0.69	·	·	·
ambition	·	·	·	0.07	0.91	·	·	·	·	·	·	·

อ่านตารางตามคอลัมน์จะเห็น “ลายนิ้วมือความหมาย” ของแต่ละเรื่อง: i1 The Matrix คือ tech + mind, i2 Inception คือ mind + crime, i3 Interstellar คือ space + family, i8 Mad Max คือ action เกือบล้วน และ i10 Toy Story คือ toys + family อ่านตามแถวจะเห็นว่ามิติ mind ปรากฏในหลายเรื่อง (i1, i2, i6, i11) เพราะคำอย่าง reality, memory, illusion ล้วนอยู่ในมิตินี้ ซึ่งจะส่งผลต่อคำแนะนำของ u1 ในหัวข้อ 3.9

3.7 สรุปการสร้างเวกเตอร์ของรายการ

ก่อนไปสู่ฝั่งผู้ใช้ หัวข้อนี้สรุปสิ่งที่ผู้ออกแบบต้องตัดสินใจเมื่อสร้างเวกเตอร์ของรายการในระบบจริง ตารางที่ 3.33 เทียบสามแบบที่เรียนมา และตารางที่ 3.34 แนะนำทางเลือกอื่นที่ควรรู้จักชื่อ

ตารางที่ 3.33 เปรียบเทียบเวกเตอร์ของรายการสามแบบ

ประเด็น	เชิงคำ (TF-IDF / BM25)	ความหมายแบบสถิต	ความหมายตามบริบท
หน่วยของมิติ	หนึ่งคำ = หนึ่งมิติ	แนวคิดแฝง (100–300 มิติ)	แนวคิดแฝง (384–4096 มิติ)
ความเบาบาง	เบาบางมาก (ศูนย์เกือบทั้งหมด)	หนาแน่น	หนาแน่น
คำพ้องความหมาย	จับไม่ได้	จับได้	จับได้
คำหลายความหมาย	จับไม่ได้	จับไม่ได้ (เวกเตอร์เดียวปนทุกความหมาย)	จับได้

ประเด็น	เชิงคำ (TF-IDF / BM25)	ความหมายแบบสถิต	ความหมายตามบริบท
ชื่อเฉพาะ/รหัสที่ต้องตรงเป๊ะ	ดีมาก	แย่	แย่
ต้องฝึก	ไม่ต้อง	ใช้ที่ฝึกแล้ว	ใช้ที่ฝึกแล้ว หรือปรับแต่งด้วย InfoNCE
อธิบายผล	รายคำ อ่านออกทันที	ต้องตีความมิติแฝง	ต้องตีความมิติแฝง
ต้นทุนเข้ารหัส	ไม่โครวินาที	ไม่โครวินาที	มิลลิวินาที (GPU)
ดัชนี	ดัชนีผกผัน (Inverted Index)	ดัชนี ANN	ดัชนี ANN
ผลบน MovieMini-12	i7–i8 = 0, i2–i5 สูงสุด	i7–i8 = 0.39, i2–i5 ยิ่งสูง	i7–i8 = 0.39, i2–i5 = 0

ตารางที่ 3.34 ทางเลือกอื่นระหว่างเชิงคำกับเชิงความหมายที่ควรรู้จัก

วิธี	สิ่งที่ควรรู้	เหมาะเมื่อ
BM25	เวกเตอร์เบาบางจากการนับคำ พร้อมการอิ่มตัวและปรับความยาว	ระบบฐานที่ต้องมีเสมอ · ชื่อเฉพาะ
SPLADE	เวกเตอร์เบาบางที่ **โมเดลเรียนรู้น้ำหนักคำ** และเติมคำที่ไม่อยู่ในเอกสารได้ (จับคำพ้องความหมายได้ทั้งที่ยังใช้ดัชนีผกผัน)	ต้องการความหมายแต่อยากคงโครงสร้างดัชนีเดิม
Bi-Encoder (E5, BGE)	เวกเตอร์หนาแน่นเดียวต่อข้อความ ค้นคืนเร็วที่สุดด้วย ANN	ขั้นค้นคืนของระบบส่วนใหญ่
ColBERT	เก็บเวกเตอร์ **ระดับโทเคน** แล้วให้คะแนนด้วย MaxSim (แต่ละโทเคนของข้อคำถามหาโทเคนของเอกสารที่เข้ากันที่สุด) แม่นกว่าเวกเตอร์เดียวแต่ใช้หน่วยความจำมาก	ต้องการความละเอียดระดับคำ
Cross-Encoder	แม่นที่สุด แต่ต้องคำนวณใหม่ทีละคู่	ขั้นจัดอันดับซ้ำบน CANDIDATE ITEM หลักร้อย

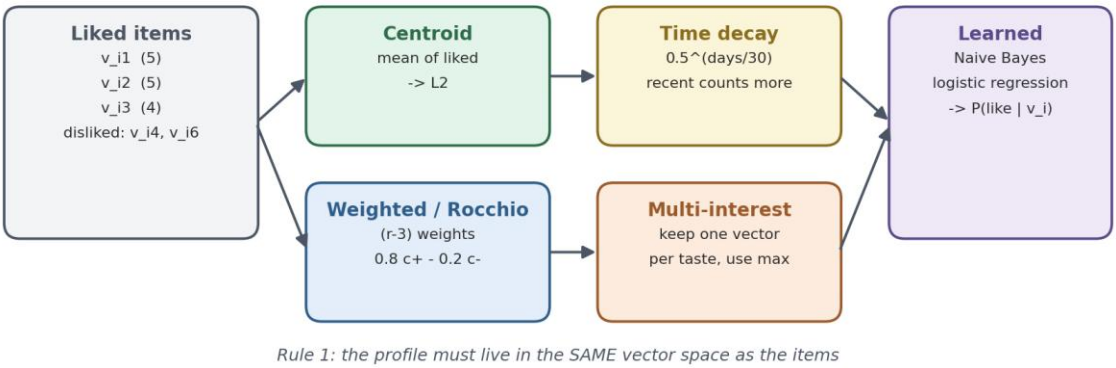
ประเด็นสำคัญ รายการตรวจสอบก่อนสร้างเวกเตอร์ของรายการในระบบจริง

1. **เขตข้อมูล** ใช้ข้อความใดบ้าง (ชื่อเรื่อง แนว เรื่องย่อ บทวิจารณ์) และให้น้ำหนักเขตสั้นแต่สำคัญอย่างไร
2. **การเตรียมข้อความ** ตัดคำ คำหยุด ลดรูปคำ และ (ภาษาไทย) ตรวจผลตัดคำด้วยตาก่อนเสมอ
3. **ชนิดของเวกเตอร์** เริ่มจาก TF-IDF/BM25 เป็นเส้นฐาน แล้วเพิ่มเวกเตอร์ความหมายเมื่อวัดผลแล้วดีขึ้นจริง ระบบจริงมักใช้ทั้งสองแบบร่วมกัน (หัวข้อ 3.12)
4. **การรวมค่าและการปรับความยาว** ใช้กลยุทธ์รวมค่าตามที่แบบจำลองกำหนด และปรับ L2 ทุกเวกเตอร์ก่อนเก็บ
5. **เวอร์ชันของแบบจำลอง** ทุกครั้งที่เปลี่ยนตัวเข้ารหัส เวกเตอร์เดิมทั้งคลังใช้ไม่ได้อีก ต้องเข้ารหัสใหม่และสร้างดัชนีใหม่ (หัวข้อ 3.11)

6. **คำนอกคลังศัพท์** วัดสัดส่วนโทเคนที่แบบจำลองไม่รู้จัก (ในบทนี้ 28%) โดยเฉพาะศัพท์เฉพาะโดเมนและภาษาไทย

3.8 การสร้างโปรไฟล์ผู้ใช้

หัวข้อ 3.3–3.7 ตอบครึ่งแรกของปัญหา คือแทน **รายการ** ด้วยเวกเตอร์ หัวข้อนี้ตอบอีกครึ่ง คือแทน **ผู้ใช้** ด้วยเวกเตอร์ที่เรียกว่า **โปรไฟล์ผู้ใช้ (User Profile)** ซึ่งในระบบอิงเนื้อหาไม่ได้เก็บว่าผู้ใช้อายุเท่าใดหรืออยู่ที่ไหน แต่เก็บว่า “ผู้ใช้ชอบเนื้อหาแบบใด” ในรูปของเวกเตอร์ที่มีมิติเดียวกับเวกเตอร์ของรายการทุกประการ หัวข้อนี้แนะนำวิธีสร้างโปรไฟล์หกแบบ จากง่ายไปซับซ้อน แต่ละแบบเหมาะกับข้อมูลที่ต่างกัน (ภาพที่ 3.5)



ภาพที่ 3.5 หกวิธีสร้างโปรไฟล์ผู้ใช้จากรายการที่เคยให้ผลป้อนกลับ ทุกวิธีให้ผลลัพธ์อยู่ในปริภูมิเดียวกับเวกเตอร์ของรายการ

3.8.1 หลักการข้อเดียว: โปรไฟล์ต้องอยู่ในปริภูมิเดียวกับรายการ

ถ้ารายการถูกแทนด้วย TF-IDF บนคลังศัพท์ 203 คำ โปรไฟล์ก็ต้องเป็นเวกเตอร์ 203 มิติบนคลังศัพท์เดียวกัน ถ้ารายการถูกแทนด้วยเวกเตอร์ความหมาย 12 มิติ โปรไฟล์ก็ต้องมี 12 มิติ มิฉะนั้นผลคูณจุดในหัวข้อ 3.9 จะไม่มีความหมาย ข้อดีที่ตามมาคือ **สูตรสร้างโปรไฟล์ทุกสูตรในหัวข้อนี้ใช้ได้กับทุกปริภูมิโดยไม่ต้องเปลี่ยน** เพียงเปลี่ยน v_i หัวข้อนี้จะแสดงตัวอย่างในสามปริภูมิสลับกัน คือ ปริภูมิแนวภาพยนตร์ 9 มิติ (ตารางที่ 3.35) ซึ่งเล็กพอที่จะคำนวณครบทุกช่อง ปริภูมิ TF-IDF 203 มิติ และปริภูมิความหมาย 12 มิติจากตารางที่ 3.32

ตารางที่ 3.35 เวกเตอร์แนวภาพยนตร์ 9 มิติของทั้ง 12 เรื่อง (1 = อยู่ในแนวนั้น)

รหัส	Action	Animation	Drama	Family	Fantasy	Musical	Romance	Sci-Fi	Thriller
i1	1	·	·	·	·	·	·	1	·
i2	·	·	·	·	·	·	·	1	1
i3	·	·	1	·	·	·	·	1	·
i4	·	·	1	·	·	·	1	·	·
i5	·	·	·	·	·	1	1	·	·
i6	·	·	1	·	·	·	1	·	·
i7	1	·	·	·	·	·	·	·	1

รหัส	Action	Animation	Drama	Family	Fantasy	Musical	Romance	Sci-Fi	Thriller
i8	1	·	·	·	·	·	·	1	·
i9	·	·	1	·	·	·	·	·	1
i10	·	1	·	1	·	·	·	·	·
i11	·	1	·	1	·	·	·	·	·
i12	·	1	·	·	1	·	·	·	·

ข้อมูลเข้าของทุกวิธีคือคะแนนของ u1 จากตารางที่ 3.5: ชอบ $I_u^+ = \{i1, i2, i3\}$ (คะแนน 5, 5, 4) ไม่ชอบ $I_u^- = \{i4, i6\}$ (คะแนน 1, 1)

3.8.2 แบบที่ 1 ค่าเฉลี่ยศูนย์กลาง (Centroid Profile)

วิธีที่ง่ายที่สุดและใช้ได้ดีอย่างน่าประหลาดใจ คือนำเวกเตอร์ของทุกรายการที่ผู้ใช้ชอบมา **เฉลี่ยรายมิติ** ได้จุดกึ่งกลางของสิ่งที่ผู้ใช้ชอบ แล้วปรับความยาวแบบ L2

นิยาม 3.9 โปรไฟล์แบบค่าเฉลี่ยศูนย์กลาง

$$c_u = \frac{1}{|I_u^+|} \sum_{i \in I_u^+} v_i \,, \qquad p_u = \frac{c_u}{\|c_u\|_2} \tag{3.11}$$

มิติที่ t ของ p_u อ่านได้ว่า “ผู้ใช้ชอบคุณลักษณะ t มากเพียงใดเมื่อเทียบกับคุณลักษณะอื่น” การหารด้วยจำนวนรายการทำให้ผู้ใช้ที่ชอบ 3 เรื่องกับ 30 เรื่องเทียบกันได้ และการปรับ L2 ทำให้โคไซน์เท่ากับผลคูณจุด

ตัวอย่างการคำนวณ 3.11 โปรไฟล์ค่าเฉลี่ยศูนย์กลางของ u1 ในปริภูมิแนวภาพยนตร์

ขั้นที่ 1 — ข้อมูลเข้า: เวกเตอร์แนวของสามเรื่องที่ชอบ แล้วเฉลี่ยรายมิติ

ตารางที่ 3.36 การคำนวณโปรไฟล์ค่าเฉลี่ยศูนย์กลางของ u1 ในปริภูมิแนวภาพยนตร์

มิติ	i1 The Matrix	i2 Inception	i3 Interstellar	ค่าเฉลี่ย c_u	โปรไฟล์ p_u
Action	1	0	0	0.3333	0.2887
Animation	0	0	0	0.0000	0.0000
Drama	0	0	1	0.3333	0.2887
Family	0	0	0	0.0000	0.0000
Fantasy	0	0	0	0.0000	0.0000
Musical	0	0	0	0.0000	0.0000

มิติ	i1 The Matrix	i2 Inception	i3 Interstellar	ค่าเฉลี่ย c_u	โปรไฟล์ p_u
Romance	0	0	0	0.0000	0.0000
Sci-Fi	1	1	1	1.0000	0.8660
Thriller	0	1	0	0.3333	0.2887

มิติ Sci-Fi ได้ $(1+1+1)/3 = 1.0000$ ส่วน Action, Drama และ Thriller ได้ $(1+0+0)/3 = 0.3333$ มิติที่เหลือเป็นศูนย์

ขั้นที่ 2 — ปรับความยาวแบบ L2

$\| c_u \|_2 = \sqrt{0.3333^2 + 0.3333^2 + 1.0000^2 + 0.3333^2} = \sqrt{1.3333} = 1.1547$ $p_{u,\text{Sci-Fi}} = \frac{1.0000}{1.1547} = 0.8660$

ขั้นที่ 3 — ตีความ

โปรไฟล์บอกว่า u1 เป็นผู้ชม Sci-Fi เป็นหลัก (0.8660) และมี Action, Drama, Thriller เท่า ๆ กันในระดับรอง (0.2887) มิติ Romance เป็นศูนย์ **ทั้งที่ u1 ให้คะแนน Titanic และ The Notebook ต่ำมาก** เพราะค่าเฉลี่ยศูนย์กลางใช้เฉพาะรายการที่ชอบ ยังไม่ได้ใช้ข้อมูลว่า u1 ไม่ชอบอะไรเลย ข้อจำกัดนี้จะแก้ในแบบที่ 2 และ 3

ตัวอย่างการคำนวณ 3.12 โปรไฟล์ค่าเฉลี่ยศูนย์กลางของ u1 ในปริภูมิ TF-IDF 203 มิติ

ขั้นที่ 1 — ข้อมูลเข้า: เวกเตอร์ TF-IDF (หลัง L2) ของ i1, i2, i3 จากหัวข้อ 3.4 สูตรเดียวกันแต่มี 203 มิติ ตารางที่ 3.37 แสดง 10 มิติที่โปรไฟล์ให้ค่าสูงสุด

ตารางที่ 3.37 โปรไฟล์ TF-IDF ของ u1: 10 มิติที่มีค่าสูงสุดจาก 57 มิติที่ไม่เป็นศูนย์

มิติ (คำ)	i1	i2	i3	ค่าเฉลี่ย c_u	โปรไฟล์ p_u
reality	0.4123	0.1726	0.0000	0.1950	0.3159
dream	0.0000	0.5734	0.0000	0.1911	0.3096
world	0.3642	0.1525	0.0000	0.1722	0.2790
time	0.0000	0.1726	0.3394	0.1707	0.2765
space	0.0000	0.0000	0.4386	0.1462	0.2369
machine	0.3715	0.0000	0.0000	0.1238	0.2006
lost	0.0000	0.1726	0.1697	0.1141	0.1848
accepts	0.0000	0.2010	0.0000	0.0670	0.1085
inside	0.0000	0.2010	0.0000	0.0670	0.1085
job	0.0000	0.2010	0.0000	0.0670	0.1085

มิติ (คำ)	i1	i2	i3	ค่าเฉลี่ย c_u	โปรไฟล์ p_u
love, woman, ship, ...	0	0	0	0	0

ขั้นที่ 2 — แทนค่ามิติ reality แล้วปรับความยาว

$$c_{u,\text{reality}} = \frac{0.4123+0.1726+0}{3} = 0.1950 \qquad \| c_u \|_2 = 0.6173 \qquad p_{u,\text{reality}} = \frac{0.1950}{0.6173} = 0.3159$$

ขั้นที่ 3 — ตีความ

คำที่โปรไฟล์ให้น้ำหนักสูงสุดคือ reality, dream, world, time, space ซึ่งตรงกับสิ่งที่มนุษย์สรุปเกี่ยวกับ u1 คำที่ปรากฏในเรื่องที่ชอบมากกว่าหนึ่งเรื่อง (reality, world, time, lost) ได้น้ำหนักสูงกว่าคำที่ปรากฏเรื่องเดียวแม้ค่าในเรื่องนั้นจะสูง เพราะการเฉลี่ยให้รางวัลแก่ ความสอดคล้องข้ามรายการ ส่วน love ซึ่งเด่นมากในสองเรื่องที่ไม่ชอบได้ค่าศูนย์ด้วยเหตุผลเดียวกับปริภูมิแนว

ตัวอย่างการคำนวณ 3.13 โปรไฟล์ค่าเฉลี่ยศูนย์กลางของ u1 ในปริภูมิความหมาย 12 มิติ

ขั้นที่ 1 — ข้อมูลเข้า: เวกเตอร์ความหมายตามบริบทของ i1, i2, i3 จากตารางที่ 3.32

ตารางที่ 3.38 โปรไฟล์ความหมายของ u1: ค่าเฉลี่ยของสามเรื่องที่ชอบแล้วปรับ L2

มิติ	i1	i2	i3	ค่าเฉลี่ย c_u	โปรไฟล์ p_u
tech	0.7561	0.0820	0	0.2793	0.3826
mind	0.5944	0.9424	0.0955	0.5441	0.7452
space	0	0.0464	0.9379	0.3281	0.4494
romance	0	0	0.0556	0.0185	0.0254
crime	0	0.3090	0	0.1030	0.1410
action	0.2741	0.0464	0.1668	0.1624	0.2224
family	0	0.0736	0.2780	0.1172	0.1605
music	0	0	0	0	0
fantasy	0	0	0	0	0
toys	0	0	0	0	0
society	0	0	0.0556	0.0185	0.0254
ambition	0	0	0	0	0

ขั้นที่ 2 — แทนค่ามิติ mind

$$c_{u,\text{mind}} = \frac{0.5944+0.9424+0.0955}{3} = 0.5441 \qquad \| c_u \|_2 = 0.7301 \qquad p_{u,\text{mind}} = 0.7452$$

ขั้นที่ 3 — ตีความ

โปรไฟล์ความหมายของ u1 คือ mind (0.7452) + space (0.4494) + tech (0.3826) และมี action, family, crime ในระดับรอง ต่างจากปริภูมิแนวที่บอกเพียง “Sci-Fi” ปริภูมิความหมายบอกได้ละเอียดกว่าว่า u1 ชอบเรื่อง “ความจริงและจิตใจ” ซึ่งจะทำให้คำแนะนำในหัวข้อ 3.9 ต่างออกไปอย่างน่าสนใจ

3.8.3 แบบที่ 2 ถ่วงน้ำหนักด้วยคะแนน: ใช้ทั้งสิ่งที่ชอบและไม่ชอบ

ผู้ใช้ส่วนใหญ่ไม่ได้บอกเพียง “ชอบ” หรือ “ไม่ชอบ” แต่ให้คะแนนหลายระดับ วิธีขยายที่ตรงไปตรงมาคือให้แต่ละรายการมีน้ำหนัก $w_i = r_{ui} - \bar{r}$ โดย $\bar{r}$ คือคะแนนกลาง (บทนี้ใช้ 3) รายการที่ได้ 5 คะแนนดึงโปรไฟล์ด้วยน้ำหนัก +2 ส่วนรายการที่ได้ 1 คะแนนผลักโปรไฟล์ด้วยน้ำหนัก −2

นิยาม 3.10 โปรไฟล์ถ่วงน้ำหนักด้วยคะแนน

$$c_u^w = \frac{\sum_{i \in I_u} (r_{ui} - \bar{r})\, v_i}{\sum_{i \in I_u} |r_{ui} - \bar{r}|} \tag{3.12}$$

โดย I_u คือรายการทั้งหมดที่ผู้ใช้เคยให้คะแนน (รายการที่ได้คะแนนกลางพอดีมีน้ำหนักศูนย์) มิติที่ติดลบหมายถึง “ยิ่งมีคุณลักษณะนี้ยิ่งไม่น่าแนะนำ” ก่อนนำไปคูณจุดกับเวกเตอร์รายการซึ่งไม่ติดลบ ต้องตัดค่าติดลบเป็นศูนย์ (Clipping) แล้วปรับ L2

ตัวอย่างการคำนวณ 3.14 โปรไฟล์ถ่วงน้ำหนักด้วยคะแนนของ u1

ขั้นที่ 1 — ข้อมูลเข้า: น้ำหนักของห้าเรื่องที่ u1 ให้คะแนน

ตารางที่ 3.39 น้ำหนัก $w_i = r_{ui} - 3$ ของแต่ละเรื่อง และผลรวมขนาดน้ำหนัก

รายการ	i1	i2	i3	i4	i6	$\sum \lvert w_i \rvert$
คะแนน r_{ui}	5	5	4	1	1	
น้ำหนัก w_i	+2	+2	+1	−2	−2	9

ขั้นที่ 2 — ปริภูมิแนวภาพยนตร์: ผลรวมถ่วงน้ำหนักรายมิติ หาร 9 แล้วตัดค่าติดลบ

ตารางที่ 3.40 โปรไฟล์ถ่วงน้ำหนักของ u1 ในปริภูมิแนวภาพยนตร์

มิติ	i1 (+2)	i2 (+2)	i3 (+1)	i4 (−2)	i6 (−2)	ผลรวมถ่วง	หาร 9	หลังตัด+L2
Action	1	0	0	0	0	+2	0.2222	0.3482
Drama	0	0	1	1	1	1 − 4 = −3	-0.3333	0
Romance	0	0	0	1	1	−4	-0.4444	0
Sci-Fi	1	1	1	0	0	2 + 2 + 1 = 5	0.5556	0.8704
Thriller	0	1	0	0	0	+2	0.2222	0.3482
มิติอื่น	0	0	0	0	0	0	0	0

ขั้นที่ 3 — ปริภูมิความหมาย: สูตรเดียวกันกับเวกเตอร์ 12 มิติ

ตารางที่ 3.41 โปรไฟล์ถ่วงน้ำหนักของ u1 ในปริภูมิความหมาย เทียบกับค่าเฉลี่ยศูนย์กลาง

มิติ	ค่าเฉลี่ยศูนย์กลาง p_u	ถ่วงน้ำหนัก (ก่อนตัด)	ถ่วงน้ำหนัก (หลังตัด + L2)
tech	0.3826	0.1862	0.5357
mind	0.7452	0.2648	0.7618
space	0.4494	0.1145	0.3294
romance	0.0254	-0.3850	0
crime	0.1410	0.0522	0.1502
action	0.2224	-0.0205	0
family	0.1605	0.0144	0.0413
music	0	0	0
fantasy	0	0	0
toys	0	0	0
society	0.0254	-0.0759	0
ambition	0	-0.0164	0

ขั้นที่ 4 — ตีความ

ในปริภูมิแนว Romance กลายเป็นค่าติดลบ (-0.4444) ซึ่งแปลว่า “ยิ่งเป็น Romance ยิ่งไม่น่าแนะนำ” และ Drama ก็ติดลบด้วยแม้ Interstellar ที่ชอบจะเป็น Drama เพราะสองเรื่องที่ไม่ชอบเป็น Drama ทั้งคู่และมีน้ำหนักรวมมากกว่า ในปริภูมิความหมาย มิติ romance ติดลบชัดเจน (-0.3850) และ society,

action ติดลบเล็กน้อยเพราะ Titanic มีคำอย่าง class, penniless, sinks หลังตัดค่าติดลบ โปรไฟล์จึงเน้น mind, tech, space มากขึ้นกว่าเดิม ค่าติดลบมีความหมายเมื่อจะใช้ตัดรายการออก แต่การคุมสัดส่วนระหว่างหลักฐานบวกกับลบยังไม่ยืดหยุ่น ซึ่งแบบที่ 3 จะแก้

3.8.4 แบบที่ 3 วิธีร็อกคีโย: ดึงเข้าหาสิ่งที่ชอบ ผลักออกจากสิ่งที่ไม่ชอบ

วิธีร็อกคีโย (Rocchio Method) เสนอไว้สำหรับปรับปรุงข้อคำถามในระบบค้นคืนสารสนเทศ แนวคิดคือเลื่อนโปรไฟล์เข้าหาจุดกึ่งกลางของสิ่งที่ชอบด้วยน้ำหนัก β และเลื่อนออกจากจุดกึ่งกลางของสิ่งที่ไม่ชอบด้วยน้ำหนัก γ ผู้ออกแบบจึงกำหนดได้เองว่าจะเชื่อหลักฐานเชิงลบมากเพียงใด

นิยาม 3.11 วิธีร็อกคีโย

$$q_u^{(m+1)} = \alpha\, q_u^{(m)} + \beta \cdot \frac{1}{|D^+|}\sum_{d\in D^+} v_d - \gamma \cdot \frac{1}{|D^-|}\sum_{d\in D^-} v_d \qquad (3.13)$$

โดย $q_u^{(m)}$ คือโปรไฟล์เดิม (รอบแรกเป็นเวกเตอร์ศูนย์) D^+ และ D^- คือเซตของรายการที่ชอบและไม่ชอบ ค่าที่ได้ต้องตัดค่าติดลบเป็นศูนย์แล้วปรับ L2 เสมอ

ตารางที่ 3.42 บทบาทของพารามิเตอร์ทั้งสามของร็อกคีโย

พารามิเตอร์	ความหมาย	ช่วงที่แนะนำ	ผลเมื่อเพิ่มค่า
α	น้ำหนักของโปรไฟล์เดิม	0.8 – 1.0	โปรไฟล์เปลี่ยนช้า ทนต่อผลป้อนกลับที่ผิดพลาด แต่ปรับตัวช้า
β	น้ำหนักของหลักฐานเชิงบวก	0.6 – 1.0	เอนไปตามสิ่งที่เพิ่งชอบ
γ	น้ำหนักของหลักฐานเชิงลบ	0.1 – 0.3	ตัดสิ่งที่ไม่ชอบแรงขึ้น สูงเกินไปจะฆ่าทั้งหมวดจากการกดไม่ชอบครั้งเดียว

ตัวอย่างการคำนวณ 3.15 โปรไฟล์ร็อกคีโยของ u1 ($\alpha = 1, \beta = 0.8, \gamma = 0.2, q^{(0)} = 0$)

ขั้นที่ 1 — ข้อมูลเข้า: ค่าเฉลี่ยฝั่งชอบ $c(D^+)$ จาก i1, i2, i3 และค่าเฉลี่ยฝั่งไม่ชอบ $c(D^-)$ จาก i4, i6 (ทั้งคู่เป็น Drama + Romance)

ขั้นที่ 2 — ปริภูมิแนวภาพยนตร์: แทนค่าทีละมิติ

ตารางที่ 3.43 การคำนวณร็อกคีโยของ u1 ในปริภูมิแนวภาพยนตร์

มิติ	$c(D^+)$	$c(D^-)$	$0.8\,c(D^+)$	$0.2\,c(D^-)$	ก่อนตัด	หลังตัด	p_u (L2)
Action	0.3333	0.0000	0.2666	0.0000	0.2667	0.2667	0.3007
Animation	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Drama	0.3333	1.0000	0.2666	0.2000	0.0667	0.0667	0.0752
Family	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Fantasy	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

มิติ	$c(D^+)$	$c(D^-)$	$0.8\,c(D^+)$	$0.2\,c(D^-)$	ก่อนตัด	หลังตัด	p_u (L2)
Musical	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Romance	0.0000	1.0000	0.0000	0.2000	-0.2000	0.0000	0.0000
Sci-Fi	1.0000	0.0000	0.8000	0.0000	0.8000	0.8000	0.9020
Thriller	0.3333	0.0000	0.2666	0.0000	0.2667	0.2667	0.3007

$$q_{\text{Drama}} = 0.8 \times 0.3333 - 0.2 \times 1.0000 = 0.0667$$

$$q_{\text{Romance}} = 0.8 \times 0 - 0.2 \times 1.0000 = -0.2000 \rightarrow 0$$

$$\| q \|_2 = \sqrt{0.2667^2 + 0.0667^2 + 0.8000^2 + 0.2667^2} = 0.8869$$

ขั้นที่ 3 — ปริภูมิความหมาย: สูตรเดียวกัน

ตารางที่ 3.44 ร็อกคีโยของ u1 ในปริภูมิความหมาย

มิติ	$c(D^+)$	$c(D^-)$	$0.8c^+ - 0.2c^-$	หลังตัด	p_u (L2)
tech	0.2793	0	0.2235	0.2235	0.4122
mind	0.5441	0.1964	0.3960	0.3960	0.7304
space	0.3281	0	0.2625	0.2625	0.4841
romance	0.0185	0.8801	-0.1612	0	0
crime	0.1030	0.0370	0.0750	0.0750	0.1383
action	0.1624	0.2481	0.0803	0.0803	0.1481
family	0.1172	0.0740	0.0790	0.0790	0.1456
music	0	0	0	0	0
fantasy	0	0	0	0	0
toys	0	0	0	0	0
society	0.0185	0.1846	-0.0221	0	0
ambition	0	0.0369	-0.0074	0	0

ขั้นที่ 4 — ปริภูมิ TF-IDF: มิติที่เปลี่ยนมากที่สุด

ตารางที่ 3.45 ร็อกคีโยของ u1 ในปริภูมิ TF-IDF: คำที่ถูกผลักลงมากที่สุด และคำที่ปรากฏทั้งสองฝั่ง

คำ	$c(D^+)$	$c(D^-)$	$0.8c^+$	$0.2c^-$	ก่อนตัด	หลังตัด
love	0.0000	0.3935	0.0000	0.0787	-0.0787	0.0000
woman	0.0000	0.1790	0.0000	0.0358	-0.0358	0.0000
aboard	0.0000	0.1065	0.0000	0.0213	-0.0213	0.0000
artist	0.0000	0.1065	0.0000	0.0213	-0.0213	0.0000
class	0.0000	0.1065	0.0000	0.0213	-0.0213	0.0000
doomed	0.0000	0.1065	0.0000	0.0213	-0.0213	0.0000
memory	0.0508	0.1998	0.0407	0.0400	0.0007	0.0007
reality	0.1950	0.0000	0.1560	0.0000	0.1560	0.1560
dream	0.1911	0.0000	0.1529	0.0000	0.1529	0.1529

ขั้นที่ 5 — ตีความ

ในปริภูมิแนว Drama ไม่ได้กลายเป็นศูนย์แม้ u1 กดไม่ชอบภาพยนตร์ Drama ถึงสองเรื่อง เพราะ Interstellar ที่ชอบก็เป็น Drama หลักฐานบวกจึงหักล้างหลักฐานลบเหลือ 0.0667 ส่วน Romance ถูกตัดเป็นศูนย์ ในปริภูมิ TF-IDF คำที่อยู่เฉพาะในเรื่องที่ไม่ชอบ เช่น love, woman ถูกตัดทิ้งทั้งหมด (เท่ากับค่าเฉลี่ยศูนย์กลางอยู่แล้ว) ส่วน memory ซึ่งอยู่ทั้งใน Inception ที่ชอบและ The Notebook ที่ไม่ชอบ ถูกลดจาก 0.0407 เหลือ 0.0007 แต่ไม่หายไป **ในปริภูมิเบาบางแบบ TF-IDF หลักฐานเชิงลบจึงมีผลจำกัดกว่าในปริภูมิแนวและปริภูมิความหมาย** ซึ่งมิติ romance ถูกกดจาก 0.0185 จนติดลบ

ประเด็นสำคัญ กฎเหล็กสองข้อของร็อกคีโย

ข้อแรก ต้องให้ $\beta > \gamma$ เสมอ (อัตราส่วน 3:1 ถึง 5:1) เพราะหลักฐานเชิงบวกบอกว่าผู้ใช้ *ต้องการอะไร* ซึ่งชี้ทิศทางได้ ส่วนหลักฐานเชิงลบบอกเพียงว่า *ไม่เอาสิ่งนี้* โดยไม่บอกว่าควรไปทางใดแทน **ข้อที่สอง ต้องตัดค่าติดลบเป็นศูนย์** ในปริภูมิ 0/1 และ TF-IDF เพราะเราไม่สามารถ “มีคำว่า Romance เป็นจำนวนลบ” ได้ ถ้าปล่อยให้ค่าติดลบผ่านไปถึงโคไซน์ จะเกิดคะแนนติดลบที่ตีความไม่ได้

3.8.5 แบบที่ 4 ลดทอนตามเวลา: สิ่งที่เพิ่งชอบสำคัญกว่า

รสนิยมเปลี่ยนตามเวลา ผู้ที่เพิ่งดู Interstellar เมื่อสัปดาห์ก่อน น่าจะอยากได้คำแนะนำที่ใกล้ Interstellar มากกว่า The Matrix ที่ดูเมื่อสามเดือนก่อน **โปรไฟล์ลดทอนตามเวลา (Time-Decayed Profile)** ให้แต่ละรายการมีน้ำหนักลดลงแบบเลขชี้กำลังตามอายุของผลป้อนกลับ

นิยาม 3.12 โปรไฟล์ลดทอนตามเวลา

$$\delta_i = 0.5^{\Delta t_i / h}, \qquad c_u^{\delta} = \frac{\sum_{i \in I_u^+} \delta_i v_i}{\sum_{i \in I_u^+} \delta_i} \tag{3.14}$$

โดย Δt_i คือจำนวนวันตั้งแต่ผู้ใช้ให้ผลป้อนกลับต่อรายการ i และ h คือ **ครึ่งชีวิต (Half-life)** เป็นวัน: รายการที่เก่า h วันมีน้ำหนักเหลือครึ่งหนึ่ง สูตรนี้รวมกับน้ำหนักคะแนนของแบบที่ 2 ได้โดยคูณกัน

ตัวอย่างการคำนวณ 3.16 โปรไฟล์ลดทอนตามเวลาของ u1 ในปริภูมิแนวภาพยนตร์ ($h = 30$ วัน)

ขั้นที่ 1 — ข้อมูลเข้า: สมมติวันที่ u1 ให้คะแนนแต่ละเรื่อง

ตารางที่ 3.46 อายุของผลป้อนกลับและน้ำหนักลดทอน $\delta_i = 0.5^{\Delta t_i/30}$

รายการ	คะแนน	ให้คะแนนเมื่อ (วันก่อน)	$\Delta t_i/30$	δ_i
i1 The Matrix	5	90	3.0000	0.1250
i2 Inception	5	60	2.0000	0.2500
i3 Interstellar	4	7	0.2333	0.8507
i4 Titanic	1	45	1.5000	0.3536
i6 The Notebook	1	3	0.1000	0.9330

ขั้นที่ 2 — เฉลี่ยถ่วงน้ำหนัก δ เฉพาะเรื่องที่ชอบ (i1, i2, i3)

$$\sum \delta_i = 0.1250 + 0.2500 + 0.8507 = 1.2257$$

ตารางที่ 3.47 โปรไฟล์ลดทอนตามเวลาของ u1 เทียบกับค่าเฉลี่ยศูนย์กลาง

มิติ	$0.125\, v_{i1}$	$0.25\, v_{i2}$	$0.8507\, v_{i3}$	ผลรวม	หาร 1.2257	p_u (L2)	ค่าเฉลี่ยศูนย์กลาง p_u
Action	0.1250	0.0000	0.0000	0.1250	0.1020	0.0824	0.2887
Drama	0.0000	0.0000	0.8507	0.8507	0.6940	0.5604	0.2887
Sci-Fi	0.1250	0.2500	0.8507	1.2257	1.0000	0.8075	0.8660
Thriller	0.0000	0.2500	0.0000	0.2500	0.2040	0.1647	0.2887

ขั้นที่ 3 — ผลต่อคะแนนของ CANDIDATE ITEM (โคไซน์กับเวกเตอร์แนว)

ตารางที่ 3.48 คะแนนจับคู่ของ CANDIDATE ITEM ที่คะแนนไม่เป็นศูนย์ เมื่อใช้โปรไฟล์ค่าเฉลี่ยศูนย์กลางเทียบกับโปรไฟล์ลดทอนตามเวลา

รายการ	แนว	ค่าเฉลี่ยศูนย์กลาง	ลดทอนตามเวลา
i8 Mad Max: Fury Road	Action, Sci-Fi	0.8165	0.6292

รายการ	แนว	ค่าเฉลี่ยศูนย์กลาง	ลดทอนตามเวลา
i9 Parasite	Drama, Thriller	0.4082	0.5127
i7 John Wick	Action, Thriller	0.4082	0.1747

ขั้นที่ 4 — ตีความ

Interstellar ซึ่งดูเมื่อ 7 วันก่อนได้น้ำหนัก 0.8507 ส่วน The Matrix ที่ดูเมื่อ 90 วันก่อนเหลือเพียง 0.1250 มิติ Drama ของโปรไฟล์จึงพุ่งจาก 0.2887 เป็น 0.5604 ผลคือ Parasite (Drama, Thriller) ขยับขึ้นมาใกล้ Mad Max และ John Wick (Action, Thriller) ตกลงชัดเจน ค่าครึ่งชีวิตเป็นไฮเพอร์พารามิเตอร์: ระบบข่าวใช้หลักชั่วโมงถึงวัน ระบบภาพยนตร์ใช้หลักเดือน

3.8.6 แบบที่ 5 โปรไฟล์หลายกลุ่มความสนใจ

โปรไฟล์เวกเตอร์เดียวมีสมมติฐานซ่อนอยู่ว่าผู้ใช้มีรสนิยม “กลุ่มเดียว” ผู้ใช้ u6 ชอบ Inception (mind, crime) และ La La Land (romance, music, ambition) ซึ่งแทบไม่มีอะไรร่วมกัน การเฉลี่ยจะได้จุดกึ่งกลางที่ **ไม่เหมือนทั้งสองเรื่อง** ทางแก้คือเก็บโปรไฟล์ไว้หลายเวกเตอร์ (หนึ่งเวกเตอร์ต่อกลุ่มความสนใจ หรือในกรณีสุดโต่งคือหนึ่งเวกเตอร์ต่อรายการที่ชอบ) แล้วให้คะแนน CANDIDATE ITEM ด้วย **ค่าสูงสุด** ของความคล้ายกับแต่ละกลุ่ม

นิยาม 3.13 โปรไฟล์หลายกลุ่มความสนใจ

$$s_{\max}(u,i) = \max_{k} \cos\left(p_u^{(k)}, v_i\right), \qquad p_u^{(k)} = \text{centroid ของกลุ่มที่ } k \tag{3.15}$$

กลุ่มได้จากการจัดกลุ่ม (เช่น k-means) เวกเตอร์ของรายการที่ชอบ ในตัวอย่างนี้ใช้กรณีง่ายที่สุดคือแต่ละรายการที่ชอบเป็นหนึ่งกลุ่ม ซึ่งเท่ากับวิธี Item-kNN แบบค่าสูงสุดในหัวข้อ 3.9.4

ตัวอย่างการคำนวณ 3.17 ผู้ใช้ u6 ในปริภูมิความหมาย: เวกเตอร์เดียวเทียบกับสองเวกเตอร์

ขั้นที่ 1 — ข้อมูลเข้า: เวกเตอร์ความหมายของสองเรื่องที่ u6 ชอบ และค่าเฉลี่ยศูนย์กลาง

ตารางที่ 3.49 โปรไฟล์ของ u6: ค่าเฉลี่ยศูนย์กลางไม่เหมือนทั้ง Inception และ La La Land

มิติ	i2 Inception	i5 La La Land	ค่าเฉลี่ยศูนย์กลาง p_u
tech	0.0820	0	0.0580
mind	0.9424	0	0.6664
space	0.0464	0	0.0328
romance	0	0.3115	0.2202
crime	0.3090	0	0.2185

มิติ	i2 Inception	i5 La La Land	ค่าเฉลี่ยศูนย์กลาง p_u
action	0.0464	0	0.0328
family	0.0736	0	0.0520
music	0	0.2805	0.1983
fantasy	0	0	0
toys	0	0	0
society	0	0.0561	0.0397
ambition	0	0.9062	0.6408

ขั้นที่ 2 — คะแนน CANDIDATE ITEM สองแบบ: โคไซน์กับค่าเฉลี่ยศูนย์กลาง เทียบกับค่าสูงสุดของโคไซน์กับแต่ละเรื่อง

ตารางที่ 3.50 คะแนน CANDIDATE ITEM ของ u6 แบบเวกเตอร์เดียว เทียบกับแบบค่าสูงสุด (เรียงตามแบบเวกเตอร์เดียว)

รายการ	cos กับ i2	cos กับ i5	เวกเตอร์เดียว (centroid)	ค่าสูงสุด (max)
i6 The Notebook	0.4131	0.2747	0.4864	0.4131
i1 The Matrix	0.6348	0.0000	0.4489	0.6348
i4 Titanic	0.0137	0.3611	0.2650	0.3611
i9 Parasite	0.2705	0.0387	0.2186	0.2705
i7 John Wick	0.3088	0.0000	0.2184	0.3088
i12 Spirited Away	0.1827	0.0000	0.1292	0.1827

ขั้นที่ 3 — ตีความ

แบบเวกเตอร์เดียวให้ The Notebook เป็นอันดับหนึ่ง (0.4864) เพราะมันมีทั้ง memory (mind) และ love (romance) จึงอยู่ใกล้ “จุดกึ่งกลาง” ของสองรสนิยมพอดี ทั้งที่ไม่ค่อยเหมือนเรื่องใดเรื่องหนึ่งที่ u6 ชอบ แบบค่าสูงสุดให้ The Matrix เป็นอันดับหนึ่ง (0.6348) เพราะคล้าย Inception มาก แม้ไม่คล้าย La La Land เลย แบบค่าสูงสุดจึงเหมาะกับผู้ใช้ที่มีรสนิยมหลายกลุ่มซึ่งไม่ควรถูกเฉลี่ยรวมกัน แลกกับต้นทุนที่ต้องเก็บหลายเวกเตอร์ต่อผู้ใช้

3.8.7 แบบที่ 6 โปรไฟล์แบบเรียนรู้ได้: ตัวจำแนกนาอีฟเบส์

ทุกแบบข้างต้นสร้างโปรไฟล์ด้วย **สูตรคงที่** เมื่อผู้ใช้มีผลป้อนกลับทั้งบวกและลบมากพอ เราอาจให้แบบจำลอง **เรียนรู้** ว่าคุณลักษณะใดทำนายความชอบได้ดี วิธีที่ง่ายที่สุดคือ **ตัวจำแนกนาอีฟเบส์ (Naive Bayes)** ซึ่งเปลี่ยนโปรไฟล์จากเวกเตอร์เป็น **ความน่าจะเป็น** $P(\text{ชอบ} \mid v_i)$ ที่ตั้งเกณฑ์ตัดสินใจได้

นิยาม 3.14 นาอีฟเบส์แบบแบร์นูลลีบนคุณลักษณะ 0/1

$$P(x_g = 1 \mid \text{ชอบ}) = \frac{n_g^+ + 1}{n^+ + 2}, \qquad P(\text{ชอบ} \mid v_i) = \frac{P(\text{ชอบ}) \prod_g P(x_g \mid \text{ชอบ})}{P(\text{ชอบ}) \prod_g P(x_g \mid \text{ชอบ}) + P(\text{ไม่ชอบ}) \prod_g P(x_g \mid \text{ไม่ชอบ})} \tag{3.16}$$

โดย n_g^+ คือจำนวนเรื่องที่ชอบซึ่งมีคุณลักษณะ g และ n^+ คือจำนวนเรื่องที่ชอบทั้งหมด การบวก 1 และ 2 คือการปรับเรียบแบบลาปลาซ (Laplace Smoothing) เพื่อไม่ให้ความน่าจะเป็นใดเป็นศูนย์ คำว่า “นาอีฟ” มาจากสมมติฐานว่าแต่ละคุณลักษณะเป็นอิสระต่อกันเมื่อรู้คลาส

ตัวอย่างการคำนวณ 3.18 นาอีฟเบส์ของ u1 บนแนวภาพยนตร์ 9 มิติ และความน่าจะเป็นที่จะชอบ i8 Mad Max

ขั้นที่ 1 — ข้อมูลเข้า: นับคุณลักษณะในฝั่งชอบ ($n^+ = 3$) และฝั่งไม่ชอบ ($n^- = 2$) แล้วประมาณความน่าจะเป็นแบบปรับเรียบ

ตารางที่ 3.51 ความน่าจะเป็นของแต่ละแนวในฝั่งชอบและไม่ชอบ (ลาปลาซ)

แนว g	n_g^+	$P(x_g = 1 \mid \text{ชอบ})$	n_g^-	$P(x_g = 1 \mid \text{ไม่ชอบ})$
Action	1	(1+1)/5 = 0.4000	0	(0+1)/4 = 0.2500
Animation	0	(0+1)/5 = 0.2000	0	(0+1)/4 = 0.2500
Drama	1	(1+1)/5 = 0.4000	2	(2+1)/4 = 0.7500
Family	0	(0+1)/5 = 0.2000	0	(0+1)/4 = 0.2500
Fantasy	0	(0+1)/5 = 0.2000	0	(0+1)/4 = 0.2500
Musical	0	(0+1)/5 = 0.2000	0	(0+1)/4 = 0.2500
Romance	0	(0+1)/5 = 0.2000	2	(2+1)/4 = 0.7500
Sci-Fi	3	(3+1)/5 = 0.8000	0	(0+1)/4 = 0.2500
Thriller	1	(1+1)/5 = 0.4000	0	(0+1)/4 = 0.2500

ความน่าจะเป็นก่อนหน้า: $P(\text{ชอบ}) = 3/5 = 0.6000$ และ $P(\text{ไม่ชอบ}) = 2/5 = 0.4000$

ขั้นที่ 2 — สำหรับ i8 Mad Max (Action = 1, Sci-Fi = 1, ที่เหลือ 0) คูณความน่าจะเป็นของทุกมิติ (มิติที่เป็น 0 ใช้ $1 - P$)

ตารางที่ 3.52 พจน์รายมิติของ i8 Mad Max ในแต่ละคลาส

แนว	x_g ของ i8	พจน์ฝั่งชอบ	พจน์ฝั่งไม่ชอบ
Action	1	0.4000	0.2500
Animation	0	0.8000	0.7500
Drama	0	0.6000	0.2500
Family	0	0.8000	0.7500
Fantasy	0	0.8000	0.7500
Musical	0	0.8000	0.7500
Romance	0	0.8000	0.2500
Sci-Fi	1	0.8000	0.2500
Thriller	0	0.6000	0.7500
ผลคูณ × ก่อนหน้า		0.6 × ... = 0.0226	0.4 × ... = 0.0004

$$P(\text{ชอบ} \mid i8) = \frac{0.0226}{0.0226+0.0004} = 0.9839$$

ขั้นที่ 3 — ทำเช่นเดียวกันกับ CANDIDATE ITEM ทุกเรื่อง

ตารางที่ 3.53 ความน่าจะเป็นที่ u1 จะชอบ CANDIDATE ITEM แต่ละเรื่องจากนาอีฟเบส์ (เรียงจากมากไปน้อย)

รายการ	แนว	$P(\text{ชอบ} \mid v_i)$	ตัดสินที่เกณฑ์ 0.5
i8 Mad Max: Fury Road	Action, Sci-Fi	0.9839	แนะนำ
i7 John Wick	Action, Thriller	0.9106	แนะนำ
i10 Toy Story	Animation, Family	0.5888	แนะนำ
i11 Coco	Animation, Family	0.5888	แนะนำ
i12 Spirited Away	Animation, Fantasy	0.5888	แนะนำ
i9 Parasite	Drama, Thriller	0.5308	แนะนำ
i5 La La Land	Musical, Romance	0.1372	ไม่แนะนำ

ขั้นที่ 4 — ตีความ

Mad Max ได้ 0.9839 เพราะ Sci-Fi ปรากฏใน 3 จาก 3 เรื่องที่ชอบ และไม่มีมิติใดที่ฝั่งไม่ชอบเด่น ส่วน La La Land ได้ 0.1372 เพราะ Romance ปรากฏใน 2 จาก 2 เรื่องที่ไม่ชอบ ข้อได้เปรียบเหนือโปรไฟล์แบบเวกเตอร์คือคำตอบเป็นความน่าจะเป็นที่ตั้งเกณฑ์ได้ (เช่น แนะนำเมื่อเกิน 0.5)

ข้อจำกัดคือต้องมีผลป้อนกลับทั้งสองฝั่ง และการประมาณจากตัวอย่างเพียง 5 เรื่องมีความแปรปรวนสูงมาก สังเกตว่าแอนิเมชันทั้งสามเรื่องได้ 0.5888 เท่ากันพอดี เพราะไม่มีมิติใดของมันเคยปรากฏในข้อมูลของ u1 เลย เมื่อมีข้อมูลหลักร้อยรายการขึ้นไป **การถดถอยโลจิสติก (Logistic Regression)** ซึ่งเรียนน้ำหนักรายมิติโดยตรงมักให้ผลดีกว่า

3.8.8 โปรไฟล์ในบทบาทข้อคำถาม: ใช้กับ BM25

BM25 (หัวข้อ 3.4.6) รับข้อคำถามเป็นชุดคำ ไม่ใช่เวกเตอร์ การใช้โปรไฟล์กับ BM25 จึงทำโดยเลือก **คำเด่นของโปรไฟล์** k อันดับแรกมาเป็นข้อคำถาม แล้วให้คะแนนทุกรายการด้วยสมการ (3.5)

ตัวอย่างการคำนวณ 3.19 ข้อคำถามจากโปรไฟล์ TF-IDF ของ u1 และคะแนน BM25 ของ CANDIDATE ITEM

ขั้นที่ 1 — ข้อมูลเข้า: ห้าคำเด่นของโปรไฟล์จากตารางที่ 3.37

q_{u1} = {reality, dream, world, time, space}

ขั้นที่ 2 — คะแนน BM25 รายคำของทุกเรื่อง ($k_1 = 1.5, b = 0.75$)

ตารางที่ 3.54 คะแนน BM25 ของข้อคำถามจากโปรไฟล์ u1 (เฉพาะเรื่องที่ได้คะแนนไม่เป็นศูนย์)

รายการ	reality	dream	world	time	space	รวม	สถานะ
i2 Inception	1.5505	3.0951	1.2341	1.5505	·	7.4302	เคยดู
i3 Interstellar	·	·	·	2.3184	2.7142	5.0326	เคยดู
i1 The Matrix	2.6544	·	2.1127	·	·	4.7671	เคยดู
i5 La La Land	·	2.3524	·	·	·	2.3524	CANDIDATE ITEM
i12 Spirited Away	·	·	2.1849	·	·	2.1849	CANDIDATE ITEM
i10 Toy Story	·	·	·	·	1.6803	1.6803	CANDIDATE ITEM

ขั้นที่ 3 — ตีความ

สามเรื่องที่ u1 ชอบได้คะแนนสูงสุด (ตรวจสอบความถูกต้องผ่าน) ในหมู่ CANDIDATE ITEM La La Land นำด้วยคำว่า dream ตามด้วย Spirited Away (world) และ Toy Story (space ranger) ซึ่งเป็นความผิดพลาดชนิดเดียวกับปริภูมิ TF-IDF เพราะ BM25 ก็ยังเป็นการจับคู่คำที่ตรงกัน ข้อดีของวิธีนี้คือใช้ดัชนีผกผันของระบบค้นหาที่มีอยู่แล้วได้ทันที โดยไม่ต้องเก็บเวกเตอร์ 203 มิติต่อรายการ

3.8.9 เลือกแบบใด: บันไดของการสร้างโปรไฟล์

ตารางที่ 3.55 หกแบบของโปรไฟล์ผู้ใช้ ข้อมูลที่ต้องมี และสถานการณ์ที่เหมาะ

แบบ	ข้อมูลที่ต้องมี	จุดแข็ง	จุดอ่อน	เหมาะเมื่อ
1 ค่าเฉลี่ยศูนย์กลาง	รายการที่ชอบ ≥ 1	ง่าย ทำงานได้ตั้งแต่ชอบเรื่องเดียว	ไม่ใช้หลักฐานลบ	ผู้ใช้ใหม่ ข้อมูลน้อย
2 ถ่วงน้ำหนักด้วยคะแนน	คะแนนหลายระดับ	ใช้ทั้งชอบและไม่ชอบ	คุมสัดส่วนบวก/ลบไม่ได้	มีคะแนน 1–5
3 ร็อกคีโย	ชอบและไม่ชอบ	ปรับ β, γ ได้ อัปเดตทีละรอบได้	ต้องตัดค่าติดลบ	มีผลป้อนกลับต่อเนื่อง
4 ลดทอนตามเวลา	เวลาของผลป้อนกลับ	ตามรสนิยมที่เปลี่ยน	ต้องเลือกครึ่งชีวิต	ข่าว สินค้าตามฤดู
5 หลายกลุ่มความสนใจ	รายการที่ชอบหลายเรื่อง	ไม่เฉลี่ยรสนิยมที่ต่างกันจนเบลอ	เก็บหลายเวกเตอร์ต่อผู้ใช้	ผู้ใช้ที่มีหลายรสนิยม
6 เรียนรู้ได้ (NB, LR)	บวกและลบหลายสิบขึ้นไป	ได้ความน่าจะเป็น ตั้งเกณฑ์ได้	ต้องการข้อมูลมาก ฟิตเกินง่าย	ผู้ใช้ที่ใช้งานหนัก

ประเด็นสำคัญ กฎสามข้อของการสร้างโปรไฟล์

ข้อแรก โปรไฟล์ต้องมีมิติเดียวกับเวกเตอร์รายการ และผ่านการปรับความยาวแบบเดียวกันก่อนเปรียบเทียบ
ข้อที่สอง เริ่มจากค่าเฉลี่ยศูนย์กลางเสมอ แล้วจึงเพิ่มน้ำหนักคะแนน หลักฐานลบ หรือเวลา เมื่อมีข้อมูล
ข้อที่สาม กฎการตัดค่าติดลบผูกกับปริภูมิ: ในปริภูมิ 0/1 และ TF-IDF ต้องตัดเสมอ แต่ในปริภูมิเวกเตอร์ฝังจริงที่ค่าติดลบมีความหมาย (ต่างจากคลังแนวคิดจำลองในบทนี้) ห้ามตัด

3.9 การจับคู่โปรไฟล์กับรายการ: คะแนนจับคู่ (Matching Score)

มีโปรไฟล์ p_u และเวกเตอร์รายการ v_i ในปริภูมิเดียวกันแล้ว คำถามถัดไปคือ “ภาพยนตร์เรื่องนี้ตรงกับโปรไฟล์มากเพียงใด” ตัวเลขที่ตอบคำถามนี้เรียกว่า **คะแนนจับคู่ (Matching Score)** และเป็นหัวใจของส่วนกรองและจัดอันดับ เพราะทุกสิ่งที่ตามมา ทั้งการจัดอันดับ การอธิบายผล และการผสานกับระบบอื่น ล้วนทำงานบนคะแนนนี้

3.9.1 ฟังก์ชันจับคู่แบบโคไซน์: จากสองเวกเตอร์สู่หนึ่งตัวเลข

นิยาม 3.15 คะแนนจับคู่แบบโคไซน์

$$s(u,i) = \cos(p_u, v_i) = \frac{p_u \cdot v_i}{\|p_u\|_2 \, \|v_i\|_2} = \sum_t p_{u,t} \; v_{i,t} \tag{3.17}$$

เครื่องหมายเท่ากับตัวสุดท้ายเป็นจริงเมื่อทั้งสองเวกเตอร์ปรับ L2 แล้ว คะแนนจึงเป็น **ผลรวมของส่วนร่วมรายมิติ** $p_{u,t} \; v_{i,t}$ มิติใดจะมีส่วนร่วมก็ต่อเมื่อ **ทั้ง** โปรไฟล์และรายการมีค่าในมิตินั้น มุมมองนี้คือที่มาของคำอธิบายผลในหัวข้อ 3.9.3

3.9.2 ตัวอย่างในสามปริภูมิ: ผู้ใช้เดียวกัน คำตอบต่างกัน

ตัวอย่างการคำนวณ 3.20 คะแนนจับคู่ของ u1 กับทั้ง 12 เรื่องในปริภูมิแนวภาพยนตร์

ขั้นที่ 1 — ข้อมูลเข้า: ค่าเฉลี่ยศูนย์กลาง c_u จากตัวอย่าง 3.11 (Sci-Fi 1.0000, Action = Drama = Thriller = 0.3333, $\| c_u \| = 1.1547$) และเวกเตอร์แนวของทุกเรื่อง ซึ่งมีสองแนวจึงยาว $\sqrt{2} = 1.4142$ เท่ากันหมด

ขั้นที่ 2 — คูณจุดแล้วหารด้วยความยาวทั้งสอง เช่น i8 Mad Max (Action, Sci-Fi)

$$c_u \cdot v_{i8} = 0.3333 \times 1 + 1.0000 \times 1 = 1.3333$$

$$s(u1, i8) = \frac{1.3333}{1.1547 \times 1.4142} = \frac{1.3333}{1.6330} = 0.8165$$

ตารางที่ 3.56 คะแนนจับคู่ของ u1 ในปริภูมิแนวภาพยนตร์ ครบทั้ง 12 เรื่อง

รายการ	แนว	$c_u \cdot v_i$	$\| c_u \| \| v_i \|$	$s(u1, i)$	สถานะ
i1 The Matrix	Action, Sci-Fi	1.3333	1.6330	0.8165	เคยดู (5)
i2 Inception	Sci-Fi, Thriller	1.3333	1.6330	0.8165	เคยดู (5)
i3 Interstellar	Drama, Sci-Fi	1.3333	1.6330	0.8165	เคยดู (4)
i4 Titanic	Drama, Romance	0.3333	1.6330	0.2041	เคยดู (1)
i5 La La Land	Musical, Romance	0.0000	1.6330	0.0000	CANDIDATE ITEM
i6 The Notebook	Drama, Romance	0.3333	1.6330	0.2041	เคยดู (1)
i7 John Wick	Action, Thriller	0.6667	1.6330	0.4082	CANDIDATE ITEM
i8 Mad Max: Fury Road	Action, Sci-Fi	1.3333	1.6330	0.8165	CANDIDATE ITEM
i9 Parasite	Drama, Thriller	0.6667	1.6330	0.4082	CANDIDATE ITEM
i10 Toy Story	Animation, Family	0.0000	1.6330	0.0000	CANDIDATE ITEM
i11 Coco	Animation, Family	0.0000	1.6330	0.0000	CANDIDATE ITEM
i12 Spirited Away	Animation, Fantasy	0.0000	1.6330	0.0000	CANDIDATE ITEM

ขั้นที่ 3 — อ่านผล

สามเรื่องที่ u1 ชอบได้คะแนนสูงสุดเท่ากันที่ 0.8165 ซึ่งเป็นการตรวจสอบความถูกต้องอย่างง่าย (Sanity Check): รายการที่ใช้สร้างโปรไฟล์ **ต้อง** ได้คะแนนสูง ในหมู่ CANDIDATE ITEM Mad Max ได้ 0.8165 เพราะมีแนวเหมือน The Matrix ทุกประการ ตามด้วย John Wick และ Parasite ที่ 0.4082 เท่ากัน ส่วนแอนิเมชันทั้งสามเรื่องและ La La Land ได้ศูนย์เพราะไม่มีแนวร่วมกับโปรไฟล์เลย

ตัวอย่างการคำนวณ 3.21 คะแนนจับคู่ของ u1 ในปริภูมิ TF-IDF: เมื่อคะแนนมาจากคำในเรื่องย่อ

ขั้นที่ 1 — ข้อมูลเข้า: โปรไฟล์ p_u จากตารางที่ 3.37 และเวกเตอร์ TF-IDF ของทุกเรื่อง ทั้งคู่ปรับ L2 แล้ว คะแนนจึงเป็นผลคูณจุดโดยตรง

ตารางที่ 3.57 คะแนนจับคู่ $s(u1, i) = p_u \cdot v_i$ ในปริภูมิ TF-IDF เรียงจากมากไปน้อย

อันดับ	รายการ	$s(u1, i)$	สถานะ	คำในโปรไฟล์ที่ทำให้ได้คะแนน
1	i2 Inception	0.6559	เคยดู (5)	dream, reality, world, time, lost, memory, family
2	i1 The Matrix	0.6084	เคยดู (5)	reality, world, machine
3	i3 Interstellar	0.5875	เคยดู (4)	space, time, lost
4	i12 Spirited Away	0.1620	CANDIDATE ITEM	world (3 ครั้ง), steals, free, family
5	i5 La La Land	0.1268	CANDIDATE ITEM	dream (2 ครั้ง), shared — คำพ้องรูปคนละความหมาย
6	i11 Coco	0.0698	CANDIDATE ITEM	family, memory, fading
7	i9 Parasite	0.0654	CANDIDATE ITEM	family, beneath, hidden
8	i6 The Notebook	0.0456	เคยดู (1)	memory, fading — ไม่ชอบแต่คะแนนไม่เป็นศูนย์
9	i10 Toy Story	0.0455	CANDIDATE ITEM	space (space ranger)
10	i7 John Wick	0.0183	CANDIDATE ITEM	rival
11	i4 Titanic	0.0000	เคยดู (1)	ไม่มีคำร่วมกับโปรไฟล์
12	i8 Mad Max: Fury Road	0.0000	CANDIDATE ITEM	ไม่มีคำร่วมกับโปรไฟล์ **ทั้งที่แนวตรงที่สุด**

ขั้นที่ 2 — อ่านผล: สี่ข้อสังเกตที่สำคัญต่อการออกแบบระบบจริง

หนึ่ง การตรวจสอบความถูกต้องผ่าน: สามเรื่องที่ชอบได้ 0.59–0.66 สูงกว่า CANDIDATE ITEM ทุกเรื่องมาก **สอง** CANDIDATE ITEM อันดับหนึ่งอาจถูกด้วยเหตุผลที่ผิด: Spirited Away ได้ 0.1620 เพราะคำว่า world ปรากฏ 3 ครั้ง (spirit world, bathhouse world, human world) ซึ่งเป็นคำสำคัญของ The Matrix เช่นกัน **สาม** คำพ้องรูปหลอกระบบได้: La La Land ได้อันดับสองจาก dream **สี่** คะแนนศูนย์ไม่ได้แปลว่าไม่น่าสนใจ: Mad Max ตรงกับรสนิยมของ u1 ที่สุดในปริภูมิแนว แต่ได้ 0.0000 ในปริภูมิคำเพราะเรื่องย่อไม่มีคำใดร่วมกับสามเรื่องที่ชอบเลย

ตัวอย่างการคำนวณ 3.22 คะแนนจับคู่ของ u1 ในปริภูมิความหมาย 12 มิติ

ขั้นที่ 1 — ข้อมูลเข้า: โปรไฟล์ความหมาย p_u จากตารางที่ 3.38 และเวกเตอร์ความหมายจากตารางที่ 3.32

ขั้นที่ 2 — ผลคูณจุดรายมิติ เช่น i8 Mad Max

$$s(u1, i8) = p_{u,\text{action}}\, v_{i8,\text{action}} + p_{u,\text{society}}\, v_{i8,\text{society}} = 0.2224 \times 0.9942 + 0.0254 \times 0.1079 = 0.2239$$

ตารางที่ 3.58 คะแนนจับคู่ของ u1 ในปริภูมิความหมาย ครบทั้ง 12 เรื่อง เรียงจากมากไปน้อย

อันดับ	รายการ	$s(u1, i)$	สถานะ	มิติที่ทำให้ได้คะแนน (สัดส่วน)
1	i2 Inception	0.8202	เคยดู (5)	mind, crime, tech
2	i1 The Matrix	0.7931	เคยดู (5)	tech, mind, action
3	i3 Interstellar	0.5771	เคยดู (4)	space, family, mind
4	i11 Coco	0.4032	CANDIDATE ITEM	mind (memory), family
5	i6 The Notebook	0.3939	เคยดู (1)	mind (memory), romance
6	i9 Parasite	0.2734	CANDIDATE ITEM	family 37.09% · mind 34.92% · crime 16.78%
7	i7 John Wick	0.2343	CANDIDATE ITEM	crime 54.98% · action 37.06% · family 7.96%
8	i8 Mad Max: Fury Road	0.2239	CANDIDATE ITEM	action 98.78% · society 1.22%
9	i12 Spirited Away	0.1536	CANDIDATE ITEM	mind 60.21% · crime 15.49% · family 15.31%
10	i10 Toy Story	0.1065	CANDIDATE ITEM	family, toys
11	i4 Titanic	0.0973	เคยดู (1)	romance, society
12	i5 La La Land	0.0093	CANDIDATE ITEM	romance 84.74% · society 15.26%

ขั้นที่ 3 — อ่านผล

คำตอบเปลี่ยนอีกครั้ง: CANDIDATE ITEM อันดับหนึ่งคือ Coco (0.4032) เพราะโปรไฟล์ของ u1 มีมิติ mind สูงมาก (0.7452 จาก reality, dream, memory, illusion) และ Coco พูดถึง memory ถึงสองครั้ง ตามด้วย Parasite (family, mind, crime) John Wick (crime, action) และ Mad Max (action) ที่ 0.2239 La La Land ตกไปอยู่ท้ายสุด (0.0093) เพราะ dream ของมันถูกตีความเป็น ambition แล้ว และ Mad Max ไม่เป็นศูนย์อีกต่อไป นี่คือทั้งจุดแข็งและข้อควรระวังของปริภูมิความหมาย: ระบบเห็น “แก่นเรื่อง” (ความทรงจำ ความจริง) ได้ลึกกว่าคำ แต่ก็อาจแนะนำแอนิเมชันครอบครัวให้แฟน Sci-Fi ด้วยเหตุผลที่ฟังดูดีแต่ผู้ใช้อาจไม่เห็นด้วย

ประเด็นสำคัญ บทเรียนที่สำคัญที่สุดของหัวข้อนี้

ผู้ใช้คนเดียวกัน โปรไฟล์วิธีเดียวกัน ฟังก์ชันจับคู่เดียวกัน แต่สามปริภูมิให้ CANDIDATE ITEM อันดับหนึ่งคนละคน: **Mad Max** (แนว) **Spirited Away** (คำ) และ **Coco** (ความหมาย) **การเลือกวิธีแทนรายการเปลี่ยนคำแนะนำได้มากกว่าการเลือกฟังก์ชันจับคู่หรือวิธีจัดอันดับ** ซึ่งสอดคล้องกับข้อสรุปในหัวข้อ 3.1.2 ระบบจริงจึงมักใช้หลายปริภูมิร่วมกันด้วยการต่อเวกเตอร์หรือผสานคะแนน (หัวข้อ 3.12)

3.9.3 อธิบายผล: มิติใดทำให้คะแนนสูง

จุดแข็งของระบบอิงเนื้อหาคืออธิบายได้ว่า “แนะนำเรื่องนี้เพราะอะไร” และคำอธิบายนั้นได้มาฟรีจากสมการ (3.17) เพราะคะแนนคือผลรวมของส่วนร่วมรายมิติ $p_{u,t}\, v_{i,t}$ การเรียงส่วนร่วมจากมากไปน้อยจึงบอกได้ทันทีว่าคำใดหรือมิติใดมีอิทธิพลต่อคะแนน

ตัวอย่างการคำนวณ 3.23 แยกคะแนนของ CANDIDATE ITEM สามอันดับแรกของ u1 (ปริภูมิ TF-IDF) เป็นรายคำ

ขั้นที่ 1 — ข้อมูลเข้า: ค่าของโปรไฟล์ $p_{u,t}$ และค่าของรายการ $v_{i,t}$ ในทุกมิติที่ทั้งคู่ไม่เป็นศูนย์

ตารางที่ 3.59 ส่วนร่วมรายคำ $p_{u,t}\, v_{i,t}$ ของคะแนนจับคู่ สำหรับ CANDIDATE ITEM สามอันดับแรกของ u1

รายการ (คะแนน)	คำ t	$p_{u,t}$ โปรไฟล์	$v_{i,t}$ รายการ	ส่วนร่วม	สัดส่วน
i12 Spirited Away (0.1620)	world	0.2790	0.4227	0.1180	72.81%
	steals	0.0932	0.1851	0.0173	10.65%
	free	0.0861	0.1851	0.0159	9.84%
	family	0.0739	0.1468	0.0108	6.70%
i5 La La Land (0.1268)	dream	0.3096	0.3561	0.1103	86.92%
	shared	0.0932	0.1780	0.0166	13.08%
i11 Coco (0.0698)	family	0.0739	0.3868	0.0286	40.97%
	memory	0.0823	0.3334	0.0275	39.35%
	fading	0.0823	0.1667	0.0137	19.68%

ขั้นที่ 2 — ตรวจสอบ: ผลรวมของส่วนร่วมต้องเท่ากับคะแนน

$$s(u1, i12) = 0.1180 + 0.0173 + 0.0159 + 0.0108 = 0.1620$$

ขั้นที่ 3 — ตีความ

ระบบเขียนคำอธิบายให้ผู้ใช้ได้ทันที เช่น “แนะนำ Coco เพราะเกี่ยวกับ family และ memory เหมือน Inception ที่คุณชอบ” และตารางนี้ยังเป็นเครื่องมือตรวจสอบระบบชั้นดี: เมื่อเห็นว่า 72.8% ของคะแนน Spirited Away มาจากคำเดียว ผู้ออกแบบจะรู้ทันทีว่าคำแนะนำนี้เปราะบาง ถ้าเรื่องย่อเขียนใหม่โดยใช้ realm แทน world คะแนนจะหายไปเกือบทั้งหมด ในปริภูมิความหมายก็อธิบายได้แบบเดียวกันเป็นรายมิติ (คอลัมน์สุดท้ายของตารางที่ 3.58) แต่ต้องแปลชื่อมิติแฝงให้ผู้ใช้เข้าใจ

3.9.4 ทางเลือกที่สอง: เทียบทีละรายการแล้วจึงรวม (Item-kNN)

การสร้างโปรไฟล์คือ “รวมก่อนแล้วเทียบ” ยังมีอีกทางคือ “เทียบก่อนแล้วรวม”: นำ CANDIDATE ITEM ไปเทียบกับรายการที่ผู้ใช้ชอบ **ทีละเรื่อง** ด้วยความคล้ายระหว่างรายการ แล้วรวมค่าเหล่านั้นเป็นคะแนนเดียว วิธีนี้เรียกว่า **Item-kNN**

นิยาม 3.16 คะแนนแบบ Item-kNN

$$s_{\text{mean}}(u,i) = \frac{1}{|I_u^+|}\sum_{j\in I_u^+}\cos\left(v_i, v_j\right), \qquad s_{\max}(u,i) = \max_{j\in I_u^+}\cos\left(v_i, v_j\right) \tag{3.18}$$

ตัวอย่างการคำนวณ 3.24 Item-kNN ของ CANDIDATE ITEM ทั้งเจ็ดของ u1 ในปริภูมิความหมาย

ขั้นที่ 1 — ข้อมูลเข้า: โคไซน์ระหว่าง CANDIDATE ITEM แต่ละเรื่องกับ i1, i2, i3 (จากเวกเตอร์ในตารางที่ 3.32)

ตารางที่ 3.60 คะแนน Item-kNN แบบเฉลี่ยและแบบสูงสุด เทียบกับคะแนนอิงโปรไฟล์ (ปริภูมิความหมาย)

รายการ	cos กับ i1	cos กับ i2	cos กับ i3	เฉลี่ย	สูงสุด	อิงโปรไฟล์
i11 Coco	0.2347	0.4211	0.2274	0.2944	0.4211	0.4032
i9 Parasite	0.0924	0.2705	0.2360	0.1996	0.2705	0.2734
i7 John Wick	0.1070	0.3088	0.0974	0.1711	0.3088	0.2343
i8 Mad Max: Fury Road	0.2725	0.0461	0.1718	0.1635	0.2725	0.2239
i12 Spirited Away	0.0907	0.1827	0.0629	0.1121	0.1827	0.1536
i10 Toy Story	0.0184	0.0344	0.1805	0.0777	0.1805	0.1065
i5 La La Land	0.0000	0.0000	0.0204	0.0068	0.0204	0.0093

ขั้นที่ 2 — ตีความ

แบบเฉลี่ยให้ลำดับเดียวกับแบบอิงโปรไฟล์ ซึ่งไม่น่าแปลกใจ เพราะเมื่อทุกเวกเตอร์ปรับ L2 แล้ว ค่าเฉลี่ยของโคไซน์ก็คือผลคูณจุดกับค่าเฉลี่ยของเวกเตอร์ ต่างกันเพียงตัวคูณคงที่ $1/\| c_u \|_2$ แต่แบบสูงสุดเปลี่ยนอันดับ: Mad Max ขึ้นมาเป็นที่สาม (0.2725) เพราะคล้าย The Matrix มาก (rebels, battle, armies ↔ fight, chase, survival) แม้ไม่คล้าย Inception เลย แบบสูงสุดจึงตอบคำถาม “คล้ายกับเรื่องใดเรื่องหนึ่งที่ชอบมากที่สุด” และให้คำอธิบายที่ผู้ใช้เข้าใจง่ายว่า “เพราะคุณชอบ The Matrix” ส่วนแบบเฉลี่ยให้รางวัลแก่เรื่องที่คล้ายกับ **ทุก** เรื่องที่ชอบพร้อมกัน

3.9.5 ฟังก์ชันจับคู่แบบอื่นที่ใช้โครงเดียวกัน

ตารางที่ 3.61 ฟังก์ชันจับคู่ระหว่างโปรไฟล์กับรายการที่พบในบทนี้

ฟังก์ชัน	สูตรหรือแนวคิด	ช่วงของค่า	เหมาะเมื่อ	หัวข้อ
โคไซน์ / ผลคูณจุด	$p_u \cdot v_i$ (หลังปรับ L2)	[0, 1] หรือ [−1, 1]	เวกเตอร์เบาบางหรือหนาแน่นที่ปรับความยาวแล้ว	3.9.1
Item-kNN	เฉลี่ยหรือค่าสูงสุดของโคไซน์กับแต่ละเรื่องที่ชอบ	[0, 1]	ต้องการอธิบายว่า “เพราะคุณชอบ X” หรือรสนิยมหลายกลุ่ม	3.9.4

ฟังก์ชัน	สูตรหรือแนวคิด	ช่วงของค่า	เหมาะเมื่อ	หัวข้อ
BM25 (โปรไฟล์เป็นข้อคำถาม)	คำเด่นของโปรไฟล์เป็น q แล้วให้คะแนนเอกสาร	$[0, \infty)$	มีดัชนีผกผันอยู่แล้ว ต้องการการอิ่มตัว	3.8.8
ความน่าจะเป็นจากตัวจำแนก	$P(\text{ชอบ} \mid v_i)$ จากนาอีฟเบส์หรือการถดถอยโลจิสติก	$[0, 1]$	มีผลป้อนกลับทั้งบวกและลบมากพอ	3.8.7
ตัวเข้ารหัสไขว้ / Two-Tower	โครงข่ายประสาทเรียนฟังก์ชันจับคู่เอง	ตามการฝึก	มีปฏิสัมพันธ์หลักแสนขึ้นไป	3.11, 3.12

ทุกฟังก์ชันรับโปรไฟล์กับรายการแล้วคืนตัวเลขหนึ่งตัวเหมือนกัน จึงเสียบเข้าแทนโคไซน์ในกระบวนงานจัดอันดับของหัวข้อถัดไปได้โดยไม่ต้องเปลี่ยนส่วนอื่น

3.10 การจัดอันดับและการสร้างรายการแนะนำ Top-K

คะแนนจับคู่ยังไม่ใช่คำแนะนำ ผู้ใช้ไม่ได้ต้องการเห็นตัวเลข 12 ตัว แต่ต้องการรายการสั้น ๆ ที่เรียงจากน่าสนใจที่สุด หัวข้อนี้แปลงคะแนนให้เป็นรายการแนะนำ Top-K ซึ่งฟังดูง่ายแต่มีการตัดสินใจซ่อนอยู่หลายจุด และปิดท้ายด้วยวิธีตรวจสอบระบบก่อนใช้งานจริง

3.10.1 ห้าขั้นจากคะแนนสู่รายการแนะนำ

ขั้นตอนวิธี การสร้างรายการแนะนำ Top-K จากโปรไฟล์

1. **รวบรวม CANDIDATE ITEM** เริ่มจากรายการทั้งหมดในคลัง (ระบบใหญ่เริ่มจาก CANDIDATE ITEM ที่ค้นคืนมาก่อนด้วยดัชนีในหัวข้อ 3.11)
2. **คำนวณคะแนนจับคู่** $s(u, i)$ ของทุก CANDIDATE ITEM ด้วยฟังก์ชันในหัวข้อ 3.9
3. **กรองรายการที่เคยเห็น** ตัดรายการที่ผู้ใช้เคยให้คะแนน ซื้อ หรือดูแล้วออก รวมถึงรายการที่กฎธุรกิจไม่ให้แสดง
4. **เรียงลำดับ** จากคะแนนมากไปน้อย โดยกำหนดกฎตัดสินเสมอไว้ล่วงหน้า
5. **เลือก K อันดับแรก** และตัดรายการที่คะแนนต่ำกว่าเกณฑ์ขั้นต่ำ θ ก่อนจัดอันดับซ้ำเพื่อความหลากหลาย (หัวข้อ 3.13)

ตัวอย่างการคำนวณ 3.25 รายการแนะนำ Top-3 ของ u1 จากโปรไฟล์ค่าเฉลี่ยศูนย์กลางในสามปริภูมิ

ขั้นที่ 1 — ข้อมูลเข้า: คะแนนจับคู่จากตารางที่ 3.56, 3.57 และ 3.58 หลังกรองห้าเรื่องที่ u1 เคยให้คะแนนออก เหลือ CANDIDATE ITEM 7 เรื่อง

ขั้นที่ 2 — เรียงลำดับและเลือกสามอันดับแรก ($K = 3$) กฎตัดสินเสมอตั้งต้นคือเรียงตามรหัสรายการ

ตารางที่ 3.62 รายการแนะนำ Top-3 ของ u1 ในสามปริภูมิ

อันดับ	ปริภูมิแนว (9 มิติ)	คะแนน	ปริภูมิ TF-IDF (203 มิติ)	คะแนน	ปริภูมิความหมาย (12 มิติ)	คะแนน
1	i8 Mad Max: Fury Road	0.8165	i12 Spirited Away	0.1620	i11 Coco	0.4032

อันดับ	ปริภูมิแนว (9 มิติ)	คะแนน	ปริภูมิ TF-IDF (203 มิติ)	คะแนน	ปริภูมิความหมาย (12 มิติ)	คะแนน
2	i7 John Wick	0.4082*	i5 La La Land	0.1268	i9 Parasite	0.2734
3	i9 Parasite	0.4082*	i11 Coco	0.0698	i7 John Wick	0.2343
ถัดไป	i5 0.0000 · i10 0.0000 · i11 0.0000 · i12 0.0000		i9 0.0654 · i10 0.0455 · i7 0.0183 · i8 0.0000		i8 0.2239 · i12 0.1536 · i10 0.1065 · i5 0.0093	

* John Wick กับ Parasite เสมอกันที่ 0.4082 เรียงตามรหัสรายการ

ขั้นที่ 3 — ตีความ

สามคอลัมน์ให้คำตอบคนละชุด คำแนะนำจากปริภูมิแนว (Mad Max, John Wick, Parasite) ตรงกับสิ่งที่มนุษย์คาดหวังจาก u1 มากที่สุด ปริภูมิคำถูกชักนำด้วยคำที่ซ้ำกันโดยบังเอิญ และปริภูมิความหมายจับแก่นเรื่อง “ความทรงจำ” จนแนะนำ Coco ก่อน Mad Max ไม่มีปริภูมิใดถูกทั้งหมด ซึ่งเป็นเหตุผลที่ต้องผสานคะแนนในหัวข้อ 3.12

3.10.2 การตัดสินเสมอ คะแนนศูนย์ และเกณฑ์คะแนนขั้นต่ำ

ปริภูมิเล็กอย่างแนวภาพยนตร์ทำให้คะแนนเสมอกันบ่อย การเรียงตามรหัสรายการทำซ้ำได้แต่ไม่มีเหตุผลเชิงเนื้อหา กฎที่ดีกว่าคือใช้ **ตัวตัดสินรอง (Secondary Key)** เช่น ความนิยม ความใหม่ หรือคะแนนจากปริภูมิที่สอง ซึ่งในตัวอย่างนี้ปริภูมิความหมายจะให้ Parasite (0.2734) นำ John Wick (0.2343)

คะแนนศูนย์ต้องแยกออกจากคะแนนต่ำ คะแนนศูนย์แปลว่าระบบ *ไม่มีหลักฐานเลย* ไม่ใช่มีหลักฐานว่าผู้ใช้จะไม่ชอบ Mad Max ในปริภูมิคำและ Spirited Away ในปริภูมิแนวคือกรณีเดียวกัน การแนะนำรายการคะแนนศูนย์เท่ากับสุ่ม ระบบจึงควรตั้ง **เกณฑ์คะแนนขั้นต่ำ** θ และเมื่อ CANDIDATE ITEM ที่ผ่านเกณฑ์มีน้อยกว่า K ให้เติมด้วยกลไกอื่น เช่น รายการยอดนิยม ค่า K เองเป็นการตัดสินใจเชิงผลิตภัณฑ์: หน้าจอโทรศัพท์แสดงได้ 3–5 รายการ หน้าเว็บ 10–20 รายการ และการประเมินในหัวข้อ 3.14 ต้องใช้ K เดียวกับที่ระบบแสดงจริง

3.10.3 ตรวจสอบก่อนใช้งานจริงด้วยการกันออกทีละรายการ

ก่อนเชื่อผลของระบบ ควรทดสอบด้วย **การกันออกทีละรายการ (Leave-One-Out: LOO)**: ซ่อนเรื่องที่ผู้ใช้ชอบไว้หนึ่งเรื่อง สร้างโปรไฟล์จากเรื่องที่เหลือ แล้วดูว่าระบบดึงเรื่องที่ซ่อนไว้กลับมาได้ที่อันดับใดในหมู่ CANDIDATE ITEM (รายการทั้งหมดยกเว้นที่ใช้สร้างโปรไฟล์)

ตัวอย่างการคำนวณ 3.26 การกันออกทีละรายการของ u1 ในสามปริภูมิ

ขั้นที่ 1 — ข้อมูลเข้า: สามรอบการทดลอง แต่ละรอบซ่อนหนึ่งเรื่อง โปรไฟล์สร้างจากอีกสองเรื่อง CANDIDATE ITEM 10 เรื่อง

ตารางที่ 3.63 อันดับที่เรื่องที่ซ่อนไว้กลับมา (CANDIDATE ITEM 10 เรื่อง) และสามอันดับแรกที่ระบบให้

เรื่องที่ซ่อน	โปรไฟล์จาก	ปริภูมิ: อันดับ (คะแนน)			Top-3 ปริภูมิความหมาย
		แนว	TF-IDF	ความหมาย	
i1 The Matrix	i2, i3	1 (0.5774)	3 (0.0859)	1 (0.4837)	i1, i11, i6
i2 Inception	i1, i3	2 (0.5774)	1 (0.1517)	1 (0.5364)	i2, i11, i6
i3 Interstellar	i1, i2	2 (0.5774)	5 (0.0585)	7 (0.1461)	i6, i11, i7

ขั้นที่ 2 — สรุปเป็นตัวชี้วัด: HR@3 (สัดส่วนรอบที่เรื่องที่ซ่อนอยู่ในสามอันดับแรก) และ MRR (ค่าเฉลี่ยของ 1/อันดับ)

$$\text{TF-IDF: } HR@3 = \frac{2}{3} = 0.6667,$$

$$MRR = \frac{1/3 + 1/1 + 1/5}{3} = 0.5111$$

$$\text{ความหมาย: } HR@3 = \frac{2}{3}, \quad MRR = \frac{1 + 1 + 1/7}{3} = 0.7143$$

ขั้นที่ 3 — ตีความ

ปริภูมิความหมายดึง The Matrix และ Inception กลับมาได้ที่อันดับหนึ่งทั้งคู่ แต่พลาด Interstellar อย่างสิ้นเชิง (อันดับ 7) เพราะโปรไฟล์จาก The Matrix + Inception เป็น tech + mind + crime ขณะที่ Interstellar เป็น space + family ปริภูมิแนวได้อันดับดีทุกรอบแต่มีการเสมอกันสามทาง (The Matrix, Mad Max, Parasite ได้ 0.5774 เท่ากันในรอบแรก) ซึ่งกฎตัดสินเสมอตามรหัสบังเอิญให้ผลดี การตรวจสอบเล็ก ๆ นี้ชี้ให้เห็นว่าจุดอ่อนอยู่ที่ **การแทนรายการ** ไม่ใช่ที่โปรไฟล์หรือการจัดอันดับ และหัวข้อ 3.14 จะขยายการทดลองนี้ไปทั้ง 8 ผู้ใช้

3.10.4 ระบบแนะนำอิงเนื้อหาอย่างง่ายฉบับสมบูรณ์

ถึงจุดนี้ทุกชิ้นส่วนครบแล้ว ตารางที่ 3.64 ร้อยหัวข้อ 3.3–3.10 เข้าเป็นระบบเดียว พร้อมระบุว่าแต่ละขั้นทำงานเมื่อใด ขั้นที่ทำ “ล่วงหน้า” ทำครั้งเดียวต่อคลังหรือต่อรายการใหม่ ส่วนขั้นที่ทำ “สด” ทำทุกครั้งที่ผู้ใช้เปิดหน้าจอ การแยกเช่นนี้คือเหตุผลที่ระบบอิงเนื้อหาขยายไปสู่รายการหลายล้านชิ้นได้

ตารางที่ 3.64 กระบวนงานทั้งหมดของระบบแนะนำอิงเนื้อหาจากข้อความ

ขั้น	ทำอะไร	ผลลัพธ์	ทำเมื่อใด	หัวข้อ
1 เตรียมข้อความ	ทำให้เป็นมาตรฐาน ตัดคำ ตัดคำหยุด	โทเคนของทุกเรื่อง	ล่วงหน้า (ต่อรายการ)	3.3

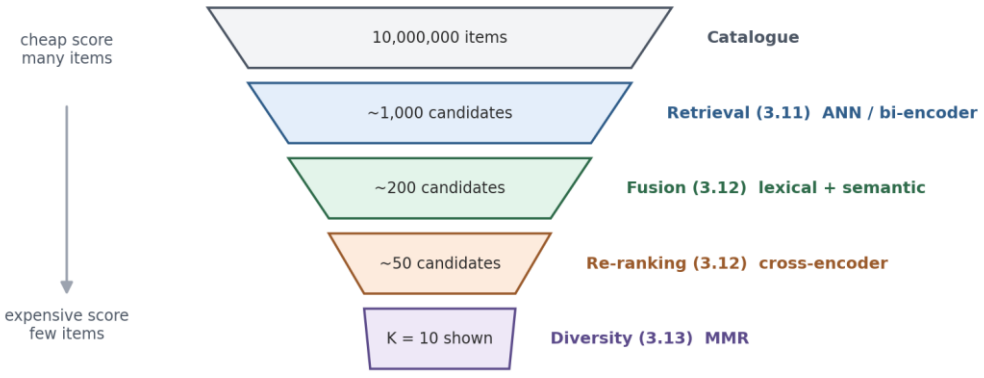
ขั้น	ทำอะไร	ผลลัพธ์	ทำเมื่อใด	หัวข้อ
2 สร้างเวกเตอร์รายการ	TF-IDF + L2 หรือเวกเตอร์ความหมาย (รวมค่า + L2)	เมทริกซ์ V ขนาด 12 × 203 หรือ 12 × 12	ล่วงหน้า (ต่อคลัง)	3.4–3.7
3 สร้างโปรไฟล์	เฉลี่ยเวกเตอร์ที่ชอบ (± ไม่ชอบ, เวลา) ปรับ L2	p_u	เมื่อมีผลป้อนกลับใหม่	3.8
4 ให้คะแนนจับคู่	$s(u,i) = p_u \cdot v_i$ ทุก CANDIDATE ITEM	คะแนน 12 ค่า	สด	3.9
5 จัดอันดับ	กรองที่เคยเห็น เรียง ตัดสินเสมอ เกณฑ์ θ เลือก K	Top-K	สด	3.10
6 อธิบายและตรวจสอบ	ส่วนร่วมรายมิติ · กันออกทีละรายการ	คำอธิบาย + HR/MRR	สดและออฟไลน์	3.9.3, 3.10.3

ประเด็นสำคัญ ระบบอย่างง่ายนี้คือโครงของทุกระบบในบท

ทุกหัวข้อหลังจากนี้เปลี่ยนเพียง **หนึ่งขั้น** ในตารางที่ 3.64 โดยคงขั้นอื่นไว้: ดัชนี ANN และ Two-Tower เปลี่ยนขั้นที่ 4 ให้ค้นในคลังหลายล้านได้ (3.11) การผสานคะแนนและตัวเข้ารหัสไขว้เปลี่ยนขั้นที่ 5 (3.12) MMR เพิ่มการจัดอันดับซ้ำหลังขั้นที่ 5 (3.13) และการประเมินขยายขั้นที่ 6 (3.14) ผู้อ่านที่เข้าใจตารางนี้ดีจึงมีแผนที่สำหรับอ่านครึ่งหลังของบทแล้ว

3.11 การค้นคืน CANDIDATE ITEM จากคลังขนาดใหญ่

ทุกอย่างในหัวข้อ 3.9 และ 3.10 ทำงานได้เพราะคลังมีเพียง 12 รายการ ระบบจริงมีรายการหลักล้านถึงหลักร้อยล้าน และต้องตอบกลับภายในเสี้ยววินาที หัวข้อนี้เปลี่ยน **เพียงขั้นที่ 4** ของตารางที่ 3.64 คือขั้นให้คะแนน จากการไล่คำนวณครบทุกรายการ ไปเป็นการ **ค้นคืน CANDIDATE ITEM (Candidate Retrieval)** ด้วยดัชนีเวกเตอร์ ส่วนขั้นอื่นทั้งหมดยังเหมือนเดิมทุกประการ



ภาพที่ 3.6 กรวยของระบบแนะนำขนาดใหญ่: ยิ่งลงล่างจำนวน CANDIDATE ITEM ยิ่งน้อย ฟังก์ชันให้คะแนนยิ่งแพงและแม่นขึ้น หัวข้อ 3.11–3.13 คือสามชั้นล่างของกรวยนี้

3.11.1 เหตุใดการให้คะแนนครบทุกรายการจึงไม่ไหว

การให้คะแนนแบบครบถ้วน (Exhaustive Scan หรือ Flat Search) ต้องคำนวณผลคูณจุดระหว่างโปรไฟล์กับเวกเตอร์ของรายการ **ทุกรายการ** ต้นทุนจึงเป็น $O(Nd)$ เมื่อ N คือจำนวนรายการและ d คือจำนวนมิติ ค่านี้เพิ่มเป็นเส้นตรงตามขนาดคลัง ขณะที่งบเวลาของการตอบกลับหนึ่งครั้งคงที่อยู่ราว 50 มิลลิวินาที

ตัวอย่างการคำนวณ 3.27 ต้นทุนของการให้คะแนนครบถ้วนเมื่อคลังใหญ่ขึ้น

ขั้นที่ 1 — ข้อมูลเข้า: เวกเตอร์ความหมาย $d = 768$ มิติ เก็บเป็น float32 (4 ไบต์ต่อค่า) เครื่องคำนวณผลคูณจุดหนึ่งคู่ได้ในเวลาประมาณ 0.4 ไมโครวินาที

ขั้นที่ 2 — แทนค่าในสองสูตร

$$T(N) = N \times 0.4\ \mu s, \qquad M(N) = N \times d \times 4 \text{ ไบต์} = N \times 768 \times 4 = N \times 3{,}072 \text{ ไบต์} \tag{3.19}$$

ตารางที่ 3.65 เวลาและหน่วยความจำของการให้คะแนนครบถ้วน เทียบกับงบเวลา 50 มิลลิวินาทีต่อคำขอ

ขนาดคลัง N	เวลาต่อคำขอ $T(N)$	หน่วยความจำเวกเตอร์ $M(N)$	อยู่ในงบ 50 ms หรือไม่
1,000 (ร้านเล็ก)	0.0004 วินาที (0.4 ms)	3.07 MB	ได้สบาย
100,000 (ร้านหนังสือ)	0.04 วินาที (40 ms)	307 MB	เฉียดงบ
1,000,000 (เพลงสตรีมมิง)	0.4 วินาที	2.86 GiB	เกินงบ 8 เท่า
10,000,000 (วิดีโอสั้น)	4 วินาที	28.61 GiB	เกินงบ 80 เท่า

ขั้นที่ 3 — ตีความ

ปัญหามีสองด้านพร้อมกัน ด้าน เวลา ทำให้ผู้ใช้รอ และด้าน หน่วยความจำ ทำให้เวกเตอร์ทั้งหมดใส่ในแรมของเครื่องเดียวไม่ได้ ทางออกคือยอมแลกความถูกต้องเล็กน้อยเพื่อความเร็วมหาศาล ด้วย**การค้นแบบ เพื่อนบ้านใกล้สุดโดยประมาณ (Approximate Nearest Neighbour: ANN)** ซึ่งไม่รับประกันว่าจะได้เพื่อนบ้านที่ใกล้ที่สุดจริง ๆ ครบทุกตัว แต่มักได้ 95–99% ของคำตอบที่ถูกต้องในเวลาที่เร็วกว่าหลักร้อยเท่า

นิยาม 3.17 อัตราการเรียกคืนของดัชนี (Index Recall@k)

$$\text{Recall@}k = \frac{|\text{ผลจากดัชนี ANN } k \text{ อันดับแรก} \cap \text{ผลจากการค้นครบถ้วน } k \text{ อันดับแรก}|}{k} \tag{3.20}$$

ค่านี้วัด **ดัชนี** ไม่ใช่วัดคุณภาพคำแนะนำ ถ้า Recall@100 = 0.98 แปลว่าใน CANDIDATE ITEM 100 อันดับแรกที่ควรได้ ดัชนีดึงกลับมาได้ 98 รายการ ส่วนอีก 2 รายการหลุดไป ระบบจริงตั้งเป้าที่ 0.95–0.99 แล้วเร่งความเร็วให้มากที่สุดภายใต้เป้านั้น

3.11.2 HNSW: ดัชนีกราฟหลายชั้น

HNSW (Hierarchical Navigable Small World) คือดัชนีเวกเตอร์ที่พบบ่อยที่สุดในระบบผู้แนะนำและระบบค้นคืนเชิงความหมาย แนวคิดเลียนแบบการเดินทาง:

ถ้าจะไปหมู่บ้านเล็กแห่งหนึ่ง เราขึ้นเครื่องบินไปเมืองใหญ่ก่อน แล้วต่อรถไฟ แล้วจึงต่อรถสองแถว ไม่ใช่เดินเท้าจากบ้านทีละก้าว

นิยาม 3.18 โครงสร้างของ HNSW

- **ชั้นล่างสุด (Layer 0)** มีรายการครบทุกตัว แต่ละตัวเชื่อมกับเพื่อนบ้านที่ใกล้ที่สุดประมาณ M ตัว (ค่าที่ใช้บ่อยคือ $M = 16$ ถึง 32)
- **ชั้นบน** สุ่มเลือกรายการขึ้นไปด้วยความน่าจะเป็น $1/M$ ต่อชั้น จำนวนรายการจึงลดลงแบบเรขาคณิต และเส้นเชื่อมในชั้นบนกลายเป็น “เส้นทางไกล”
- **การค้นหา** เริ่มที่จุดเข้าในชั้นบนสุด เดินแบบละโมบ (Greedy) ไปยังเพื่อนบ้านที่ใกล้ข้อคำถามกว่าเดิม เมื่อไปต่อไม่ได้แล้วจึงลงชั้นถัดไป ทำซ้ำจนถึงชั้น 0

$$\text{จำนวนรายการในชั้น } l \;\approx\; N \times M^{-l} \qquad \text{เวลาค้นหา} \;\approx\; O(\log N) \tag{3.21}$$

ตัวอย่างการคำนวณ 3.28 ขนาดของแต่ละชั้นเมื่อคลังมี 10 ล้านรายการและ $M = 32$

ขั้นที่ 1 — ข้อมูลเข้า: $N = 10{,}000{,}000$ และความน่าจะเป็นที่รายการหนึ่งจะขึ้นไปอยู่ชั้น l เท่ากับ 32^{-l}

ขั้นที่ 2 — แทนค่าทีละชั้น

ตารางที่ 3.66 จำนวนรายการในแต่ละชั้นของ HNSW และบทบาทของชั้นนั้นในการค้นหา

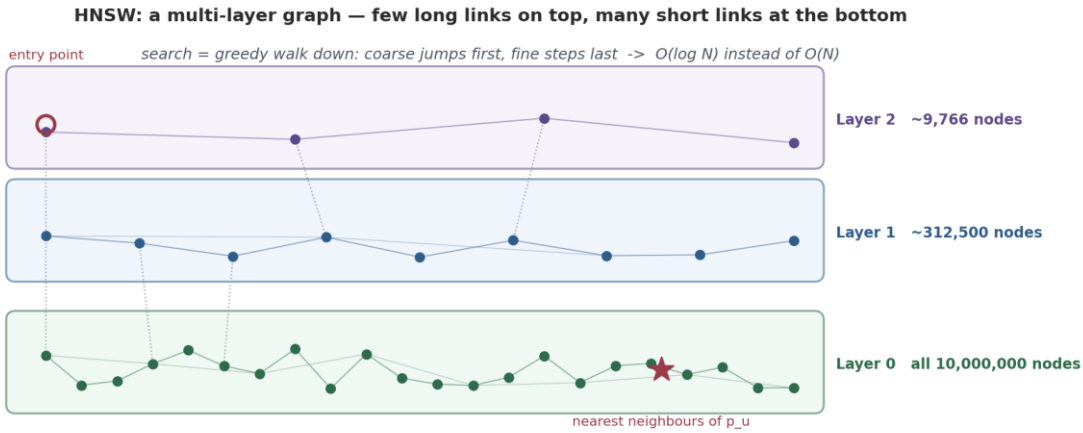
ชั้น l	ความน่าจะเป็น 32^{-l}	จำนวนรายการ $N \times 32^{-l}$	บทบาทในการค้นหา
3	3.05176e-05	305	ชั้นบนสุด: จุดเข้าของการค้นหา
2	0.000976562	9,766	ชั้นหยาบ: กระโดดข้ามคลังเป็นช่วงใหญ่
1	0.03125	312,500	ชั้นกลาง: ย่นระยะจากหลักแสนเหลือหลักพัน
0	1	10,000,000	ชั้นละเอียด: เดินทีละก้าวสั้นจนพบเพื่อนบ้านจริง

ขั้นที่ 3 — เปรียบเทียบจำนวนการคำนวณ

$$\text{ค้นครบถ้วน} = 10{,}000{,}000 \text{ ครั้ง} \qquad \text{HNSW} \approx 305 + 9{,}766^{\text{บางส่วน}} + \cdots \approx 2{,}000\text{–}5{,}000 \text{ ครั้ง}$$

ขั้นที่ 4 — ตีความ

การเดินลงชั้นทำให้จำนวนการคำนวณลดจากสิบล้านเหลือหลักพัน คือเร็วขึ้นราวสองพันเท่า โดยยังได้ Recall@100 ประมาณ 0.95–0.99 ราคาที่ต้องจ่ายคือหน่วยความจำสำหรับเก็บเส้นเชื่อม (ประมาณ $M \times 8$ ไบต์ต่อรายการ) และเวลาสร้างดัชนี ค่าปรับแต่งสำคัญมีสองตัวคือ M (จำนวนเพื่อนบ้าน ยิ่งมากยิ่งแม่นและยิ่งกินหน่วยความจำ) และ `efSearch` (จำนวน CANDIDATE ITEM ที่เก็บไว้ระหว่างเดิน ยิ่งมากยิ่งแม่นและยิ่งช้า) ทั้งคู่คือปุ่มปรับความสมดุลระหว่าง Recall กับเวลา



ภาพที่ 3.7 โครงสร้างสามชั้นของ HNSW สำหรับคลัง 10 ล้านรายการ การค้นหาเริ่มจากจุดเข้าในชั้นบน เดินแบบละโมบแล้วลงชั้นทีละชั้นจนถึงชั้น 0

3.11.3 IVF-PQ: เมื่อเวกเตอร์ทั้งหมดใหญ่เกินหน่วยความจำ

HNSW แก้ปัญหาด้านเวลา แต่ยังเก็บเวกเตอร์เต็มไว้ทุกตัว เมื่อคลังใหญ่จนหน่วยความจำไม่พอ (ตารางที่ 3.65 แถวสุดท้าย) ต้องใช้ดัชนีอีกตระกูลหนึ่งคือ IVF-PQ ซึ่งรวมสองความคิดง่าย ๆ เข้าด้วยกัน

แบ่งคลังเป็นกลุ่ม → ค้นเฉพาะกลุ่มที่ใกล้ข้อคำถาม → ใช้รหัสบีบอัดแทนเวกเตอร์เต็ม (3.22)

- IVF (Inverted File Index) จับกลุ่มเวกเตอร์ทั้งคลังด้วย k-means ให้ได้ n_{list} กลุ่ม (นิยมตั้ง $n_{\text{list}} \approx \sqrt{N}$) เวลาค้น ระบบเทียบข้อคำถามกับ **ศูนย์กลางกลุ่ม** ก่อน แล้วเปิดดูเฉพาะ n_{probe} กลุ่มที่ใกล้ที่สุด
- PQ (Product Quantization) ตัดเวกเตอร์ 768 มิติเป็นท่อนย่อย เช่น 64 ท่อน ท่อนละ 12 มิติ แล้วแทนแต่ละท่อนด้วย **รหัส 1 ไบต์** ที่ชี้ไปยังตัวแทน 256 ตัวในสมุดรหัส (Codebook) ของท่อนนั้น เวกเตอร์หนึ่งตัวจึงเหลือเพียง 64 ไบต์

ตัวอย่างการคำนวณ 3.29 IVF-PQ กับคลัง 10 ล้านรายการ: ลดหน่วยความจำและเวลาพร้อมกัน

ขั้นที่ 1 — ข้อมูลเข้า: $N = 10{,}000{,}000$, $d = 768$, PQ ใช้ 64 ท่อน ท่อนละ 1 ไบต์ (512 บิตต่อเวกเตอร์) และเก็บค่าใช้จ่ายส่วนหัวอีก 8 ไบต์ต่อรายการ

ขั้นที่ 2 — หน่วยความจำ

$$M_{\text{เต็ม}} = \frac{10^7 \times 768 \times 4}{2^{30}} = 28.61 \text{ GiB} \qquad M_{\text{PQ}} = \frac{10^7 \times (64+8)}{2^{30}} = 0.6706 \text{ GiB}$$

ขั้นที่ 3 — เวลา: ตั้ง $n_{\text{list}} = \sqrt{10^7} \approx 3{,}163$ กลุ่ม และเปิดดู $n_{\text{probe}} = 16$ กลุ่ม

$$\text{สัดส่วนที่ต้องคำนวณ} = \frac{n_{\text{probe}}}{n_{\text{list}}} = \frac{16}{3{,}163} = 0.0051 \Rightarrow 10^7 \times 0.0051 \approx 51{,}000 \text{ รายการ}$$

ตารางที่ 3.67 เปรียบเทียบสามวิธีค้นคืนบนคลังเดียวกัน (ตัวเลขเป็นค่าโดยประมาณเพื่อการเปรียบเทียบ)

วิธี	หน่วยความจำ	จำนวนการคำนวณต่อคำขอ	Recall@100 โดยทั่วไป	เหมาะเมื่อ
ค้นครบถ้วน (Flat)	28.61 GiB	10,000,000	1.00 (นิยาม)	คลังเล็ก หรือใช้เป็นค่าอ้างอิงตอนประเมิน

วิธี	หน่วยความจำ	จำนวนการคำนวณต่อคำขอ	Recall@100 โดยทั่วไป	เหมาะเมื่อ
HNSW	28.61 GiB + เส้นเชื่อม	ประมาณ 2,000–5,000	0.95–0.99	หน่วยความจำพอ ต้องการเร็วและแม่น
IVF-PQ	0.6706 GiB	ประมาณ 51,000 (บนรหัสบีบอัด)	0.80–0.95	คลังใหญ่มากจนหน่วยความจำไม่พอ

ขั้นที่ 4 — ตีความ

หน่วยความจำลดจาก 28.61 GiB เหลือ 0.6706 GiB คือเล็กลงประมาณ 43 เท่า จนใส่เครื่องเดียวได้สบาย แลกกับความคลาดเคลื่อนของระยะทางที่ประมาณจากรหัส ระบบจริงจึงนิยมใช้ IVF-PQ **ค้นหยาบ** เพื่อดึง CANDIDATE ITEM หลักพัน แล้วให้คะแนนใหม่ด้วยเวกเตอร์เต็มเฉพาะ CANDIDATE ITEM เหล่านั้น (เรียกว่า Re-scoring) ซึ่งเป็นแนวคิดเดียวกับการจัดอันดับซ้ำในหัวข้อ 3.12 รายละเอียดการสร้างสมุดรหัสและการประมาณระยะทางแบบไม่สมมาตรเป็นเรื่องของการนำไปพัฒนา ไม่จำเป็นต่อการเข้าใจระบบแนะนำในบทนี้

3.11.4 Two-Tower: ตัวเข้ารหัสคู่ในบทบาทของระบบผู้แนะนำ

ดัชนีเวกเตอร์จะใช้ได้ก็ต่อเมื่อ **โปรไฟล์กับรายการอยู่ในปริภูมิเดียวกัน** ซึ่งเป็นกฎข้อเดียวของหัวข้อ 3.8.1 อีกครั้ง โครงสร้างที่ทำให้เงื่อนไขนี้เป็นจริงโดยการเรียนรู้เรียกว่า **Two-Tower** และมันคือ **ตัวเข้ารหัสคู่ (Bi-Encoder)** ในหัวข้อ 3.6.3 ที่เปลี่ยนชื่อฝั่งซ้ายจาก “ข้อคำถาม” เป็น “ผู้ใช้” เท่านั้น

ตารางที่ 3.68 ตัวเข้ารหัสคู่ในงานค้นคืนกับ Two-Tower ในระบบผู้แนะนำคือแนวคิดเดียวกัน

ส่วนประกอบ	ตัวเข้ารหัสคู่ (งานค้นคืน)	Two-Tower (ระบบผู้แนะนำ)	ในบทนี้
หอซ้าย	เข้ารหัสข้อคำถาม q	เข้ารหัสผู้ใช้: ประวัติ + บริบท + ข้อมูลประชากร	โปรไฟล์ p_u (3.8)
หอขวา	เข้ารหัสเอกสาร d	เข้ารหัสรายการ: ข้อความ + แนว + คุณลักษณะ	เวกเตอร์รายการ v_i (3.4–3.7)
การจับคู่	$\cos(q, d)$	$\cos(p_u, v_i)$ หรือผลคูณจุด	คะแนนจับคู่ (3.9)
คำนวณล่วงหน้าได้	เวกเตอร์เอกสารทั้งคลัง	เวกเตอร์รายการทั้งคลัง	ทำให้สร้างดัชนีได้ (3.11)
การฝึก	InfoNCE บนคู่ (ข้อคำถาม, เอกสารที่ตรง)	InfoNCE บนคู่ (ผู้ใช้, รายการที่คลิกหรือซื้อ)	หัวข้อ 3.6.4

เพราะเป็นโครงเดียวกัน วิธีฝึกจึงเหมือนกันด้วย: ใช้ **คู่บวก** คือรายการที่ผู้ใช้คลิกหรือให้คะแนนสูง และ **คู่ลบ** คือรายการที่ไม่ได้ถูกเลือก แล้วปรับพารามิเตอร์ด้วยฟังก์ชันสูญเสีย InfoNCE ตามสมการ (3.10) ในหัวข้อ 3.6.4 คู่ลบในระบบผู้แนะนำนิยมสุ่มจากในกลุ่มการฝึกเดียวกัน (In-batch Negatives) เพราะได้คู่ลบจำนวนมากโดยแทบไม่มีต้นทุนเพิ่ม

ข้อควรระวัง กับดักสี่ข้อที่ทำให้ Two-Tower ได้ผลดีตอนทดสอบแต่ล้มเหลวตอนใช้งานจริง

1. **ข้อมูลรั่ว (Data Leakage)** ใส่คุณลักษณะที่ในเวลาจริงยังไม่มี เช่น จำนวนคลิกของรายการ *หลัง* ช่วงเวลาที่ทำนาย ผลคือคะแนนออฟไลน์สูงเกินจริง
2. **คู่ลบง่ายเกินไป** ถ้าคู่ลบสุ่มมาจากทั้งคลัง แบบจำลองจะแยกเพียงว่า “ภาพยนตร์ต่างจากเครื่องซักผ้า” แต่แยก “Sci-Fi กับ Sci-Fi อีกเรื่อง” ไม่ได้ ต้องเติม **คู่ลบยาก** ตามหัวข้อ 3.6.4
3. **คู่ลบเทียม (False Negative)** รายการที่ผู้ใช้ยังไม่เห็นไม่ได้แปลว่าไม่ชอบ การลงโทษรายการเหล่านี้แรงเกินไปทำให้แบบจำลองเรียนผิด
4. **แบ่งข้อมูลแบบสุ่มแทนการแบ่งตามเวลา** การสุ่มทำให้แบบจำลองเห็นอนาคตของผู้ใช้คนเดียวกัน ค่าที่วัดได้จึงดีเกินจริง ต้องแบ่งตามเวลาเสมอ (หัวข้อ 3.14.1)

3.11.5 ทางเลือกของการค้นคืน: เบาบาง หนาแน่น และระดับโทเคน

ตัวเข้ารหัสคู่เก็บเวกเตอร์หนาแน่นหนึ่งตัวต่อข้อความ ซึ่งเร็วที่สุดแต่บีบข้อความทั้งชิ้นเหลือจุดเดียว จึงมีแนวทางอื่นที่พยายามรักษาความละเอียดของคำไว้ โดยยังค้นคืนได้เร็ว ตารางที่ 3.69 สรุปสี่ทางเลือกหลัก ผู้เรียนควรรู้ว่ามีทางเลือกเหล่านี้และรู้ว่าแต่ละแบบแลกอะไรกับอะไร ส่วนรายละเอียดของฟังก์ชันจุดประสงค์และโครงสร้างดัชนีเฉพาะทางเป็นเรื่องของวิชาการค้นคืนสารสนเทศโดยตรง

ตารางที่ 3.69 สี่แนวทางการให้คะแนนและค้นคืนข้อความ

แนวทาง	สิ่งที่เก็บต่อหนึ่งข้อความ	วิธีให้คะแนน	จุดเด่น	จุดอ่อน
TF-IDF / BM25 (3.4)	เวกเตอร์เบาบางจากคำที่ปรากฏจริง	ผลรวมน้ำหนักของคำร่วม	ตรงตามคำ อธิบายง่าย ไม่ต้องฝึก	ไม่รู้จักคำพ้องความหมาย
SPLADE	เวกเตอร์เบาบางที่ **แบบจำลองเรียนรู้** น้ำหนักและขยายคำเอง	เหมือน BM25 แต่บนคำที่ขยายแล้ว	ใช้ดัชนีผกผันเดิมได้ แก้ปัญหาคำพ้อง	ต้องฝึก และเวกเตอร์เบาบางน้อยลงจึงช้ากว่า BM25
Bi-Encoder (3.6.3)	เวกเตอร์หนาแน่นหนึ่งตัว (เช่น 768 มิติ)	โคไซน์กับเวกเตอร์ข้อคำถาม	เร็วที่สุด สร้างดัชนี ANN ได้	บีบข้อความยาวเหลือจุดเดียว รายละเอียดหาย
ColBERT	เวกเตอร์ **ระดับโทเคน** หลายตัวต่อหนึ่งข้อความ	MaxSim: จับคู่โทเคนที่คล้ายที่สุดแล้วรวม	ละเอียดกว่าเวกเตอร์เดียว ยังค้นคืนได้	หน่วยความจำมากกว่าหลายสิบเท่า
Cross-Encoder (3.6.3)	ไม่เก็บ ต้องคำนวณใหม่ทุกคู่	อ่านข้อคำถามกับรายการพร้อมกัน	แม่นที่สุด	ใช้ค้นคืนไม่ได้ ใช้ได้เฉพาะจัดอันดับซ้ำ

$$\text{MaxSim (ColBERT):}\quad s(q,d) = \sum_{t \in q} \max_{t' \in d} \cos(e_t, e_{t'}) \tag{3.23}$$

ประเด็นสำคัญ วิธีเลือกในทางปฏิบัติ

ระบบจำนวนมากไม่ได้เลือกอย่างใดอย่างหนึ่ง แต่ใช้ **สองทางพร้อมกัน**: ค้นคืนด้วย BM25 และด้วยตัวเข้ารหัสคู่ แล้วผสานผลลัพธ์เข้าด้วยกัน เพราะสองแบบนี้ผิดพลาดคนละแบบ BM25 พลาดเมื่อผู้ใช้ใช้คำต่างจากในเอกสาร ส่วนเวกเตอร์หนาแน่นพลาดเมื่อคำเฉพาะสำคัญ เช่น ชื่อเฉพาะหรือรหัสรุ่นสินค้า วิธีผสานคือเนื้อหาของหัวข้อถัดไป

3.12 การผสานคะแนนและการจัดอันดับซ้ำ

หัวข้อ 3.9 แสดงให้เห็นแล้วว่าปริภูมิต่างกันให้คำแนะนำต่างกัน และหัวข้อ 3.11 อธิบายว่าระบบจริงมักค้นคืนจากหลายแหล่งพร้อมกัน คำถามจึงเปลี่ยนจาก “ปริภูมิใดดีที่สุด” เป็น “จะรวมคะแนนจากหลายแหล่งให้เป็นอันดับเดียวได้อย่างไร” หัวข้อนี้ตอบคำถามนั้นด้วยสี่เครื่องมือ คือ การปรับมาตรา การผสานคะแนน การจัดอันดับซ้ำด้วยตัวเข้ารหัสไขว้ และการเรียนรู้เพื่อจัดอันดับ

3.12.1 ปัญหา: สองแหล่งให้คำตอบคนละชุด

ตารางที่ 3.70 วางอันดับ CANDIDATE ITEM ทั้งเจ็ดของ u1 จากสองแหล่งไว้ข้างกัน แหล่งแรกคือคะแนน TF-IDF (เชิงคำ) และแหล่งที่สองคือคะแนนความหมาย ทั้งคู่มาจากตารางที่ 3.57 และ 3.58

ตารางที่ 3.70 อันดับ CANDIDATE ITEM ของ u1 จากสองแหล่ง: คะแนนดิบและอันดับ

รายการ	คะแนนเชิงคำ	อันดับเชิงคำ	คะแนนความหมาย	อันดับความหมาย	ผลต่างของอันดับ
i12 Spirited Away	0.1620	1	0.1536	5	4
i5 La La Land	0.1268	2	0.0093	7	5
i11 Coco	0.0698	3	0.4032	1	2
i9 Parasite	0.0654	4	0.2734	2	2
i10 Toy Story	0.0455	5	0.1065	6	1
i7 John Wick	0.0183	6	0.2343	3	3
i8 Mad Max: Fury Road	0.0000	7	0.2239	4	3

สองแหล่งขัดกันอย่างรุนแรง La La Land อยู่อันดับ 2 ในเชิงคำแต่อันดับ 7 ในเชิงความหมาย ส่วน Mad Max อยู่อันดับ 7 ในเชิงคำแต่อันดับ 4 ในเชิงความหมาย การจะรวมคะแนนได้ต้องแก้ปัญหาข้อแรกก่อน นั่นคือคะแนนสองชุดนี้ **อยู่คนละมาตรา**: คะแนนเชิงคำสูงสุดคือ 0.1620 ส่วนคะแนนความหมายสูงสุดคือ 0.4032 ถ้านำมาบวกกันตรง ๆ คะแนนความหมายจะครอบงำผลลัพธ์เพียงเพราะตัวเลขใหญ่กว่า

3.12.2 ขั้นที่หนึ่ง: ปรับมาตราให้เทียบกันได้

นิยาม 3.19 การปรับมาตราแบบต่ำสุด–สูงสุด (Min-Max Normalization)

$$\tilde{s}_i = \frac{s_i - \min_j s_j}{\max_j s_j - \min_j s_j} \in [0,1] \tag{3.24}$$

คำนวณจาก **ชุด CANDIDATE ITEM ของคำขอนั้น** เท่านั้น ผลคือรายการที่ดีที่สุดของแต่ละแหล่งได้ 1 และแย่ที่สุดได้ 0 เสมอ ข้อดีคือง่ายและตีความได้ ข้อเสียคือค่าขึ้นกับชุด CANDIDATE ITEM และถ้ามีคะแนนโดดเดี่ยวหนึ่งค่า (Outlier) มาตราจะถูกบีบ ทางเลือกอื่นคือการปรับแบบ z-score $(s_i - \mu)/\sigma$ หรือการใช้อันดับแทนคะแนนตามหัวข้อ 3.12.4

ตัวอย่างการคำนวณ 3.30 ปรับมาตราคะแนนทั้งสองแหล่งของ u1

ขั้นที่ 1 — ข้อมูลเข้า: ค่าต่ำสุดและสูงสุดของแต่ละแหล่งจากตารางที่ 3.70

เชิงคำ: $\min = 0.0000,\ \max = 0.1620$ ความหมาย: $\min = 0.0093,\ \max = 0.4032$

ขั้นที่ 2 — แทนค่า เช่น Coco (i11) ในทั้งสองแหล่ง

$$\tilde{s}_{i11}^{\text{คำ}} = \frac{0.0698-0.0000}{0.1620-0.0000} = 0.4309 \qquad \tilde{s}_{i11}^{\text{ความหมาย}} = \frac{0.4032-0.0093}{0.4032-0.0093} = 1.0000$$

ตารางที่ 3.71 คะแนนดิบและคะแนนที่ปรับมาตราแล้วของ CANDIDATE ITEM ทั้งเจ็ด

รายการ	คะแนนเชิงคำ $s^{\text{คำ}}$	ปรับมาตรา $\tilde{s}^{\text{คำ}}$	คะแนนความหมาย $s^{\text{คม}}$	ปรับมาตรา $\tilde{s}^{\text{คม}}$
i11 Coco	0.0698	0.4309	0.4032	1.0000
i9 Parasite	0.0654	0.4037	0.2734	0.6705
i7 John Wick	0.0183	0.1130	0.2343	0.5712
i8 Mad Max: Fury Road	0.0000	0.0000	0.2239	0.5448
i12 Spirited Away	0.1620	1.0000	0.1536	0.3663
i10 Toy Story	0.0455	0.2809	0.1065	0.2468
i5 La La Land	0.1268	0.7827	0.0093	0.0000

ขั้นที่ 3 — ตีความ

การปรับมาตราไม่เปลี่ยนอันดับภายในแหล่งเดียวกันแม้แต่น้อย เพราะเป็นการแปลงเชิงเส้นที่เพิ่มขึ้นตามค่าเดิม สิ่งที่เปลี่ยนคือ **ทั้งสองแหล่งพูดภาษาเดียวกันแล้ว** ข้อควรระวังคือ Mad Max (i8) ได้ 0.0000 ในคอลัมน์เชิงคำ ไม่ใช่เพราะแย่ที่สุดจริง ๆ แต่เพราะมันเป็นค่าต่ำสุดของชุดนี้พอดี การปรับมาตราแบบนี้จึงเป็นการเทียบ **ภายในชุด** CANDIDATE ITEM เสมอ

3.12.3 ขั้นที่สอง: ผสานคะแนนแบบถ่วงน้ำหนัก

นิยาม 3.20 การผสานแบบนูน (Convex Combination)

$$s_{\text{ผสาน}}(u,i) = w\tilde{s}^{\text{คำ}}(u,i) + (1-w)\tilde{s}^{\text{ความหมาย}}(u,i), \qquad w \in [0,1] \tag{3.25}$$

$w = 1$ คือใช้เฉพาะคะแนนเชิงคำ $w = 0$ คือใช้เฉพาะคะแนนความหมาย และค่าระหว่างกลางคือการประนีประนอม ค่า w ไม่ใช่สิ่งที่เดาเอง แต่เลือกจากผลการประเมินในหัวข้อ 3.14 ขยายเป็นหลายแหล่งได้โดยใช้ $\sum_m w_m \tilde{s}_m$ โดยที่ $\sum_m w_m = 1$

ตัวอย่างการคำนวณ 3.31 ผสานคะแนนของ u1 ที่ค่า w ห้าระดับ

ขั้นที่ 1 — ข้อมูลเข้า: คะแนนที่ปรับมาตราแล้วทั้งสองคอลัมน์จากตารางที่ 3.71

ขั้นที่ 2 — แทนค่าที่ $w = 0.5$ ครบทั้งเจ็ดรายการ

$$
\begin{aligned}
s_{i5} &= 0.5 \times 0.7827 + 0.5 \times 0.0000 = 0.3913 \\
s_{i7} &= 0.5 \times 0.1130 + 0.5 \times 0.5712 = 0.3421 \\
s_{i8} &= 0.5 \times 0.0000 + 0.5 \times 0.5448 = 0.2724 \\
s_{i9} &= 0.5 \times 0.4037 + 0.5 \times 0.6705 = 0.5371 \\
s_{i10} &= 0.5 \times 0.2809 + 0.5 \times 0.2468 = 0.2638 \\
s_{i11} &= 0.5 \times 0.4309 + 0.5 \times 1.0000 = 0.7155 \\
s_{i12} &= 0.5 \times 1.0000 + 0.5 \times 0.3663 = 0.6832
\end{aligned}
$$

ตารางที่ 3.72 คะแนนผสานและอันดับเมื่อเปลี่ยนน้ำหนัก w ของคะแนนเชิงคำ

รายการ	$w = 0$ (ความหมายล้วน)	$w = 0.3$	$w = 0.5$	$w = 0.7$	$w = 1$ (คำล้วน)
i11 Coco	1.0000 (อันดับ 1)	0.8293 (อันดับ 1)	0.7155 (อันดับ 1)	0.6016 (อันดับ 2)	0.4309 (อันดับ 3)
i12 Spirited Away	0.3663 (อันดับ 5)	0.5564 (อันดับ 3)	0.6832 (อันดับ 2)	0.8099 (อันดับ 1)	1.0000 (อันดับ 1)
i9 Parasite	0.6705 (อันดับ 2)	0.5905 (อันดับ 2)	0.5371 (อันดับ 3)	0.4837 (อันดับ 4)	0.4037 (อันดับ 4)
i5 La La Land	0.0000 (อันดับ 7)	0.2348 (อันดับ 7)	0.3913 (อันดับ 4)	0.5479 (อันดับ 3)	0.7827 (อันดับ 2)
i7 John Wick	0.5712 (อันดับ 3)	0.4337 (อันดับ 4)	0.3421 (อันดับ 5)	0.2505 (อันดับ 6)	0.1130 (อันดับ 6)
i8 Mad Max: Fury Road	0.5448 (อันดับ 4)	0.3814 (อันดับ 5)	0.2724 (อันดับ 6)	0.1634 (อันดับ 7)	0.0000 (อันดับ 7)
i10 Toy Story	0.2468 (อันดับ 6)	0.2570 (อันดับ 6)	0.2638 (อันดับ 7)	0.2707 (อันดับ 5)	0.2809 (อันดับ 5)

ขั้นที่ 3 — ตีความ

Coco (i11) เป็นอันดับหนึ่งที่ $w \leq 0.5$ แต่ Spirited Away (i12) แซงขึ้นมาเมื่อ $w = 0.7$ เพราะคะแนนเชิงคำของมันสูงสุดพอดี CANDIDATE ITEM ที่ **ดีปานกลางทั้งสองแหล่ง** อย่าง Parasite (i9) ทรงตัวอยู่อันดับ 2–3 แทบทุกค่า w ซึ่งเป็นพฤติกรรมที่พึงประสงค์ของการผสาน: มันให้รางวัลแก่รายการที่ **ทั้งสองแหล่งเห็นพ้อง** และลงโทษรายการที่ดีเพียงแหล่งเดียวอย่าง La La Land (i5) ซึ่งร่วงจากอันดับ 2 เมื่อใช้คำล้วน มาอยู่อันดับ 4 ที่ w=0.5 นี่คือวิธีที่การผสานกำจัดข้อผิดพลาดจากคำพ้องรูปได้โดยไม่ต้องเขียนกฎเฉพาะกิจ

3.12.4 ผสานด้วยอันดับแทนคะแนน: RRF

เมื่อสองแหล่งมีมาตราต่างกันมากจนปรับให้เทียบกันได้ยาก หรือแหล่งหนึ่งไม่คืนคะแนนมาให้เลย (เช่น บริการค้นหาภายนอก) ยังมีอีกทางคือใช้ **อันดับ** แทนคะแนน วิธีที่นิยมที่สุดคือ **Reciprocal Rank Fusion (RRF)** ซึ่งให้คะแนนเป็นส่วนกลับของอันดับ

นิยาม 3.21 Reciprocal Rank Fusion

$$
s_{\mathrm{RRF}}(i) = \sum_m \frac{1}{k_0 + r_m(i)}, \qquad k_0 = 60 \text{ (ค่ามาตรฐาน)} \tag{3.26}
$$

$r_m(i)$ คืออันดับของรายการ i ในแหล่งที่ m ค่าคงที่ k_{0} ทำให้ความแตกต่างระหว่างอันดับต้น ๆ ไม่รุนแรงเกินไป และทำให้รายการที่ติดอันดับกลางของหลายแหล่งมีโอกาสชนะรายการที่อยู่อันดับหนึ่งของแหล่งเดียว

ตัวอย่างการคำนวณ 3.32 RRF บน CANDIDATE ITEM ทั้งเจ็ดของ u1

ขั้นที่ 1 — ข้อมูลเข้า: อันดับจากทั้งสองแหล่ง (คอลัมน์ที่ 3 และ 5 ของตารางที่ 3.70)

ขั้นที่ 2 — แทนค่า เช่น Coco (อันดับ 3 เชิงคำ อันดับ 1 เชิงความหมาย)

$$s_{\text{RRF}}(i11) = \frac{1}{60+3} + \frac{1}{60+1} = 0.015873 + 0.016393 = 0.032266$$

ตารางที่ 3.73 คะแนน RRF เทียบกับการผสานแบบนูนที่ $w = 0.5$

รายการ	อันดับเชิงคำ	อันดับความหมาย	$\mathbf{1}/(\mathbf{60}+r_{\text{คำ}})$	$\mathbf{1}/(\mathbf{60}+r_{\text{คม}})$	คะแนน RRF	อันดับ RRF	อันดับ $w = 0.5$
i11 Coco	3	1	0.015873	0.016393	0.032266	1	1
i12 Spirited Away	1	5	0.016393	0.015385	0.031778	2	2
i9 Parasite	4	2	0.015625	0.016129	0.031754	3	3
i5 La La Land	2	7	0.016129	0.014925	0.031054	4	4
i7 John Wick	6	3	0.015152	0.015873	0.031025	5	5
i8 Mad Max: Fury Road	7	4	0.014925	0.015625	0.030550	6	6
i10 Toy Story	5	6	0.015385	0.015152	0.030536	7	7

ขั้นที่ 3 — ตีความ

RRF ให้สี่อันดับแรกเป็น i11 Coco > i12 Spirited Away > i9 Parasite > i5 La La Land ซึ่งตรงกับการผสานแบบนูนที่ $w = 0.5$ ทั้งที่ไม่เคยเห็นคะแนนดิบเลย นี่คือจุดแข็งของมัน คือใช้ได้ทันทีโดยไม่ต้องปรับมาตราและไม่ต้องจูนน้ำหนัก ข้อแลกเปลี่ยนคือมันทิ้งข้อมูลเรื่อง **ระยะห่างของคะแนน** ไป รายการที่ชนะขาดกับชนะเฉียดฉิวถูกมองเท่ากันหมด ระบบที่มีข้อมูลประเมินเพียงพอจึงมักได้ผลดีกว่าด้วยการผสานแบบนูนที่จูนน้ำหนักมาแล้ว

3.12.5 ขั้นที่สาม: จัดอันดับซ้ำด้วยตัวเข้ารหัสไขว้

ถึงตรงนี้ CANDIDATE ITEM ถูกกรองจากหลักล้านเหลือหลักสิบแล้ว จำนวนที่น้อยลงทำให้ใช้ฟังก์ชันให้คะแนนที่แพงกว่าเดิมได้ ซึ่งก็คือ **ตัวเข้ารหัสไขว้ (Cross-Encoder)** จากหัวข้อ 3.6.3 มันอ่านโปรไฟล์กับรายการ **พร้อมกันในครั้งเดียว** จึงเห็นความสัมพันธ์ระหว่างคำสองฝั่งที่เวกเตอร์เดี่ยวมองไม่เห็น แต่เพราะต้องคำนวณใหม่ทุกคู่ จึงใช้ได้เฉพาะกับ CANDIDATE ITEM จำนวนน้อย

ตัวอย่างการคำนวณ 3.33 จัดอันดับซ้ำสี่อันดับแรกของ u1 ด้วยตัวเข้ารหัสไขว้

ขั้นที่ 1 — ข้อมูลเข้า: สี่อันดับแรกจากการผสานที่ $w = 0.5$ และค่าล็อกอิต z ที่ตัวเข้ารหัสไขว้ให้กับแต่ละคู่ (ค่าสมมุติเพื่อการสาธิต) จากนั้นแปลงเป็นความน่าจะเป็นด้วยฟังก์ชันซิกมอยด์

$$P(\text{ชอบ} \mid u, i) = \sigma(z) = \frac{1}{1+e^{-z}} \tag{3.27}$$

ขั้นที่ 2 — แทนค่า เช่น Parasite (i9) ที่ $z = 1.2$

$$\sigma(1.2) = \frac{1}{1+e^{-1.2}} = \frac{1}{1+0.3012} = 0.7685$$

ตารางที่ 3.74 อันดับก่อนและหลังการจัดอันดับซ้ำด้วยตัวเข้ารหัสไขว้

รายการ	คะแนนผสาน ($w = 0.5$)	อันดับก่อน	ล็อกอิต z	$\sigma(z)$	อันดับหลัง	การเปลี่ยนแปลง
i11 Coco	0.7155	1	-0.40	0.4013	3	ลง 2 อันดับ
i12 Spirited Away	0.6832	2	0.30	0.5744	2	คงที่
i9 Parasite	0.5371	3	1.20	0.7685	1	ขึ้น 2 อันดับ
i5 La La Land	0.3913	4	-1.50	0.1824	4	คงที่

ขั้นที่ 3 — ตีความ

Parasite ขึ้นจากอันดับ 3 เป็นอันดับ 1 เพราะตัวเข้ารหัสไขว้อ่านเรื่องย่อของทั้งสองฝั่งพร้อมกันแล้วเห็นความเชื่อมโยงที่เวกเตอร์เดี่ยวมองไม่เห็น เช่น “ครอบครัวที่ซ่อนตัวอยู่ใต้บ้าน” กับ “ความจริงที่ซ่อนอยู่เบื้องหลัง” ของ The Matrix และ Inception ส่วน La La Land ที่ติดมาด้วยคำพ้องรูป dream ถูกกดลงไปอยู่ท้ายสุด ข้อสังเกตเชิงระบบ: ถ้าจัดอันดับซ้ำ 50 CANDIDATE ITEM แต่ CANDIDATE ITEM ที่ดีที่สุดหลุดไปตั้งแต่ขั้นค้นคืน ตัวเข้ารหัสไขว้ก็ช่วยอะไรไม่ได้ คุณภาพของขั้นค้นคืนจึงเป็นเพดานของทั้งระบบเสมอ

3.12.6 การปรับเทียบคะแนนให้เป็นความน่าจะเป็น

คะแนนผสานในตารางที่ 3.72 เรียงลำดับได้ แต่ **ตีความไม่ได้**: คะแนน 0.7155 ไม่ได้แปลว่าผู้ใช้มีโอกาสชอบ 71.55% การแปลงคะแนนดิบให้เป็นความน่าจะเป็นที่เชื่อถือได้เรียกว่า **การปรับเทียบ (Calibration)** ซึ่งจำเป็นเมื่อระบบต้องตั้งเกณฑ์ (เช่น ส่งการแจ้งเตือนเมื่อโอกาสชอบเกิน 80%) หรือต้องรวมคะแนนกับสิ่งที่มีหน่วยชัดเจนอย่างรายได้ต่อคลิก

$$\text{คะแนนดิบ } s \;\rightarrow\; \text{ฟังก์ชันปรับเทียบ} \;\rightarrow\; P(\text{ชอบ} \mid u, i) \tag{3.28}$$

ตัวอย่างการคำนวณ 3.34 ปรับเทียบคะแนนผสานของ u1 ด้วยวิธีของแพลตต์

ขั้นที่ 1 — ข้อมูลเข้า: คะแนนผสานที่ $w = 0.5$ และพารามิเตอร์สองตัวที่ เรียนจากข้อมูลจริง **ด้วยการถดถอยโลจิสติกบนชุดตรวจสอบ ในที่นี้ได้ $A = -6.00$ และ $B = 1.50$**

$$P(\text{ชอบ} \mid s) = \frac{1}{1+\exp(As+B)} \tag{3.29}$$

ขั้นที่ 2 — แทนค่า เช่น Coco ที่ $s = 0.7155$

$$As + B = -6.00 \times 0.7155 + 1.50 = -2.7930 \qquad P = \frac{1}{1+e^{-2.7930}} = 0.9423$$

ตารางที่ 3.75 คะแนนผสานก่อนและหลังการปรับเทียบ

รายการ	คะแนนผสาน s	$As + B$	P(ชอบ)	ผ่านเกณฑ์ 0.80 หรือไม่
i11 Coco	0.7155	-2.7930	0.9423	ผ่าน
i12 Spirited Away	0.6832	-2.5992	0.9308	ผ่าน
i9 Parasite	0.5371	-1.7226	0.8485	ผ่าน
i5 La La Land	0.3913	-0.8478	0.7001	ไม่ผ่าน
i7 John Wick	0.3421	-0.5526	0.6347	ไม่ผ่าน
i8 Mad Max: Fury Road	0.2724	-0.1344	0.5335	ไม่ผ่าน
i10 Toy Story	0.2638	-0.0828	0.5207	ไม่ผ่าน

ขั้นที่ 3 — ตีความ

อันดับไม่เปลี่ยนเลย เพราะฟังก์ชันปรับเทียบเป็นฟังก์ชันเพิ่มทางเดียว สิ่งที่ได้เพิ่มมาคือ **ความหมายของตัวเลข** ระบบตอบได้แล้วว่าควรส่งการแจ้งเตือนเฉพาะสามเรื่องแรก และหากเป็นระบบโฆษณาก็คูณค่านี้กับมูลค่าต่อคลิกเพื่อจัดอันดับตามรายได้คาดหวังได้ วิธีของแพลตต์ (Platt Scaling) เหมาะเมื่อข้อมูลตรวจสอบน้อย ส่วนการถดถอยแบบไอโซโทนิก (Isotonic Regression) ยืดหยุ่นกว่าแต่ต้องใช้ข้อมูลมากกว่า ข้อควรจำคือการปรับเทียบไม่ได้ทำให้ระบบ **แม่นขึ้น** แต่ทำให้ตัวเลขที่ระบบรายงาน **เชื่อถือได้**

3.12.7 การเรียนรู้เพื่อจัดอันดับ: ให้ข้อมูลเลือกน้ำหนักแทนเรา

การจูน w ด้วยมือทำได้เมื่อมีสองแหล่ง แต่ระบบจริงมีสัญญาณนับสิบ ทั้งความคล้ายเชิงความหมาย ความตรงของคำ ความใหม่ ความนิยม ราคา และความพร้อมของสินค้า การหาน้ำหนักด้วยมือจึงเป็นไปไม่ได้ **การเรียนรู้เพื่อจัดอันดับ (Learning to Rank: LTR)** คือการให้แบบจำลองเรียนน้ำหนักเหล่านั้นจากข้อมูลผลป้อนกลับที่ผ่านมา

$$[\text{semantic, lexical, freshness, popularity, ...}] \rightarrow \text{แบบจำลองจัดอันดับ} \rightarrow \text{คะแนนสุดท้าย} \tag{3.30}$$

ตัวอย่างการคำนวณ 3.35 จัดอันดับ CANDIDATE ITEM ของ u1 ด้วยคุณลักษณะสี่ตัว

ขั้นที่ 1 — ข้อมูลเข้า: คุณลักษณะสี่ตัวต่อหนึ่ง CANDIDATE ITEM ทุกตัวปรับให้อยู่ในช่วง [0, 1] แล้ว
โดยความใหม่คำนวณจากปี $(\text{ปี} - 1995)/(2019 - 1995)$
และความนิยมคือจำนวนผู้ใช้ที่เคยให้คะแนนหารด้วยค่าสูงสุด

ขั้นที่ 2 — น้ำหนักที่แบบจำลองเรียนได้ และการแทนค่าของ Coco

$s = 0.50\,x_1 + 0.20\,x_2 + 0.15\,x_3 + 0.15\,x_4$ $s_{i11} = 0.50 \times 1.0000 + 0.20 \times 0.4309 + 0.15 \times 0.9167 + 0.15 \times 1.0000 = 0.8737$

ตารางที่ 3.76 คุณลักษณะสี่ตัวและคะแนนสุดท้ายจากแบบจำลองจัดอันดับ

รายการ	ปี	ความหมาย (0.50)	เชิงคำ (0.20)	ความใหม่ (0.15)	ความนิยม (0.15)	คะแนนสุดท้าย
i11 Coco	2017	1.0000	0.4309	0.9167	1.0000	0.8737
i9 Parasite	2019	0.6705	0.4037	1.0000	0.2500	0.6035
i7 John Wick	2014	0.5712	0.1130	0.7917	0.5000	0.5020
i12 Spirited Away	2001	0.3663	1.0000	0.2500	0.5000	0.4956
i8 Mad Max: Fury Road	2015	0.5448	0.0000	0.8333	0.5000	0.4724
i5 La La Land	2016	0.0000	0.7827	0.8750	0.7500	0.4003
i10 Toy Story	1995	0.2468	0.2809	0.0000	0.5000	0.2546

ขั้นที่ 3 — ตีความ

Spirited Away ร่วงจากอันดับ 2 ของการผสานสองแหล่ง มาอยู่อันดับ 4 เพราะเป็นภาพยนตร์ปี 2001 ที่ได้คะแนนความใหม่ต่ำ ส่วน Parasite (2019) ขึ้นมาอันดับ 2 ด้วยคะแนนความใหม่เต็ม นี่แสดงให้เห็นว่า LTR ทำให้เป้าหมายเชิงธุรกิจเข้ามาอยู่ในสูตรได้อย่างเป็นระบบ แต่ก็เตือนให้ระวังด้วยว่า **คุณลักษณะที่ไม่เกี่ยวกับเนื้อหาสามารถกลบสัญญาณเนื้อหาได้** ถ้าน้ำหนักความนิยมสูงเกินไป ระบบจะค่อย ๆ กลายเป็นรายการยอดนิยมที่เหมือนกันสำหรับทุกคน ซึ่งวัดได้ด้วยตัวชี้วัด Coverage และ Novelty ในหัวข้อ 3.14.3

3.12.8 บทบาทของแบบจำลองภาษาขนาดใหญ่

แบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM) เข้ามาช่วยระบบแนะนำอิงเนื้อหาได้ในหลายจุด แต่จุดที่คุ้มค่าที่สุดคือ **ส่วนวิเคราะห์เนื้อหา** ไม่ใช่ส่วนจัดอันดับ เพราะส่วนวิเคราะห์เนื้อหาทำงานล่วงหน้าได้ ขณะที่ส่วนจัดอันดับต้องตอบภายในมิลลิวินาทีและต้องทำซ้ำได้

ตารางที่ 3.77 ห้าบทบาทของ LLM ในระบบแนะนำอิงเนื้อหา และความเหมาะสมของแต่ละบทบาท

บทบาท	ทำอะไร	ทำเมื่อใด	ข้อควรระวัง
เติมเนื้อหาให้รายการใหม่	สรุปเรื่องย่อ เติมแท็ก แนว อารมณ์ กลุ่มผู้ชม จากข้อมูลที่มีน้อย	ล่วงหน้า (ต่อรายการ)	ต้องตรวจความถูกต้อง เพราะแบบจำลองแต่งข้อมูลที่ไม่มีจริงได้
สร้างเวกเตอร์ความหมาย	ใช้เป็นตัวเข้ารหัสสร้าง v_i แทน BERT ขนาดเล็ก	ล่วงหน้า (ต่อคลัง)	ต้นทุนสูง และต้องสร้างใหม่ทั้งคลังเมื่อเปลี่ยนรุ่นแบบจำลอง
เขียนคำอธิบายคำแนะนำ	แปลงส่วนร่วมรายมิติจากหัวข้อ 3.9.3 เป็นประโยคที่ผู้ใช้อ่านเข้าใจ	สด (เฉพาะรายการที่แสดง)	คำอธิบายต้องมาจากคะแนนจริง ไม่ใช่ให้แบบจำลองเดาเหตุผลเอง

บทบาท	ทำอะไร	ทำเมื่อใด	ข้อควรระวัง
ตีความคำขอของผู้ใช้	แปลง “อยากดูอะไรเบา ๆ ก่อนนอน” เป็นเงื่อนไขค้นหาและตัวกรอง	สด	ต้องจำกัดผลลัพธ์ให้อยู่ในรูปแบบที่ระบบรับได้
จัดอันดับซ้ำจำนวนน้อย	เรียง CANDIDATE ITEM 10–20 รายการสุดท้ายตามคำอธิบายภาษาธรรมชาติ	สด (แพงมาก)	ช้า ราคาแพง ผลไม่คงที่ และอธิบายย้อนกลับได้ยาก

ประเด็นสำคัญ ข้อสรุปเรื่อง LLM

LLM มีประโยชน์ที่สุดกับ **ปัญหารายการใหม่ (Item Cold Start)** เพราะเติมคำบรรยายให้รายการที่แทบไม่มีข้อมูลได้ทันที และมีประโยชน์รองลงมาในการเขียนคำอธิบาย แต่ **ไม่ควรแทนที่ส่วนจัดอันดับทั้งหมด** ด้วยเหตุผลสามข้อคือ ความหน่วงเวลา ต้นทุน และการทำซ้ำไม่ได้ ระบบที่ออกแบบดีจึงใช้ LLM เป็นเครื่องมือเสริมของขั้นที่ 1–2 ในตารางที่ 3.64 แล้วปล่อยให้ขั้นที่ 4–5 ยังทำงานด้วยคณิตศาสตร์ที่ตรวจสอบได้ตามที่บทนี้อธิบายมาทั้งหมด

3.13 ความหลากหลายและการจัดอันดับซ้ำด้วย MMR

3.13.1 อาการเฉพาะทางเกินไป

ระบบอิงเนื้อหาสร้างโปรไฟล์จากสิ่งที่ผู้ใช้เคยชอบ แล้วแนะนำสิ่งที่คล้ายกับโปรไฟล์ ผลลัพธ์ที่ตามมาอย่างหลีกเลี่ยงไม่ได้คือรายการแนะนำที่ **คล้ายกันเองด้วย** อาการนี้เรียกว่า **ความเฉพาะทางเกินไป (Over-Specialization)** และเป็นข้อจำกัดที่รุนแรงที่สุดของแนวทางนี้

ตัวอย่างการคำนวณ 3.36 สามอันดับแรกของ u1 ในปริภูมิความหมายคล้ายกันเองเพียงใด

ขั้นที่ 1 — ข้อมูลเข้า: CANDIDATE ITEM หกอันดับแรกจากตารางที่ 3.58 พร้อมความคล้ายระหว่างกันเอง

ตารางที่ 3.78 คะแนนความเกี่ยวข้อง $\mathrm{rel}(i)$ และเมทริกซ์ความคล้ายระหว่าง CANDIDATE ITEM หกเรื่อง

รายการ	$\mathrm{rel}(i)$	i11	i9	i7	i8	i12	i10
i11 Coco	0.4032	—	0.4714	0.0774	0.0000	0.3850	0.2551
i9 Parasite	0.2734	0.4714	—	0.3935	0.1332	0.1670	0.2459
i7 John Wick	0.2343	0.0774	0.3935	—	0.3881	0.1952	0.0707
i8 Mad Max: Fury Road	0.2239	0.0000	0.1332	0.3881	—	0.0617	0.0666
i12 Spirited Away	0.1536	0.3850	0.1670	0.1952	0.0617	—	0.0603
i10 Toy Story	0.1065	0.2551	0.2459	0.0707	0.0666	0.0603	—

ขั้นที่ 2 — สามอันดับแรกถ้าเรียงตามคะแนนอย่างเดียวคือ Coco, Parasite, John Wick วัดความคล้ายเฉลี่ยของคู่ในรายการนี้

$$\overline{\text{sim}} = \frac{\text{sim}(i11,i9)+\text{sim}(i11,i7)+\text{sim}(i9,i7)}{3} = \frac{0.4714+0.0774+0.3935}{3} = 0.3141$$

ขั้นที่ 3 — ตีความ

Coco กับ Parasite คล้ายกันถึง 0.4714 (ทั้งคู่เป็นเรื่องของครอบครัว) และ Parasite กับ John Wick คล้ายกัน 0.3935 (ทั้งคู่มีมิติ crime) รายการแนะนำจึงพูดเรื่องเดียวกันซ้ำสามครั้ง ผลเสียมีสองชั้น ชั้นแรกคือผู้ใช้ได้ข้อมูลใหม่น้อย เพราะถ้าไม่ชอบเรื่องแรกก็มักไม่ชอบอีกสองเรื่อง ชั้นที่สองคือระบบไม่มีทางค้นพบความสนใจใหม่ของผู้ใช้เลย ซึ่งเรียกว่า **ฟองกรอง (Filter Bubble)**

3.13.2 MMR: เลือกทีละรายการโดยหักลบความซ้ำ

ทางแก้ที่ตรงไปตรงมาที่สุดคือเปลี่ยนเกณฑ์การเลือกจาก “คะแนนสูงสุด” เป็น “คะแนนสูงและต่างจากสิ่งที่เลือกไปแล้ว” วิธีมาตรฐานที่ทำเช่นนี้เรียกว่า **Maximal Marginal Relevance (MMR)** ซึ่งเลือกรายการ **ทีละหนึ่ง** และทุกครั้งที่เลือก จะหักคะแนนของ CANDIDATE ITEM ที่คล้ายกับสิ่งที่เลือกไปแล้ว

นิยาม 3.22 Maximal Marginal Relevance

$$\text{MMR}(i) = \lambda\,\text{rel}(u,i) \;-\; (1-\lambda)\max_{j\in S}\text{sim}(i,j) \tag{3.31}$$

S คือเซตของรายการที่เลือกไปแล้ว (รอบแรก S ว่าง จึงให้พจน์หลังเป็น 0) $\text{rel}(u,i)$ คือคะแนนจับคู่จากหัวข้อ 3.9 และ $\text{sim}(i,j)$ คือความคล้ายระหว่างรายการสองรายการ ค่า $\lambda \in [0,1]$ คือปุ่มปรับ: $\lambda = 1$ คือไม่สนใจความหลากหลายเลย (ได้ผลเท่ากับการเรียงตามคะแนน) และ λ ยิ่งน้อยยิ่งให้น้ำหนักความต่างมากขึ้น

ขั้นตอนวิธี MMR สำหรับเลือก K รายการ

1. ตั้ง $S = \varnothing$ และให้ C เป็นเซต CANDIDATE ITEM ที่ผ่านการจัดอันดับมาแล้ว (ปกติ 20–100 รายการแรก)
2. ทำซ้ำ K รอบ: คำนวณ $\text{MMR}(i)$ ของทุก $i \in C \backslash S$ ตามสมการ (3.31)
3. เลือก i^* ที่ให้ค่าสูงสุด ย้ายเข้า S
4. คืน S เป็นรายการแนะนำตามลำดับที่ถูกเลือก

ต้นทุนคือ $O(K \times |C|)$ ครั้งของการคำนวณความคล้าย ซึ่งถูกมากเมื่อ $|C|$ อยู่ในหลักสิบ MMR จึงเป็นขั้นตอนสุดท้ายที่แทบไม่เพิ่มภาระให้ระบบเลย

ตัวอย่างการคำนวณ 3.37 เดิน MMR ครบสามรอบที่ $\lambda = 0.7$

ขั้นที่ 1 — ข้อมูลเข้า: คะแนนความเกี่ยวข้องและเมทริกซ์ความคล้ายจากตารางที่ 3.78

ขั้นที่ 2 — รอบที่ 1: S ว่าง พจน์ความคล้ายเป็น 0 ทุกตัว จึงเหลือ $0.7 \times \text{rel}(i)$

ตารางที่ 3.79 รอบที่ 1 ของ MMR ($\lambda = 0.7$)

รายการ	$\text{rel}(i)$	$\max_{j \in S} \text{sim}(i, j)$	$0.7\,\text{rel} - 0.3\,\max$	ผล
i11 Coco	0.4032	0.0000	0.2822	**เลือก**
i9 Parasite	0.2734	0.0000	0.1914	
i7 John Wick	0.2343	0.0000	0.1640	
i8 Mad Max: Fury Road	0.2239	0.0000	0.1567	
i12 Spirited Away	0.1536	0.0000	0.1075	
i10 Toy Story	0.1065	0.0000	0.0745	

ขั้นที่ 3 — รอบที่ 2: $S = \{i11\}$ CANDIDATE ITEM ทุกตัวถูกหักด้วยความคล้ายกับ Coco เช่น Parasite

$$\text{MMR}(i9) = 0.7 \times 0.2734 - 0.3 \times 0.4714 = 0.1914 - 0.1414 = 0.0500$$

ตารางที่ 3.80 รอบที่ 2 ของ MMR หลังเลือก Coco แล้ว

รายการ	$\text{rel}(i)$	$\text{sim}(i, i11)$	$0.7\,\text{rel} - 0.3\,\text{sim}$	ผล
i9 Parasite	0.2734	0.4714	0.0500	
i7 John Wick	0.2343	0.0774	0.1408	
i8 Mad Max: Fury Road	0.2239	0.0000	0.1567	**เลือก**
i12 Spirited Away	0.1536	0.3850	-0.0080	
i10 Toy Story	0.1065	0.2551	-0.0020	

ขั้นที่ 4 — รอบที่ 3: $S = \{i11, i8\}$ พจน์ที่หักคือความคล้าย สูงสุด กับสมาชิกใน S

ตารางที่ 3.81 รอบที่ 3 ของ MMR หลังเลือก Coco และ Mad Max แล้ว

รายการ	$\text{rel}(i)$	$\max(\text{sim กับ } i11, i8)$	ค่า MMR	ผล
i9 Parasite	0.2734	0.4714	0.0500	**เลือก**
i7 John Wick	0.2343	0.3881	0.0476	
i12 Spirited Away	0.1536	0.3850	-0.0080	

รายการ	$\mathbf{rel}(i)$	$\max(\text{sim กับ } i11, i8)$	ค่า MMR	ผล
i10 Toy Story	0.1065	0.2551	-0.0020	

ขั้นที่ 5 — ตีความ

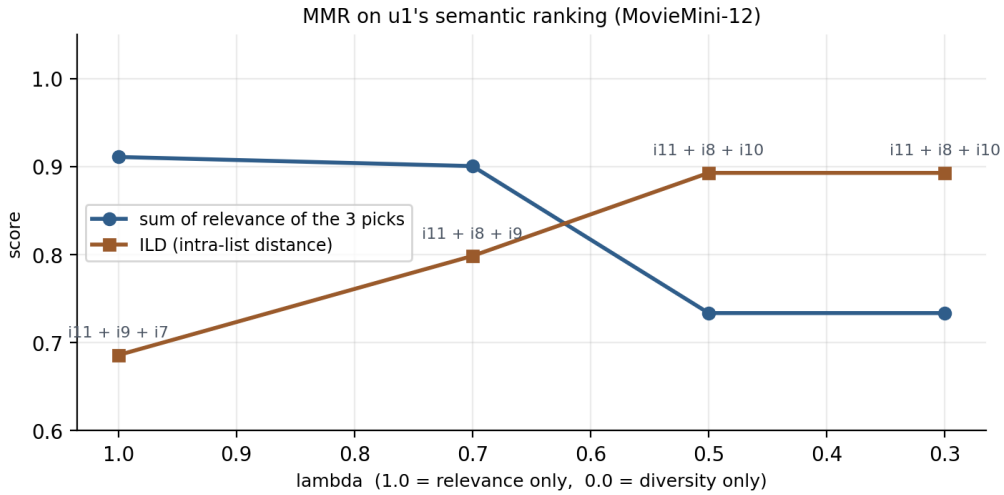
รายการสุดท้ายคือ i11 Coco ⟶ i8 Mad Max: Fury Road ⟶ i9 Parasite ต่างจากการเรียงตามคะแนนล้วน (i11 Coco ⟶ i9 Parasite ⟶ i7 John Wick) ตรงที่ **Mad Max แทรกขึ้นมาเป็นอันดับสอง Parasite ถูกเลื่อนไปอันดับสาม และ John Wick หลุดออกจากรายการ** เหตุผลอ่านได้จากตัวเลขในรอบที่ 2 โดยตรง: Parasite มีคะแนนความเกี่ยวข้องสูงกว่า Mad Max (0.2734 เทียบกับ 0.2239) แต่มันคล้าย Coco ถึง 0.4714 ขณะที่ Mad Max คล้าย Coco เพียง 0.0000 ค่าที่หักออกจึงพลิกผลการเลือก ผลที่ได้คือรายการแนะนำที่ครอบคลุมสามรสนิยมของ u1 พร้อมกัน คือ แอนิเมชันเรื่องความทรงจำ หนังแอ็กชันไซไฟ และหนังอาชญากรรม แทนที่จะเป็นเรื่องครอบครัวสามเรื่องติดกัน

3.13.3 เลือกค่า λ อย่างไร

ค่า λ ไม่มีค่าที่ถูกต้องสากล ต้องเลือกจากการวัด ตารางที่ 3.82 แสดงผลของ λ สี่ค่าบน CANDIDATE ITEM ชุดเดียวกัน โดยวัดสองด้านพร้อมกันคือ **ผลรวมความเกี่ยวข้อง** ของสามรายการที่เลือก และ **ระยะห่างภายในรายการ (Intra-List Distance: ILD)** ซึ่งเป็นค่าเฉลี่ยของ $1 - \text{sim}$ ระหว่างทุกคู่ในรายการแนะนำ

ตารางที่ 3.82 ผลของค่า λ ต่อรายการแนะนำ ความเกี่ยวข้องรวม และความหลากหลาย

λ	รายการที่เลือก (ตามลำดับ)	ผลรวม $\mathbf{rel}$	ILD	ลักษณะของผลลัพธ์
1.00	i11 Coco ⟶ i9 Parasite ⟶ i7 John Wick	0.9109	0.6859	เหมือนเรียงตามคะแนนทุกประการ
0.70	i11 Coco ⟶ i8 Mad Max: Fury Road ⟶ i9 Parasite	0.9005	0.7985	เสียความเกี่ยวข้องเพียง 1% แต่ ILD เพิ่ม 16%
0.50	i11 Coco ⟶ i8 Mad Max: Fury Road ⟶ i10 Toy Story	0.7336	0.8928	เสียความเกี่ยวข้อง 19% เพื่อ ILD เพิ่ม 30%
0.30	i11 Coco ⟶ i8 Mad Max: Fury Road ⟶ i10 Toy Story	0.7336	0.8928	ผลเท่ากับ 0.5 เพราะ CANDIDATE ITEM มีน้อย



ภาพที่ 3.8 การแลกเปลี่ยนระหว่างความเกี่ยวข้องกับความหลากหลายเมื่อลดค่า λ จุดที่คุ้มที่สุดของตัวอย่างนี้คือ $\lambda = 0.7$ ซึ่งเสียความเกี่ยวข้องเล็กน้อยแต่ได้ความหลากหลายเพิ่มชัดเจน

รูปแบบในภาพที่ 3.8 เกิดซ้ำในระบบจริงเสมอ คือช่วงแรกของการลด λ ได้ความหลากหลายมาเกือบฟรี แล้วจึงเริ่มจ่ายแพงขึ้นเรื่อย ๆ ค่าที่ใช้ได้ผลดีในทางปฏิบัติจึงมักอยู่ระหว่าง 0.6 ถึง 0.8 การตัดสินใจขั้นสุดท้ายควรมาจากการทดลองกับผู้ใช้จริง เพราะความหลากหลายส่งผลต่อพฤติกรรมระยะยาวที่ตัวชี้วัดออฟไลน์มองไม่เห็น

ประเด็นสำคัญ ความหลากหลายมีหลายความหมาย เลือกให้ตรงกับปัญหา

- **ความหลากหลายภายในรายการ (Intra-List Diversity)** คือสิ่งที่ MMR แก้ วัดด้วย ILD ตามหัวข้อ 3.14.3
- **ผลกระทบเชิงพอร์ต (Portfolio Effect)** คือการแนะนำรายการที่แทบเป็นสิ่งเดียวกัน เช่น ภาพยนตร์เรื่องเดียวกันฉบับพากย์ต่างภาษา แก้ด้วยการรวมกลุ่มรายการซ้ำก่อนจัดอันดับ
- **ความหลากหลายข้ามเวลา** คือการไม่แนะนำสิ่งเดิมซ้ำทุกวัน แก้ด้วยการจำว่าเคยแสดงอะไรไปแล้วและลดคะแนนลง
- **ความเป็นธรรมต่อผู้ผลิตเนื้อหา** คือการกระจายโอกาสให้รายการหางยาว วัดด้วย Coverage ในหัวข้อ 3.14.3
- นอกจาก MMR ยังมีวิธีที่พิจารณาความหลากหลายของทั้งชุดพร้อมกัน เช่น **Determinantal Point Process (DPP)** ซึ่งให้ผลดีกว่าเล็กน้อยแต่คำนวณซับซ้อนกว่ามาก

3.14 การประเมินผลของระบบ

ทุกทางเลือกในบทนี้ ตั้งแต่วิธีสร้างเวกเตอร์ วิธีสร้างโปรไฟล์ ค่าน้ำหนัก w ของการผสาน ไปจนถึงค่า λ ของ MMR ล้วนต้องตัดสินด้วย **ตัวเลข** ไม่ใช่ด้วยความรู้สึก หัวข้อนี้จึงเป็นขั้นที่ 6 ของตารางที่ 3.64 และเป็นหัวข้อที่ทำให้ทุกหัวข้อก่อนหน้านำไปใช้จริงได้

3.14.1 แบ่งข้อมูลอย่างไรจึงจะเชื่อผลได้

การประเมินระบบแนะนำคือการ **ซ่อนความจริงบางส่วนแล้วดูว่าระบบทายกลับมาได้หรือไม่** สิ่งที่ตัดสินความน่าเชื่อถือของผลคือวิธีซ่อน ไม่ใช่ตัวชี้วัด

ตารางที่ 3.83 สามวิธีแบ่งข้อมูลเพื่อการประเมิน

วิธีแบ่ง	ทำอย่างไร	เหมาะเมื่อ	ข้อควรระวัง
กันออกทีละรายการ (Leave-One-Out)	ซ่อนรายการที่ชอบทีละหนึ่ง สร้างโปรไฟล์จากที่เหลือ ทำซ้ำทุกรายการ	ข้อมูลน้อยมาก เช่น ตัวอย่างในบทนี้	ใช้ข้อมูลอนาคตสร้างโปรไฟล์ได้ จึงมองโลกในแง่ดีเกินจริง
แบ่งตามเวลา (Time Split)	ใช้ผลป้อนกลับก่อนเวลา T ฝึก และหลังเวลา T ทดสอบ	ระบบจริงทุกกรณี	ต้องมีเวลาประทับของผลป้อนกลับ
แบ่งแบบสุ่ม (Random Split)	สุ่มแบ่งผลป้อนกลับเป็นชุดฝึกกับชุดทดสอบ	ไม่ควรใช้กับระบบแนะนำ	แบบจำลองเห็นอนาคตของผู้ใช้คนเดียวกัน ผลดีเกินจริงมาก

บทนี้ใช้การกันออกทีละรายการเพราะ MovieMini-12 มีผลป้อนกลับเพียง 20 รายการที่ผู้ใช้ชอบ (คะแนน 4 ขึ้นไป) และไม่มีเวลาประทับ ผู้อ่านต้องจำไว้เสมอว่าตัวเลขที่ได้เป็น **ขอบบน** ของสมรรถนะจริง ในโครงงานที่มีเวลาประทับ ให้ใช้การแบ่งตามเวลาเสมอ

3.14.2 ตัวชี้วัดความแม่นยำ

นิยาม 3.23 ตัวชี้วัดสามตัวเมื่อมีรายการที่ถูกต้องหนึ่งรายการต่อหนึ่งรอบ

$$\mathrm{HR@}K = \frac{1}{n}\sum_{r=1}^{n} \mathbb{1}\left[\,\mathrm{rank}_r \le K\,\right] \qquad \mathrm{MRR} = \frac{1}{n}\sum_{r=1}^{n} \frac{1}{\mathrm{rank}_r} \qquad \mathrm{nDCG@}K = \frac{1}{n}\sum_{r=1}^{n} \frac{\mathbb{1}[\,\mathrm{rank}_r \le K\,]}{\log_2(\mathrm{rank}_r + 1)} \tag{3.32}$$

- **HR@K (Hit Rate)** ตอบว่า “ติดอันดับที่แสดงจริงหรือไม่” เป็นศูนย์หรือหนึ่ง ไม่สนใจว่าอยู่อันดับที่เท่าใดในนั้น
- **MRR (Mean Reciprocal Rank)** ให้รางวัลแก่การอยู่อันดับต้น ๆ อันดับ 1 ได้ 1.0 อันดับ 2 ได้ 0.5 อันดับ 10 ได้ 0.1
- **nDCG@K** ลดค่าตามลอการิทึมของอันดับ จึงอยู่ระหว่าง HR กับ MRR และเป็นตัวชี้วัดที่ใช้กันมากที่สุดเมื่อมีรายการที่ถูกต้องหลายรายการ
- **Precision@K และ Recall@K** ใช้เมื่อมีรายการที่ถูกต้องหลายรายการต่อรอบ โดย $\mathrm{P@}K$ คือสัดส่วนของรายการที่ถูกต้องใน K อันดับแรก และ $\mathrm{R@}K$ คือสัดส่วนของรายการที่ถูกต้องทั้งหมดที่ระบบดึงกลับมาได้

ตัวอย่างการคำนวณ 3.38 คำนวณตัวชี้วัดทั้งสามของ u1 จากผลการกันออกทีละรายการ

ขั้นที่ 1 — ข้อมูลเข้า: อันดับที่เรื่องที่ซ่อนไว้กลับมา จากตารางที่ 3.63 สามรอบ ในสามปริภูมิ

ตารางที่ 3.84 อันดับของเรื่องที่ซ่อนและตัวชี้วัดของ u1 แยกตามปริภูมิ

ปริภูมิ	รอบ 1 (ซ่อน i1)	รอบ 2 (ซ่อน i2)	รอบ 3 (ซ่อน i3)	HR@3	MRR	nDCG@3
แนวภาพยนตร์	1	2	2	1.0000	0.6667	0.7539
TF-IDF	3	1	5	0.6667	0.5111	0.5000
ความหมาย	1	1	7	0.6667	0.7143	0.6667

ขั้นที่ 2 — แทนค่าของปริภูมิความหมาย (อันดับ 1, 1 และ 7)

$$\text{HR@3} = \frac{1+1+0}{3} = 0.6667 \qquad \text{MRR} = \frac{\frac{1}{1}+\frac{1}{1}+\frac{1}{7}}{3} = \frac{2.1429}{3} = 0.7143$$

$$\text{nDCG@3} = \frac{\frac{1}{\log_2 2}+\frac{1}{\log_2 2}+0}{3} = \frac{1+1+0}{3} = 0.6667$$

ขั้นที่ 3 — ตีความ

สามตัวชี้วัดจัดอันดับปริภูมิไม่ตรงกัน ปริภูมิแนวชนะด้าน HR@3 (ติดสามอันดับแรกทุกรอบ) แต่ปริภูมิความหมายชนะด้าน MRR (ได้อันดับหนึ่งสองรอบ) บทเรียนคือ **ต้องเลือกตัวชี้วัดให้ตรงกับหน้าจอจริง**: ถ้าหน้าจอแสดงสามรายการเท่ากันหมด HR@3 คือสิ่งที่สำคัญ แต่ถ้าผู้ใช้มักคลิกรายการแรกเท่านั้น MRR สำคัญกว่า และตัวเลขจากผู้ใช้คนเดียวสามรอบยังเชื่อไม่ได้ ต้องขยายไปทั้งแปดคน

ตัวอย่างการคำนวณ 3.39 ประเมินทั้งแปดผู้ใช้ 20 รอบ

ขั้นที่ 1 — ข้อมูลเข้า: ผู้ใช้แปดคนมีรายการที่ชอบ (คะแนน 4 ขึ้นไป) รวม 20 รายการ จึงได้ 20 รอบการทดลอง แต่ละรอบซ่อนหนึ่งรายการ

ตารางที่ 3.85 อันดับที่เรื่องที่ซ่อนไว้กลับมาในแต่ละรอบ แยกตามผู้ใช้และปริภูมิ (ตัวเลขยิ่งน้อยยิ่งดี)

ผู้ใช้	รายการที่ชอบ	อันดับ: ปริภูมิแนว	อันดับ: TF-IDF	อันดับ: ความหมาย
u1	i1, i2, i3	1, 2, 2	3, 1, 5	1, 1, 7
u2	i4, i5, i6	1, 2, 1	2, 1, 1	1, 1, 1
u3	i10, i11, i12	1, 1, 1	6, 5, 4	5, 1, 3
u4	i7, i8, i1	3, 1, 1	8, 9, 6	2, 2, 3
u5	i9, i3	6, 2	9, 7	1, 6
u6	i2, i5	4, 7	1, 1	8, 11
u7	i12, i11	2, 2	4, 4	3, 1
u8	i6, i4	1, 1	1, 2	1, 1

ขั้นที่ 2 — รวมทุกรอบเป็นตัวชี้วัดเดียว และเพิ่มระบบผสาน (ปรับมาตราแล้วเฉลี่ยเท่ากันที่ $w = 0.5$ ตามหัวข้อ 3.12.3)

ตารางที่ 3.86 ตัวชี้วัดความแม่นยำจาก 20 รอบการทดลอง

ระบบ	จำนวนรอบ	HR@3	MRR	nDCG@3	อันดับความแม่นยำ
ปริภูมิแนวภาพยนตร์ (9 มิติ)	20	0.8500	0.6946	0.7143	ที่ 1
ปริภูมิความหมาย (12 มิติ)	20	0.7500	0.6363	0.6381	ที่ 2

ระบบ	จำนวนรอบ	HR@3	MRR	nDCG@3	อันดับความแม่นยำ
ผสาน TF-IDF + ความหมาย ($w = 0.5$)	20	0.5500	0.5514	0.4815	ที่ 3
ปริภูมิ TF-IDF (203 มิติ)	20	0.4500	0.4653	0.3881	ที่ 4

ขั้นที่ 3 — ตีความ

ผลลัพธ์ขัดกับสัญชาตญาณอย่างมีประโยชน์ **ปริภูมิที่ง่ายที่สุดชนะ**: เวกเตอร์แนวภาพยนตร์เพียง 9 มิติที่มนุษย์ติดป้ายไว้ ให้ HR@3 = 0.8500 สูงกว่าปริภูมิ TF-IDF 203 มิติที่ได้เพียง 0.4500 เหตุผลคือคลังนี้มีเรื่องย่อสั้นมาก คำร่วมจึงน้อยและมักเป็นคำร่วมโดยบังเอิญ (42 จาก 66 คู่มีคะแนนเป็นศูนย์ตามหัวข้อ 3.4.5) ขณะที่ป้ายแนวสะอาดและตรงกับรสนิยมของผู้ใช้ในชุดข้อมูลนี้โดยตรง ที่น่าสนใจกว่านั้นคือ **การผสานไม่ได้ดีกว่าองค์ประกอบที่ดีที่สุดเสมอไป**: ระบบผสานได้ 0.5500 ซึ่งอยู่ระหว่างสองแหล่งที่นำมารวม เพราะแหล่งที่อ่อน (TF-IDF) ดึงค่าเฉลี่ยลง ข้อสรุปเชิงปฏิบัติจึงมีสองข้อ หนึ่งคือ **ต้องวัดก่อนเชื่อ** และสองคือ **คุณลักษณะที่มนุษย์ติดป้ายไว้แล้วมีค่ามาก** อย่ารีบทิ้งไปเพื่อใช้แบบจำลองที่ซับซ้อนกว่า

ข้อควรระวัง ตัวเลขชุดนี้บอกอะไรไม่ได้บ้าง

คลังมี 12 รายการ CANDIDATE ITEM ในแต่ละรอบมีประมาณ 10 รายการ การเดาสุ่มจึงได้ HR@3 ประมาณ 0.30 อยู่แล้ว และ 20 รอบน้อยเกินกว่าจะมีนัยสำคัญทางสถิติ ผลนี้ใช้เพื่อ **เรียนรู้วิธีคำนวณและวิธีอ่านผล** เท่านั้น ไม่ใช่ข้อสรุปว่าป้ายแนวดีกว่า TF-IDF ในทุกงาน ในคลังจริงที่มีเรื่องย่อยาวและรายการนับแสนลำดับนี้มักกลับกัน

3.14.3 ตัวชี้วัดที่ความแม่นยำมองไม่เห็น

ระบบที่แนะนำเฉพาะรายการยอดนิยมสิบอันดับแรกให้ทุกคนจะได้คะแนนความแม่นยำสูงพอสมควร แต่ไร้ประโยชน์ในเชิงธุรกิจ เพราะผู้ใช้ค้นพบสิ่งใหม่ไม่ได้และรายการหางยาวไม่มีโอกาสถูกเห็นเลย จึงต้องวัดอีกสามด้านควบคู่กันเสมอ

นิยาม 3.24 ความหลากหลาย ความครอบคลุม และความแปลกใหม่

$$\text{ILD} = \frac{2}{K(K-1)} \sum_{a<b} \left(1 - \text{sim}(i_a, i_b)\right) \qquad \text{Coverage} = \frac{\left|\bigcup_u \text{Top-}K(u)\right|}{|I|} \qquad \text{Novelty} = \frac{1}{K} \sum_i -\log_2 \frac{|U_i|}{|U|} \tag{3.33}$$

- **ILD (Intra-List Distance)** วัดว่ารายการในหน้าจอเดียวกันต่างกันเพียงใด ยิ่งสูงยิ่งหลากหลาย เป็นตัวชี้วัดที่ MMR ในหัวข้อ 3.13 ตั้งใจเพิ่มโดยตรง
- **Coverage** วัดว่าคลังถูกแนะนำไปกี่ส่วนเมื่อรวมผู้ใช้ทุกคน ค่าต่ำแปลว่าระบบวนอยู่กับรายการไม่กี่ชิ้น
- **Novelty** วัดความ “ไม่เป็นที่รู้จัก” ของรายการที่แนะนำ โดย $|U_i|$ คือจำนวนผู้ใช้ที่เคยให้ผลป้อนกลับกับรายการ i รายการที่คนรู้จักน้อยได้ค่าสูง

ตัวอย่างการคำนวณ 3.40 วัดสามด้านนี้กับรายการแนะนำ Top-3 ของผู้ใช้ทั้งแปดคน

ขั้นที่ 1 — ข้อมูลเข้า: ความนิยมของแต่ละรายการ (จำนวนผู้ใช้ที่เคยให้คะแนน จากผู้ใช้ทั้งหมด 8 คน) และค่าความแปลกใหม่ของแต่ละรายการ

ตารางที่ 3.87 ความนิยมและค่าความแปลกใหม่รายรายการ

รายการ	i1	i2	i3	i4	i5	i6	i7	i8	i9	i10	i11	i12
จำนวนผู้ใช้ $\lvert U_i \rvert$	3	3	3	3	3	4	2	2	1	2	4	2
สัดส่วน $\lvert U_i \rvert / 8$	0.38	0.38	0.38	0.38	0.38	0.50	0.25	0.25	0.12	0.25	0.50	0.25
$-\log_2(\lvert U_i \rvert / 8)$	1.42	1.42	1.42	1.42	1.42	1.00	2.00	2.00	3.00	2.00	1.00	2.00

ขั้นที่ 2 — แทนค่าของ u1 ในปริภูมิความหมาย ซึ่งได้ Top-3 เป็น i11 Coco, i9 Parasite, i7 John Wick

$$\text{Novelty} = \frac{-\log_2\frac{4}{8} - \log_2\frac{1}{8} - \log_2\frac{2}{8}}{3} = \frac{1+3+2}{3} = 2.0000 \qquad \text{ILD} = 1 - 0.3141 = 0.6859$$

ขั้นที่ 3 — รวมทั้งแปดผู้ใช้ และเทียบสามปริภูมิพร้อมกับความแม่นยำจากตารางที่ 3.86

ตารางที่ 3.88 ความแม่นยำ ความหลากหลาย ความครอบคลุม และความแปลกใหม่ ของสามปริภูมิ

ปริภูมิ	HR@3	ILD เฉลี่ย	รายการที่ถูกแนะนำ (จาก 12)	Coverage	Novelty เฉลี่ย
แนวภาพยนตร์	0.8500	0.7848	9	0.7500	1.7523
TF-IDF	0.4500	0.7448	9	0.7500	1.6659
ความหมาย	0.7500	0.7364	9	0.7500	1.7877

ขั้นที่ 4 — ตีความ

สามปริภูมิครอบคลุมคลังได้เท่ากันที่ 9 จาก 12 รายการ แต่ **เป็นคนละ 9 รายการ**: ปริภูมิแนวไม่เคยแนะนำ The Notebook, Coco, Spirited Away เลย ขณะที่ปริภูมิความหมายไม่เคยแนะนำ Interstellar, Toy Story, Spirited Away และปริภูมิ TF-IDF ไม่เคยแนะนำ Interstellar, John Wick, Toy Story เมื่อรวมทั้งสามปริภูมิเข้าด้วยกัน คลังถูกแนะนำครบทั้ง 12 รายการ ซึ่งเป็นเหตุผลเชิงความครอบคลุมอีกข้อหนึ่งของการผสานหลายแหล่งในหัวข้อ 3.12 ปริภูมิแนวได้ทั้งความแม่นยำสูงสุดและ ILD สูงสุด ซึ่งเกิดจากการที่แต่ละเรื่องมีเพียงสองแนวเวกเตอร์จึงตั้งฉากกันบ่อย ส่วนปริภูมิความหมายได้ Novelty สูงสุด (1.7877) เพราะแนะนำ Parasite ซึ่งมีผู้ใช้เพียงคนเดียวเคยให้คะแนน **ข้อสรุป**: อย่ารายงานตัวชี้วัดความแม่นยำเพียงตัวเดียว ให้รายงานอย่างน้อยสามตัวคู่กันเสมอ คือความแม่นยำ ความหลากหลาย และความครอบคลุม

3.14.4 ออฟไลน์ ออนไลน์ และการเลือกตัวชี้วัดให้ตรงเป้าหมาย

ตารางที่ 3.89 เลือกตัวชี้วัดจากคำถามที่ต้องการตอบ

คำถามของผู้ออกแบบ	ตัวชี้วัดที่ตอบได้	วัดที่ไหน
ระบบดึงสิ่งที่ผู้ใช้ชอบกลับมาได้หรือไม่	HR@K, Recall@K	ออฟไลน์
สิ่งที่ถูกต้องอยู่อันดับต้นหรือไม่	MRR, nDCG@K	ออฟไลน์
ดัชนี ANN ทำให้เสียคำตอบไปมากไหม	Recall@k ของดัชนี (สมการ 3.20)	ออฟไลน์
หน้าจอหนึ่งซ้ำซากเกินไปหรือไม่	ILD และสัดส่วนแนวซ้ำ	ออฟไลน์
คลังถูกใช้ทั่วถึงหรือกระจุกตัว	Coverage, Gini, Novelty	ออฟไลน์
ผู้ใช้คลิกและดูจริงมากขึ้นหรือไม่	CTR, เวลาที่ใช้, อัตราการเล่นจบ	ออนไลน์ (A/B Test)
ผู้ใช้กลับมาใช้ซ้ำในระยะยาวหรือไม่	อัตราคงอยู่ (Retention), ความหลากหลายสะสม	ออนไลน์ระยะยาว

ลำดับการทำงานที่ปลอดภัยคือคัดกรองด้วยตัวชี้วัดออฟไลน์ก่อน เพราะเร็วและถูก แล้วจึงนำผู้ชนะสองถึงสามแบบไปทดสอบกับผู้ใช้จริง ตัวชี้วัดออฟไลน์ที่ดีขึ้นไม่รับประกันว่าผู้ใช้จะพอใจขึ้น โดยเฉพาะเรื่องความหลากหลายซึ่งผลจะปรากฏในระยะยาวเท่านั้น

3.15 ข้อจำกัดของระบบและการวินิจฉัยปัญหา

3.15.1 สรุปข้อดีอย่างสั้น

ก่อนพูดถึงข้อจำกัด ควรทบทวนว่าเหตุใดแนวทางนี้ยังเป็นพื้นฐานของระบบแนะนำสมัยใหม่ทุกระบบ ข้อดีสี่ข้อคือ **หนึ่ง** แนะนำรายการใหม่ได้ตั้งแต่วันแรกเพราะใช้เพียงเนื้อหาของรายการ (แก้ปัญหา Item Cold Start) **สอง** อธิบายเหตุผลได้ตรงไปตรงมาจากส่วนร่วมรายมิติตามหัวข้อ 3.9.3 **สาม** ไม่ต้องพึ่งข้อมูลผู้ใช้คนอื่น จึงใช้ได้แม้มีผู้ใช้เพียงคนเดียวและรักษาความเป็นส่วนตัวได้ดีกว่า และ **สี่** ควบคุมได้ง่ายเพราะทุกขั้นมีสูตรที่ตรวจสอบด้วยมือได้ ดังที่บทนี้ทำมาตลอด

3.15.2 ข้อจำกัดห้าข้อและหลักฐานจากตัวอย่างในบทนี้

ตารางที่ 3.90 ข้อจำกัดของระบบแนะนำอิงเนื้อหา หลักฐานจาก MovieMini-12 และแนวทางบรรเทา

ข้อจำกัด	อาการ	หลักฐานในบทนี้	แนวทางบรรเทา
ความเฉพาะทางเกินไป (Over-Specialization)	แนะนำแต่สิ่งที่คล้ายของเดิม ผู้ใช้ไม่พบสิ่งใหม่	Top-3 ของ u1 ในปริภูมิความหมายคล้ายกันเองเฉลี่ย 0.3141 (หัวข้อ 3.13.1)	MMR และการควบคุมความหลากหลาย (3.13)
ฟองกรอง (Filter Bubble)	โปรไฟล์แคบลงเรื่อย ๆ เพราะผู้ใช้เห็นแต่สิ่งที่ระบบเลือกให้	โปรไฟล์ของ u1 มีมิติ mind สูงมากจนกลบมิติอื่นทั้งหมด	เติมรายการสำรวจ (Exploration) และวัด Coverage/Novelty (3.14.3)
ผลกระทบเชิงพอร์ต (Portfolio Effect)	แนะนำรายการที่แทบเป็นสิ่งเดียวกัน เช่น ฉบับรีเมก ฉบับพากย์ หรือสินค้าต่างสีเดียวกัน	ไม่ปรากฏใน MovieMini-12 เพราะไม่มีรายการซ้ำ แต่พบเสมอในคลังจริง	รวมกลุ่มรายการที่เกือบเหมือนกันให้เหลือตัวแทนเดียวก่อนจัดอันดับ

ข้อจำกัด	อาการ	หลักฐานในบทนี้	แนวทางบรรเทา
ผู้ใช้ใหม่ยังทำอะไรไม่ได้ (User Cold Start)	ผู้ใช้ที่ยังไม่มีผลป้อนกลับสร้างโปรไฟล์ไม่ได้เลย	โปรไฟล์ทุกแบบในหัวข้อ 3.8 ต้องการรายการที่ชอบอย่างน้อยหนึ่งรายการ	ถามความชอบตอนสมัคร ใช้รายการยอดนิยม หรือใช้ข้อมูลประชากร
คุณภาพผลลัพธ์ถูกจำกัดด้วยคุณภาพเนื้อหา	เนื้อหาสั้น ไม่มีข้อมูล หรือเขียนไม่ดี ทำให้ระบบตัดสินผิด	Mad Max ได้คะแนน 0.0000 ในปริภูมิ TF-IDF ทั้งที่ตรงรสนิยมของ u1 ที่สุด	เพิ่มแหล่งข้อความ ใช้เวกเตอร์ความหมาย หรือใช้ LLM เติมคำบรรยาย (3.12.8)

ข้อจำกัดสองข้อสุดท้ายอธิบายว่าเหตุใดระบบจริงจึงไม่ใช้แนวทางอิงเนื้อหาเพียงอย่างเดียว แต่ใช้ร่วมกับการกรองด้วยความร่วมมือ (Collaborative Filtering) ซึ่งเรียนจากพฤติกรรมของผู้ใช้กลุ่มใหญ่ ทั้งสองแนวทางแก้จุดอ่อนของกันและกันพอดี คือแนวทางอิงเนื้อหาแก้ปัญหารายการใหม่ ส่วนการกรองด้วยความร่วมมือแก้ปัญหาความเฉพาะทางเกินไป

3.15.3 ลำดับการปรับจูนที่ให้ผลคุ้มค่าที่สุด

เมื่อระบบให้ผลไม่ดี นักพัฒนาส่วนใหญ่มักกระโดดไปเปลี่ยนแบบจำลองทันที ซึ่งเป็นลำดับที่ผิดและแพงที่สุด ลำดับด้านล่างเรียงจากสิ่งที่แก้ง่ายและได้ผลมากที่สุดลงไป

ขั้นตอนวิธี เจ็ดขั้นของการปรับจูนระบบแนะนำอิงเนื้อหา

1. **ข้อมูลและการเตรียมข้อความ** ตรวจการตัดคำ คำหยุด การรวมรูปคำ และความยาวของเนื้อหา ขั้นนี้เปลี่ยนผลได้มากที่สุดและแก้ถูกที่สุด (3.3)
2. **การแทนรายการ** ลองเปลี่ยนจาก TF-IDF เป็น BM25 หรือเป็นเวกเตอร์ความหมาย และลองเติมคุณลักษณะที่มนุษย์ติดป้ายไว้ เช่น แนวภาพยนตร์ (3.4–3.7)
3. **โปรไฟล์ผู้ใช้** ลองถ่วงน้ำหนักด้วยคะแนน ลองร็อกคีโย ลองลดทอนตามเวลา และตรวจว่าผู้ใช้มีรสนิยมหลายกลุ่มหรือไม่ (3.8)
4. **ฟังก์ชันจับคู่และการจัดอันดับ** ตรวจกฎตัดสินเสมอ เกณฑ์คะแนนขั้นต่ำ และการกรองรายการที่เคยเห็น (3.9–3.10)
5. **การผสานหลายแหล่ง** เพิ่มแหล่งที่สองแล้วจูนน้ำหนัก w ด้วยผลการประเมิน (3.12)
6. **ความหลากหลาย** เพิ่ม MMR และจูน λ เมื่อความแม่นยำนิ่งแล้ว (3.13)
7. **แบบจำลองเชิงลึก** ฝึกตัวเข้ารหัสคู่หรือ Two-Tower เป็นขั้นสุดท้าย เพราะต้องใช้ข้อมูลปฏิสัมพันธ์จำนวนมากจึงจะชนะวิธีง่าย ๆ ได้ (3.11.4)

หลักฐานสนับสนุนลำดับนี้อยู่ในบทนี้เอง: การกันออกทีละรายการในหัวข้อ 3.14.2 ชี้ว่าจุดอ่อนของระบบตัวอย่างอยู่ที่ **การแทนรายการ** ไม่ใช่ที่โปรไฟล์หรือวิธีจัดอันดับ การไปจูนค่า λ หรือเปลี่ยนแบบจำลองก่อนแก้เรื่องเนื้อหาจึงเสียเวลาเปล่า

3.15.4 ตารางวินิจฉัยอาการ

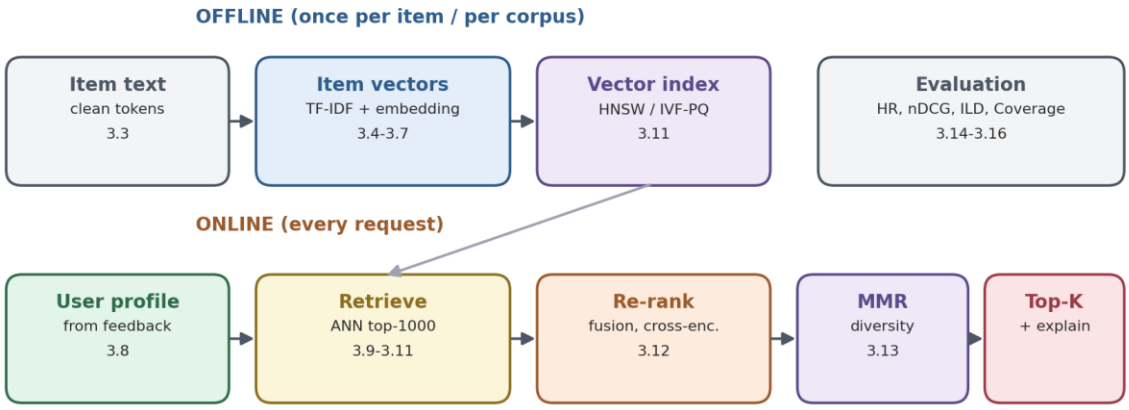
ตารางที่ 3.91 เป็นเครื่องมือที่นำไปใช้กับโครงงานจริงได้ทันที วิธีใช้คือเริ่มจากอาการที่สังเกตเห็น ทำการตรวจสอบในคอลัมน์ที่สามก่อนเสมอ แล้วจึงลงมือแก้

ตารางที่ 3.91 อาการ สาเหตุที่เป็นไปได้ วิธีตรวจสอบ และวิธีแก้

อาการที่สังเกตเห็น	สาเหตุที่พบบ่อย	ตรวจสอบอย่างไร	แก้อย่างไร (หัวข้อ)
คำแนะนำซ้ำซากคล้ายกันทั้งหน้าจอ	ความเฉพาะทางเกินไป	คำนวณ ILD ของ Top-K	ใส่ MMR และลด λ (3.13)
คะแนนของ CANDIDATE ITEM จำนวนมากเป็นศูนย์	เวกเตอร์เบาบางเกินไป เนื้อหาสั้น หรือตัดคำผิด	นับสัดส่วนคู่ที่โคไซน์เป็นศูนย์ และดูจำนวนคำที่เหลือหลังตัดคำหยุด	เพิ่มแหล่งข้อความ ใช้ n-gram หรือย้ายไปเวกเตอร์ความหมาย (3.3–3.6)
แนะนำรายการที่ไม่เกี่ยวข้องเลย	คำพ้องรูป หรือคำเดียวครองคะแนนทั้งหมด	ดูตารางส่วนร่วมรายคำ ถ้าคำเดียวเกิน 50% ของคะแนนคือสัญญาณอันตราย	ใช้เวกเตอร์ตามบริบท หรือใช้ BM25 ที่มีการอิ่มตัว (3.4.6, 3.6)
ผู้ใช้บางคนได้คำแนะนำแปลกทั้งหมด	โปรไฟล์ปนหลายรสนิยม จนเฉลี่ยแล้วไม่เหลือความหมาย	ดูการกระจายของคะแนนที่ผู้ใช้ให้ และความคล้ายระหว่างรายการที่ชอบ	ใช้โปรไฟล์หลายกลุ่มความสนใจ หรือ Item-kNN แบบสูงสุด (3.8.6, 3.9.4)
คำแนะนำไม่เปลี่ยนเลย แม้ผู้ใช้เปลี่ยนความสนใจ	โปรไฟล์ถ่วงน้ำหนักเท่ากันทุกช่วงเวลา	เทียบโปรไฟล์ที่สร้างจากข้อมูลสามเดือนล่าสุดกับทั้งหมด	ใช้การลดทอนตามเวลา (3.8.5)
ผู้ใช้ใหม่ไม่ได้รับคำแนะนำ	ผู้ใช้ใหม่ยังไม่มีผลป้อนกลับ	นับจำนวนผู้ใช้ที่มีผลป้อนกลับน้อยกว่าสองรายการ	ถามความชอบตอนสมัคร หรือใช้รายการยอดนิยมชั่วคราว (3.15.2)
รายการยอดนิยมครองทุกอันดับของทุกคน	น้ำหนักความนิยมในแบบจำลองจัดอันดับสูงเกินไป	วัด Coverage และ Novelty	ลดน้ำหนักความนิยม หรือเพิ่มพจน์ส่งเสริมความแปลกใหม่ (3.12.7, 3.14.3)
ผลออฟไลน์ดีมากแต่ออนไลน์แย่	ข้อมูลรั่ว หรือแบ่งข้อมูลแบบสุ่มแทนการแบ่งตามเวลา	ตรวจว่าคุณลักษณะทุกตัวมีอยู่จริง ณ เวลาที่ทำนาย และเปลี่ยนไปแบ่งตามเวลา	แก้การแบ่งข้อมูลและตัดคุณลักษณะที่รั่ว (3.11.4, 3.14.1)
ระบบตอบช้าเมื่อคลังโตขึ้น	ยังให้คะแนนครบทุกรายการ	วัดเวลาต่อคำขอเทียบกับงบ 50 มิลลิวินาที	สร้างดัชนี ANN แล้วจัดอันดับซ้ำเฉพาะ CANDIDATE ITEM (3.11)
ตั้งเกณฑ์คะแนนไม่ได้ ตีความคะแนนไม่ออก	คะแนนยังไม่ได้ปรับเทียบ	พล็อตคะแนนเทียบอัตราการชอบจริงเป็นช่วง ๆ	ปรับเทียบด้วยวิธีของแพลตต์หรือไอโซโทนิก (3.12.6)

3.16 สรุปกระบวนการ

บทนี้เดินจากข้อความดิบไปจนถึงรายการแนะนำที่ประเมินผลได้ครบหนึ่งรอบ ภาพที่ 3.9 สรุปทุกขั้นพร้อมกัน โดยแยกชัดว่าขั้นใดทำล่วงหน้าและขั้นใดทำสดตอนผู้ใช้เปิดหน้าจอ



ภาพที่ 3.9 กระบวนการทั้งระบบของระบบแนะนำอิงเนื้อหาจากข้อความ
ส่วนบนทำล่วงหน้าครั้งเดียวต่อรายการหรือต่อคลัง ส่วนล่างทำทุกครั้งที่มีคำขอ

ตารางที่ 3.92 สรุปทุกขั้นของบทที่ 3: ทำอะไร ด้วยสูตรใด และเห็นอะไรจาก MovieMini-12

ขั้น	หัวข้อ	สูตรหลัก	สิ่งที่ตัวอย่างในบทนี้สอน
1 เตรียมข้อความ	3.3	ทำให้เป็นมาตรฐาน ⟶ ตัดคำ ⟶ ตัดคำหยุด	263 โทเคนเหลือคำศัพท์ 203 คำ การตัดคำผิดทำลายทุกขั้นถัดไป
2ก เวกเตอร์เชิงคำ	3.4	$w_{i,t} = \text{tf} \times \text{idf}$ แล้วปรับ L2	42 จาก 66 คู่มีคะแนนเป็นศูนย์ คำร่วมโดยบังเอิญหลอกระบบได้
2ข เวกเตอร์ความหมายแบบสถิต	3.5	ค่าเฉลี่ยถ่วง IDF ของเวกเตอร์คำ	จับคำพ้องความหมายได้ (assassin ↔ killers) แต่คำพ้องรูปยังปนกัน
2ค เวกเตอร์ตามบริบท	3.6	เมทริกซ์โทเคน ⟶ รวมค่า ⟶ L2	dream ใน Inception ต่างจาก dream ใน La La Land
3 โปรไฟล์ผู้ใช้	3.8	$p_u = \text{L2}\left(\frac{1}{\lvert I_u^+ \rvert}\sum v_i\right)$ และแบบอื่นอีกห้าแบบ	โปรไฟล์ต้องอยู่ในปริภูมิเดียวกับรายการเสมอ
4 คะแนนจับคู่	3.9	$s(u,i) = p_u \cdot v_i$	ปริภูมิต่างกันให้ CANDIDATE ITEM อันดับหนึ่งคนละเรื่อง
5 จัดอันดับ Top-K	3.10	กรอง ⟶ เรียง ⟶ ตัดสินเสมอ ⟶ เกณฑ์ θ	คะแนนศูนย์ไม่เท่ากับคะแนนต่ำ
6 ค้นคืนในคลังใหญ่	3.11	ดัชนี HNSW หรือ IVF-PQ	10 ล้านรายการ: 4 วินาที เหลือระดับมิลลิวินาที
7 ผสานและจัดอันดับซ้ำ	3.12	$w\tilde{s}_1 + (1-w)\tilde{s}_2$ แล้วให้ตัวเข้ารหัสไขว้จัดซ้ำ	การผสานลงโทษรายการที่ดีเพียงแหล่งเดียว
8 ความหลากหลาย	3.13	$\lambda\,\text{rel} - (1-\lambda)\text{maxsim}$	$\lambda = 0.7$ เสียความเกี่ยวข้อง 1% แต่ได้ ILD เพิ่ม 16%
9 ประเมินผล	3.14	HR@K · MRR · nDCG@K · ILD · Coverage · Novelty	ปริภูมิที่ง่ายที่สุดชนะในคลังเล็ก ต้องวัดก่อนเชื่อ

ขั้น	หัวข้อ	สูตรหลัก	สิ่งที่ตัวอย่างในบทนี้สอน
10 วินิจฉัยและปรับจูน	3.15	ตารางอาการ–สาเหตุ–วิธีแก้	แก้ข้อมูลและการแทนรายการก่อนเปลี่ยนแบบจำลองเสมอ

3.16.1 สิบสองข้อสรุปที่ควรจำจากบทนี้

1. ระบบแนะนำอิงเนื้อหาตัดสินจากสิ่งที่รายการ **เป็น** จึงแนะนำรายการใหม่ได้ตั้งแต่วันแรกและอธิบายเหตุผลได้
2. ทุกระบบในบทนี้คือสถาปัตยกรรมสามส่วนเดิมเสมอ คือ ตัววิเคราะห์เนื้อหา ตัวเรียนรู้โปรไฟล์ และส่วนกรองและจัดอันดับ
3. การเตรียมข้อความคือขั้นที่ถูกมองข้ามบ่อยที่สุดแต่เปลี่ยนผลลัพธ์มากที่สุด
4. TF-IDF ให้น้ำหนักคำที่ปรากฏบ่อยในเอกสารหนึ่งแต่หายากในคลัง ส่วน BM25 เพิ่มการอิ่มตัวและการปรับความยาวเอกสาร
5. เวกเตอร์ความหมายแก้ปัญหาคำพ้องความหมาย และเวกเตอร์ตามบริบทแก้ปัญหาคำพ้องรูปเพิ่มอีกชั้นหนึ่ง
6. โปรไฟล์ผู้ใช้ทุกแบบมีกฎข้อเดียวร่วมกัน คือต้องอยู่ในปริภูมิเดียวกับเวกเตอร์ของรายการ
7. การเลือกวิธีแทนรายการเปลี่ยนคำแนะนำได้มากกว่าการเลือกฟังก์ชันจับคู่หรือวิธีจัดอันดับ
8. คะแนนคือผลรวมของส่วนร่วมรายมิติ ซึ่งให้คำอธิบายแก่ผู้ใช้และให้เครื่องมือวินิจฉัยแก่ผู้พัฒนาไปพร้อมกัน
9. ตัวเข้ารหัสคู่ใช้ค้นคืน ตัวเข้ารหัสไขว้ใช้จัดอันดับซ้ำ และ Two-Tower คือตัวเข้ารหัสคู่ในบริบทของระบบผู้แนะนำ
10. ดัชนี ANN แลกความถูกต้องเล็กน้อยกับความเร็วมหาศาล และคุณภาพของขั้นค้นคืนคือเพดานของทั้งระบบ
11. การผสานคะแนนต้องปรับมาตรก่อนเสมอ และ MMR แก้ข้อจำกัดสำคัญที่สุดของแนวทางนี้คือความเฉพาะทางเกินไป
12. ต้องรายงานความแม่นยำ ความหลากหลาย และความครอบคลุมพร้อมกันเสมอ เพราะระบบที่ดูแม่นอาจไร้ประโยชน์ในทางธุรกิจ

เส้นทางทั้งหมดของบทนี้จึงสรุปได้ว่า

ข้อความ ⟶ เวกเตอร์ ⟶ โปรไฟล์ ⟶ การจับคู่ ⟶ การจัดอันดับ ⟶ การค้นคืน ⟶ การจัดอันดับซ้ำ ⟶ ความหลากหลาย ⟶ การประเมินผล

เมื่อเข้าใจครบทุกขั้นตอนเป็นพื้นฐานสำหรับบทต่อไปที่เป็นระบบแนะนำจากรูปภาพ เสียง หรือวิดีโอได้ทันที เพราะเปลี่ยนเพียงขั้นที่ 2 คือวิธีแปลงเนื้อหาให้เป็นเวกเตอร์ ส่วนขั้นอื่นทั้งหมดยังเหมือนเดิมทุกประการ